MIDAS GUI — Development History

A commit-by-commit record of what changed and when, so you can (a) find **when** a particular behaviour/feature entered the code and (b) know **what to roll back** to undo a specific effect.

> Keep this file updated when you add commits: append a new entry at the bottom of the chronological log and add a row to the quick-reference index. Regenerate the PDF the same way as the other docs (pandoc + headless Chrome) if you keep one.

How to use this document

- **Find when a change landed:** search the change → commit index below, or `git log --oneline -- <path>` for a specific file, or `git log -S"<code string>"` to find the commit that introduced/removed a string.
- **See a commit's exact diff:** `git show <hash>` (whole commit) or `git show <hash> -- <path>` (one file).
- **Undo one commit cleanly (keeps later history):** `git revert <hash>`. For a range: `git revert <oldest>^..<newest>`.
- **Undo just one file back to a commit:** `git checkout <hash> -- <path>` then commit. To see the file *as of* a commit without changing anything: `git show <hash>:<path>`.
- **Hard reset to a point (destructive — local only, never on pushed history):** `git reset --hard <hash>`. Prefer `git revert` on a shared branch.
- **Dependencies matter.** Many later commits build on earlier ones (e.g. the unified data-loader, the config overlay). The "Roll back" note on each entry flags when reverting will disturb dependents.

Branch: `main`. All commits are authored by Dishant Beniwal (AI pair-authored). Dates are commit dates (YYYY-MM-DD).

Change → commit quick-reference index

Packaging / build / launch

Change	Commit
Initial v1.0.0 release (9-tab app, all backends)	`f19ee8c`
setuptools.build_meta backend (broader pip compat)	`3cd1f36`
`launch.py` standalone launcher	`2e9f419`
Startup crash diagnostics + robust launcher (Windows)	`916ece2`
MIDAS-style packaging (LICENSE, pyproject, release.sh, smoke tests)	`e862135`
Ship test_data sample data in the repo (fresh clone works out of the box)	`7e157e0`
Drop midas_suite from env (unblock conda env create; backends via -e .)	`72a1770`
Pin environment.yml to verified NumPy-1 set + README recreate steps	`68313f8`
Upgrade midas-hkls/midas-calibrate-v2; midas-pdf switched vendored→PyPI	`acb43c1`

Change	Commit
PyQt5 moved pip→conda-forge (fixes beamline silent startup hang)	`97ae1ea`
Add pip `requirements.txt` mirroring `environment.yml` + README pip path	`683d150`
Reorganize `test_data/` into per-dataset subfolders (`gui_synthetic/`, `s17bm/`, `trr_s25ide/`, `trr_s7id/`)	`c8d98f1`
Fix stale `DEFAULT_*` test-data paths after the `gui_synthetic/` reorg	`3b785c9`
Upgrade all 8 MIDAS backend packages to latest PyPI releases; add midas-params, pin zarr/numcodecs	`a74b7d6`

Data Viewer (Tab 0)

Change	Commit
Radial-integration plot, beam-centre ring picking, zoom persistence	`acf0866`
Zoom/pan preserved across frames (autoRange off)	`d9e527e`
Intensity-range mask, calibration-file loading, click-to-draw ring	`d50b588`
Compact 3-per-row lattice + cubic quick-set; mask default max(99.99pct,1e5)	`41f28f5`
Projection Skip-frames field; compact geometry/intensity fields; Lsd separators	`108ea7d`
Intensity-statistics panel + histogram (bottom of loader)	`2b7a695`
Tilt/distortion-aware radial integration when a calibration is loaded	`df544d2`
Data Viewer Top-N brightest pixels + mask-aware stats + full-geometry receive	`bc86d74`
Calibrate per-coefficient distortion, frame averaging, seed feedback	`b0a5cc3`
Batch read-only “Calibration values” card	`b6f1681`
Pump Probe publication plotting, delay colour-bar, default detector mask	`49623ae`
Gitignore local context, internal docs, large sample sets	`b251ca8`
Live Data card: in-tab EPICS PVA stream via pvapy, reload button	`1769f69`
Live viewer: manual colormap/level changes now survive new frames	`234ded9`
Wider image/radial-plot splitter handle	`69f470c`
Radial profile: Y-min defaults to 0.9x data min, manual Y-range sticks	`96f8add`
Simulate rings is now a live toggle (auto-recomputes on param change)	`ecf6d01`
Live PV field becomes a device dropdown (Preferences ▸ Devices)	`aa82cb6`
Fix image-view autorange drift; bound pan/zoom to image	`c156c62`

Change	Commit
Radial profile plot: bound pan/zoom to the data extent	`cde4bb2`
Ring simulation ty/tz tilt fields; configurable spin-box step sizes	`5b8e3b3`
Tilt-aware profile w/o calibration file; ring 2θ-cutoff/thickness/live-button fixes; save-calibration buttons; radial-plot X-floor + zoom persistence	`e14c1ea`
Native-menu-backed A/M manual axis limits (radial plot + histogram); Top-N “I >” intensity floor	`3c15ae7`
Multi-material ring simulation (per-row checkbox/color/name, Material dialog)	`0e9ed21`
Load-calibration card moved below Simulate button; loads now sync ty/tz too; Intensity-range title bar is the on/off checkbox	`05ce224`
Live Data “Use Buffer” last-N-frames ring buffer (Projection/stack analysis on live data)	`911f8ac`
Fix: lock live ring buffer against GUI/worker-thread race	`a88ba1f`
Fix: refresh-timer starvation during fast live streaming	`2cfd9cd`
Fix: cap live buffer to 100 frames (memory bound)	`839770d`
Fix: “All frames” stats computed off the GUI thread	`08392c6`
Fix: debounce intensity-mask pixel≤/pixel> spinbox edits	`57ced2f`
Fix: warn when composite mask silently drops a shape-mismatched source	`81b8ea8`
Sim Detector: hardware-free fake PVA stream in the Live Data dropdown	`3d96cb1`
Auto-detect pixel size (filename tag) + wavelength (HDF5 beam energy) on file load, 1-ID-E/20-ID-D/20-ID-E only	`9e4b5c0`
Image viewer: persist manual color-scale window (incl. histogram zoom) across live frames	`3b12fbf`
Fix: “Use Buffer” state now carries into Start instead of resetting	`f3a1b91`
Unrestrict λ/Lsd/pixel-size ranges; tighten decimals; Rings/Labels/thickness on one row	`2525277`
“?” help button explaining radial-integration (R-bin) calculation	`b7a1518`
Exclude-range controls moved into radial-plot toolbar; int-safe pixel bounds	`de15d57`
Pixel-size field allows a second decimal place (near-field detectors)	`1d45c40`
Box/Circle/Line ROI tool with live floating stats popups	`ecfbf36`
ROI tool drops Circle (Box/Line only); Clear ROIs resets numbering; Pick Clear also removes BC marker	`d5654c1`
Line ROI drawn as single arrow shape (no separate arrowhead item)	`bb78c9a`

Change	Commit
Box-ROI popup's zoomed crop image resizes with the popup window	`786c94c`
Projection N-frames cap; λ menu gains energy-to-wavelength entry	`468b417`
B-PILOT bridge: local-socket server auto-starts Live Data on scan dispatch	`c336778`
ROI popups always-on-top + minimize-to-ribbon; Project-stack button green highlight	`6c79f13`
ROI/histogram axis label font size bumped 9pt -> 12pt (readability)	`1181b58`
Transforms: Flip Y/Flip Z/Transpose checkboxes (MIDAS ImTransOpt); saved/loaded calibration files round-trip it	`068bd0d`
Image origin flipped to bottom-left `(0,0)` (MIDAS convention); ROI box-corner annotations now label the bottom-left corner	`246dba7`
Hydra-mode ribbon scaffold (Single detector/Hydra); ring-sim/calibration/radial logic extracted into DetectorGeometryCard	`3d2d99d`
Hydra geometry-compositing engine (`hydra.py`) + sibling auto-discovery + tests	`dd44f38`
Hydra loader/toolbar/per-panel calibration UI + composite wired into the tab	`1cc4dab`
Hydra multi-curve radial profile plot (4 panels + toggleable composite curve)	`8707372`
Hydra Phase 6 pass: shared λ/2θ/px fields, dark/bright/background correction, composite orientation fix (CCW + vertical mirror), stale radial geometry fix, zero-excluded vmin%	`0aa5feb`
Hydra: Pick BC/Pick Ring point leakage across panels fixed (shared viewer's pick state now cleared on panel switch)	`eadb14d`
Transforms checkboxes extracted into their own boxed Transforms card (was an inline label inside Ring simulation); Hydra ge1-4 gain a per-panel Rotate field; Hydra gains a per-panel Projection card (Max/Sum/Average); fixed Flip Y/Flip Z/Transpose not refreshing the Hydra image; Hydra radial plot pan/zoom bounded to data extent; Material dialog Preset fills Name field	`93dafa2`
Lab-frame axes overlay (APS/MIDAS coordinate compass), ported from midas-gui-swaxs	`87d9df7`
Fix ImTransOpt propagation: Hydra radial-profile geometry now applies the flip via the MIDAS backend instead of dropping it	`2358ae4`
Single-detector mode gains an Eta vs R Cake tab next to Radial Profile	`943a91d`

Mask Builder (Tab 1)

Change	Commit
Unbounded σ fields; independent spatial/temporal auto-mask; Viewer↔Calibrate geometry hand-off	`10df828`
Temporal-constancy accepts a 3-D HDF5 (time,y,x) stack	`2385f75`
Image/Stack browse gains “Import from…” (other tabs’ loaded path/buffer)	`6c79f13`
Stack browse menu gains “Files (multi-select)…” for hand-picked temporal stacks	`cc63d5a`
“5 · Post-processing” card: configurable bad-pixel dilation (px)	`429d41a`
Dilation switched from 4-connected to 8-neighbor full-block growth	`188ea77`
Transforms: Flip Y/Flip Z/Transpose checkboxes (MIDAS ImTransOpt), applied to the single image and the stack source alike; synced from an incoming Calibrate result	`068bd0d`
Image origin flipped to bottom-left `(0,0)` (MIDAS convention)	`246dba7`
Fix ImTransOpt propagation: azimuthal/learnable mask computations now honor the active flip via the MIDAS backend where possible	`2358ae4`
A mask logged to a Project attempt is embedded only when it includes something hand-drawn/computed in Mask Builder; a file/folder-only mask is referenced by path+hash instead	`943a91d`

Calibrate (Tab 2) / Refinement (Tab 3)

Change	Commit
Abort buttons; dark/bright/background field correction; auto-load on browse	`7d010d7`
Calibration input accepts paramstest/poni/json	`41f28f5`
Load-calibration-file (geometry + λ) helpers	`b381d8d`
Multi-column paramstest results readout + Send-to-Data-Viewer button	`455ca4e`
Calibrate Results all-text (drop distortion table; named distortion, bigger font)	`3a6ecb9`
Fix calibrate() initial_BC_y TypeError (kwargs filter) + pin scikit-image	`b1642fa`
Pick-Ring fit overlay recolored blue (was same amber as simulated rings)	`3eb4a44`
“Corrected” rings drawn synchronously from fitted ty/tz (drop background worker)	`5b8e3b3`
One-shot honors partial distortion-coefficient selection; live dark/bright/bg preview	`cc6e90e`
Existing Transforms checkboxes now round-trip through saved/loaded paramstest.txt + calibration.json (previously in-memory only)	`068bd0d`

Change	Commit
Transforms checkboxes now live-update the preview and drive the actual calibration run (all pipeline modes); Send-to-Data-Viewer carries Transforms; Average frames drops the skip/stride control	`73a7a2a`
Image origin flipped to bottom-left `(0,0)` (MIDAS convention)	`246dba7`
Calibrate tab split into Single-detector/Hydra modes: per-panel fit of all 4 GE panels from one calibrant dataset, Sequential/Parallel run modes, geometry hand-off with the Data Viewer's Hydra page	`93dafa2`
Hydra seed-mode (manual seed/feed-back) linked across all 4 panels; new Eta vs R Cake tab (Single-detector + Hydra)	`162fef1`
Auto-detect pixel size + wavelength on file load (1-ID-E/20-ID-D/20-ID-E only; Single-detector + Hydra)	`9e4b5c0`
Radial Profile/Eta vs R Cake pan/zoom bounded to data extent; Cake right-drag zooms η (Y) only	`08c917f`
Eta vs R Cake auto-level: vmin% default 1→30, percentile calc excludes exact-zero bins	`d0cd6ce`
Fix ImTransOpt propagation: calibration/refinement now pass the raw image + `im_trans` straight through, letting the MIDAS backend apply the flip instead of pre-flipping in Python	`2358ae4`
Hydra Overall Radial Profile becomes a toggle button (was a checkbox); Eta vs R Cake tab gains GE1-4 checkboxes + an Overall button showing the summed cake across all 4 panels; calibration attempts embed their computed Radial Profile/Eta vs R Cake so Populate needs no recompute	`943a91d`
Fix Flip Z ignored when Multi-panel detector is checked: seed and solve were running in two different image frames	`ccce056`
Feed Multi-panel detector's per-panel shifts + grid geometry to downstream integration (Results-tab preview, Batch Integrate, and saved paramstest.txt)	`101558a`
Fix Multi-panel detector pipelines never actually refining panel shifts (spec built without panel parameters); Save calibration.json/paramstest.txt rewrite a co-located sidecar; Project attempts embed + rematerialize panel shifts on reopen	`c67ad1b`

Batch Integrate (Tab 4)

Change	Commit
Live folder MONITOR + per-file legend	`1149afc`
Clear results + inline per-file labels & plot controls	`524f5f1`
Stacked profiles: publication themes, point+line, symbol size, x-units, grid	`d135c80`
Waterfall R/2θ/Q x-axis selector	`2385f75`

Change	Commit
Batch Integrate tab split into Single-detector/Hydra modes: per-panel integration of all 4 GE panels with independently fitted geometry, automatic Calibrate→Batch hand-off, per-panel masks, Sequential/Parallel run modes	`6b961d4`
Waterfall/Stacked profiles pan/zoom bounded to data extent; Waterfall gains a color-scale histogram sidebar	`08c917f`
Waterfall auto-level: vmin% default 1→30, percentile calc excludes exact-zero pixels	`d0cd6ce`
Fix ImTransOpt propagation: geometry's flip/transpose now applied by the MIDAS backend (`spec.TransOpt`), not silently dropped	`2358ae4`
Browse… parity: Multiple files + filestem-filtered (live folder+prefix) sources, MONITOR re-scans a filestem filter	`af8066f`
Cosmetic overhaul: Drift hidden, "View calibration" popup, checkbox output formats, green Save/red Abort, Sequential/Batch-Parallel workers (single-detector + Hydra)	`a27790a`
Output format + Run mode become popups (button/tooltip, not permanent widgets); fix View-calibration popup rebuilding stale on first open	`e6f2e50`
Fix Batch-Parallel frame ordering in live waterfall/stacked-profile view	`0332683`

PDF Analysis (Tab 6)

Change	Commit
Polyatomic midas_pdf reduction pipeline + vendored midas_pdf + calibration loading	`b381d8d`
Defaults to real I(Q) test data	`1149afc`
midas_pdf switched from vendored copy to the real PyPI package	`acb43c1`
Rebuilt for full Stage 2-3 workflow (absorption/efficiency/normalization/multiple-scattering/fluorescence/structure-fit/Δ-PDF), new 4-tab layout	`23cd55e`

Pump Probe (TR-XRD) tab

Change	Commit
New tab: TRR pooling + MIDAS-engine integration + ΔI(q,delay) + 4 pyqtgraph views + publication-quality plots + default MIDAS calibration	`590b410`
Fix ImTransOpt propagation: worker now passes the active calibration's `im_trans` through to the MIDAS backend	`2358ae4`

Cross-cutting UI / infrastructure

Change	Commit
Linux visible files/folders style fix	`fc754d0`
Pixel-weighted azimuthal mean; readable QMenu; no-scroll combos	`41f28f5`
Unified Data-Loader panel + draggable 3-panel splitter (tabs 0/2/3/4)	`aa9395e`
Clickable λ label with K-edge foil menu	`b628446`
Clickable px label with detector pixel-size menu	`3248638`
Per-user JSON configuration system + Preferences dialog	`d30be2c`
Modular tab visibility (show/hide optional tabs)	`590b410`
Multi-profile config (Preferences ▸ Profile row)	`5b8e3b3`
File ▸ Save/Load GUI State (all 10 tabs, with mask/calibration sidecars)	`5b8e3b3`
User-adjustable interface scaling (HiDPI / 4K, QT_SCALE_FACTOR)	`d5fafd8`
Default optional tabs reduced (Corrections/PDF/Texture/Export hidden)	`7e157e0`
Lsd displayed in mm (calculations & calibration files stay in μm)	`c4e1c12`
Fix “Populating font family aliases” warning (real fixed-width fonts)	`d6df1f8`
Fix fresh-env launch crash — colormap resolution without matplotlib	`6332b3d`
Fix native Bus-error crash on Run Calibration — cap BLAS/OMP thread pools	`0b4326d`
Preferences ▸ Devices tab; Live PV field becomes a device dropdown	`aa82cb6`
Widen non-data-derived numeric field caps (tilts to ±180°, iteration/geometry/hyperparameter fields to effectively unbounded) across Data Viewer/Calibrate/Corrections/Mask/Refine/Batch	`53f8f19`
Cross-tab DataSourceRegistry + “Import from…” menus (Data/Dark/Bright/Background) + buffer-save button	`6c79f13`
File ▸ Project: opt-in FAIR provenance record (HDF5), auto-logs Calibrate/Batch Integrate runs	`e8dea6b`
Header Profile combo (instant switching); Hydra gated to 1-ID-E; fix stale device/Calibrant dropdowns + pixel/K-edge popups on profile change	`81056e4`
File ▸ Workspace (renamed from GUI State) + Recent Projects/Workspaces, unsaved-changes indicator, autosave/crash-recovery, Project History viewer	`6b1564b`
Workspace/Project record + restore the active beamline Profile on load/open; Eta vs R Cake plots’ right-click-	`943a91d`

Change	Commit
drag zooms only the axis actually dragged (was always η-only)	
Data Loader Browse… popup (Single/Multiple files/Full folder/Files sharing a name stem) + Hydra gains real cross-tab Import from…	`ac13797`
Browse… popup polish: folder/file-path display (not a count), project-root default dir, wider name column	`a54f796`
Merge Workspace (JSON) and FAIR Project (HDF5) into one `.h5` Project file; File menu collapses to Save/Save As/Open Project	`ae3b665`
Project schema redesign (clean cutover, no back-compat): `/gui_workspace` per-tab groups + `/analysis/{mask,calibrate,integrate}` FAIR history; Mask Builder gets `/analysis/mask` provenance + "Log to Project"; unified Open Project dialog (browse + live checkbox-tree preview) replaces the old Yes/No + separate attempt-picker flow	`21faaf8`

Stability / performance / consistency (review-driven, phases 1–3)

Change	Commit
Clean worker shutdown, drop QThread.terminate(), guard stdout, offload projection	`399f3d7`
Waterfall O(N^2)→O(N), frame-slider debounce, radial-grid + stats caching	`4462ee1`
Dedupe distortion map + paramstest writer, align calibrant lattices, texture colormap	`8846619`

Documentation

Change	Commit
README not-on-PyPI clone/conda instructions	`cd05ced`
gui_documentation.md added	`2e5f7c9`
gui_documentation.pdf added	`75c135c`
README/environment install via `pip install midas_suite`	`aceb40f`
implementation_details.md/.pdf added	`524f5f1`
config_gui.md + config.example.json added	`d30be2c`
development_history.md/.pdf added	`6d6f3fb`
Commit `.context/` (was gitignored, contradicting standing policy)	`778a2f3`
README troubleshooting: corrupted pip cache import errors	`08fe8f6`

Chronological commit log (oldest → newest)

`f19ee8c` — Initial release: midas_gui v1.0.0 (2026-06-30)

Effect: First version. Nine-tab PyQt5 GUI over `midas_calibrate_v2` / `midas_integrate_v2`: mask building, auto-calibration, refinement, batch integration + waterfall, corrections, PDF, texture, export. Entry points `midas-gui` / `python -m midas_gui`. **Files:** entire `midas_gui/` package + `pyproject.toml`, `README`, `environment.yml`. **Roll back:** this is the root — everything depends on it; don't revert.

`3cd1f36` — setuptools.build_meta backend (2026-06-30)

Effect: Build backend switched for setuptools≥61 compatibility. **Files:** `pyproject.toml`. **Roll back:** `git revert 3cd1f36` (only affects builds).

`2e9f419` — add launch.py standalone launcher (2026-06-30)

Effect: `python /path/to/launch.py` runs the app from anywhere. **Files:** `launch.py`. **Roll back:** safe to revert; later hardened by `916ece2`.

`fc754d0` — Linux style fix for invisible files/folders (2026-06-30)

Effect: Stylesheet fix so file/folder text is visible on Linux. **Files:** `midas_gui/style.py`. **Roll back:** `git revert fc754d0` (cosmetic).

`acf0866` — Data Viewer: radial plot, ring picking, zoom persistence (2026-06-30)

Effect: Added the radial-integration plot, beam-centre ring picking, and zoom-persistence across a stack in the Data Viewer; widened left panels. **Files:** `tab_view.py` (+1-line width bumps in other tabs). **Roll back:** revert to remove the Data Viewer radial plot/ring picking.

`d9e527e` — preserve zoom/pan across frames (2026-06-30)

Effect: Disabled pyqtgraph autoRange on redisplay so zoom/pan survive frame changes. **Files:** `widgets.py`. **Roll back:** revert if you *want* auto-refit per frame.

`d50b588` — Data Viewer: intensity mask, calib loading, click-to-draw ring (2026-07-01)

Effect: 99.9-pct intensity-range mask default; load calibration (json/poni/ paramstest); click-to-draw ring; test-data defaults point at local `test_data/`. **Files:** `tab_view.py`, `widgets.py`, `helpers.py`, `constants.py`, `tab_mask.py`. **Roll back:** revert to drop these Data Viewer features (note `helpers.py` geometry parsing added here is reused later — see `b381d8d`).

`916ece2` — startup crash diagnostics + robust launcher (2026-07-01)

Effect: Logs uncaught exceptions/native faults to `~/midas_gui_error.log`, prevents PyQt slot-exception aborts, isolates failing tabs, hardens the launcher. **Files:** `app.py`, `launch.py`. **Roll back:** revert only if you want raw exceptions to propagate; not recommended.

`7d010d7` — Abort buttons + dark/bright/background + auto-load (2026-07-01)

Effect: Abort on calibrate/batch; dark/bright/background field correction (file/folder/HDF5, index-range averaging, divide/subtract); browse auto-loads, "Load" buttons removed; compact FieldSelector. **Files:** `tab_batch/calibrate/corrections/mask/pdf/refine/texture/view.py`, `widgets.py`, `workers.py`,

`helpers.py`. **Roll back:** wide-reaching; reverting removes field correction + auto-load across tabs. The Abort behaviour here was later re-worked by `399f3d7` (no `terminate()`).

`e862135` — MIDAS-style packaging (2026-07-01)

Effect: BSD-3 LICENSE, MIDAS-convention `pyproject.toml`, `release.sh` + `RELEASING.md`, headless `tests/` smoke suite. **Files:** `LICENSE`, `README.md`, `RELEASING.md`, `pyproject.toml`, `release.sh`, `tests/`. **Roll back:** revert affects packaging/tests only.

`41f28f5` — compact lattice, pixel-weighted azimuthal mean, readable menus (2026-07-06)

Effect: Data Viewer 3-per-row lattice + cubic quick-set; intensity-mask default max(99.99pct,1e5); **pixel-weighted azimuthal mean** (selectable vs legacy η-bin mean) fixing off-detector-BC distortion; batch calibration accepts paramstest/poni/ json; readable right-click menus; no-scroll combos. **Files:** `helpers.py`, `workers.py`, `tab_view/batch/calibrate/pdf/refine/texture.py`, `widgets.py`, `style.py`. **Roll back:** to undo the azimuthal-averaging change specifically, prefer editing the `weighted` default rather than reverting the whole commit (it bundles UI changes).

`cd05ced` — docs: not-on-PyPI install instructions (2026-07-06)

Effect: README clone+conda+run instructions; relative env self-install. **Files:** `README.md`, `environment.yml`. **Roll back:** docs only.

`2e5f7c9` — docs: add gui_documentation.md (2026-07-06)

Effect: First committed user guide. **Files:** `documentation/gui_documentation.md`. **Roll back:** docs only.

`75c135c` — docs: add gui_documentation.pdf (2026-07-06)

Effect: PDF build of the guide. **Files:** `documentation/gui_documentation.pdf`. **Roll back:** docs only.

`aceb40f` — docs: install via `pip install midas_suite` (2026-07-06)

Effect: Corrected backend-install instructions. **Files:** `README.md`, `environment.yml`. **Roll back:** docs only.

`aa9395e` — Unified Data-Loader panel + 3-panel splitter (tabs 0/2/3/4) (2026-07-07)

Effect: Shared `DataLoaderPanel` (Data/Dark/Bright/Background/Mask, per-mode frame controls) + `MaskSelector`; corrections applied on tabs 0/2/3/4; each of those tabs restructured into a draggable `[Data Loader | Parameters | Display]` splitter. **Files:** `widgets.py` (+442), `tab_view/calibrate/batch/refine.py` (large rewrites), `app.py`, `helpers.py`, `workers.py`, docs. **Roll back: high impact** — this is the base for the Batch/Calibrate/Refine and later Pump Probe layouts. Reverting breaks the 3-panel tabs and the Pump Probe tab's left panel. Avoid a blanket revert; fix forward instead.

`b381d8d` — Tab 6 polyatomic midas_pdf pipeline + calibration loading (2026-07-08)

Effect: PDF tab rewritten to the total-scattering pipeline; **vendored `midas_pdf`** under `midas_gui/_vendor/` + `pdf_backend.py`; `helpers` `geometry_fields_from_file` / `result_ns_from_geometry_file` (paramstest/json/poni) used by Calibrate and PDF "Load calibration file…". **Files:** `_vendor/midas_pdf/**` (new, large), `pdf_backend.py`, `tab_pdf.py`, `workers.py`, `helpers.py`, `tab_calibrate.py`, `constants.py`, `pyproject.toml`, docs. **Roll back:** reverting removes the PDF pipeline **and** the shared geometry-file loaders that later tabs (incl. Pump Probe via `spec_from_geometry_file`) rely on — revert with care.

`1149afc` — Batch live MONITOR + legend; PDF real-data default (2026-07-08)

Effect: MONITOR button (stream mode) watches a folder and integrates new frames via `FolderMonitorWorker`, reusing a cached detector map (factored `build_integration_context()` + `write_profile()`, shared with `BatchWorker`); per-file legend; PDF tab defaults to real I(Q). **Files:** `tab_batch.py`, `workers.py`, `widgets.py`, `constants.py`, `tab_pdf.py`, docs. **Roll back:** to remove MONITOR only, revert this commit — but `build_integration_context()` introduced here is **reused by the Pump Probe worker** (`590b410`), so a full revert would break Pump Probe integration.

`524f5f1` — Batch Clear results + inline labels; implementation-details doc (2026-07-09)

Effect: "Clear results" button; inline per-file curve labels + line-width/font toolbar on stacked profiles; new `implementation_details.md/.pdf`. **Files:** `tab_batch.py`, `widgets.py`, `documentation/implementation_details.*`. **Roll back:** safe, localized to Batch + the new doc.

`10df828` — Mask unbounded σ + split auto-mask; Viewer↔Calibrate geometry (2026-07-09)

Effect: Removed σ upper limits; split statistical auto-mask into independent Spatial-outlier / Temporal-constancy checkboxes; geometry hand-off `DataViewer.pushGeometry` / `Calibrate.pullGeometry` → `apply_geometry`. **Files:** `tab_mask.py`, `tab_view.py`, `tab_calibrate.py`, `app.py`, `workers.py`, docs. **Roll back:** revert to restore bounded σ / combined auto-mask / no geometry hand-off.

`d135c80` — Batch stacked-profiles: themes, point+line, x-units, grid (2026-07-09)

Effect: `StackedProfileViewer` publication/dark themes, point+line, symbol-size, R/2θ/Q x-unit selector, grid toggle. (This viewer's styling is the template the Pump Probe plots later mirror.) **Files:** `widgets.py` (+228), `tab_batch.py`, docs. **Roll back:** localized to the Batch stacked-profile viewer.

`2385f75` — Batch waterfall R/2θ/Q axis; Mask 3-D HDF5 stack (2026-07-10)

Effect: Waterfall x-unit selector via a converting `AxisItem` + `_convert_radial()`; Mask temporal-constancy reads a single 3-D (time,y,x) HDF5. **Files:** `widgets.py`, `tab_batch.py`, `tab_mask.py`, `workers.py`, docs. **Roll back:** `_convert_radial` / `_UnitAxis` introduced here are reused by the Pump Probe heatmap — don't fully revert if Pump Probe is present.

`b628446` — clickable λ label with K-edge foil menu (2026-07-10)

Effect: Compact clickable "λ" label → K-edge foils menu (sets λ from edge energy). `constants.K_EDGE_FOILS`, `HC_KEV_A`, `helpers.make_kedge_label`. **Files:** `constants.py`, `helpers.py`, `tab_view/calibrate/pdf.py`, docs. **Roll back:** localized; `make_pixel_label` (next) reuses the factored helper.

`108ea7d` — Data Viewer projection Skip-frames + compact fields (2026-07-10)

Effect: Projection "Skip frames" field; narrower Axis/Skip/pixel fields; Lsd thousands-separators. **Files:** `tab_view.py`, docs. **Roll back:** localized.

`3248638` — clickable px label with detector pixel-size menu (2026-07-10)

Effect: Clickable "px" label → GE/Varex/Pilatus/Eiger sizes; `constants.PIXEL_PRESETS`; factored `helpers._clickable_menu_label`. **Files:** `constants.py`, `helpers.py`, `tab_view.py`, `tab_calibrate.py`, docs.

Roll back: localized.

`2b7a695` — Data Viewer intensity-statistics panel + histogram (2026-07-10)

Effect: `IntensityStatsPanel` (histogram + p70/p90/p99/p99.9/p99.99 with pixel counts) pinned to the loader bottom (stack mode); Current/All/projected scope. **Files:** `widgets.py`, `tab_view.py`, docs. **Roll back:** localized.

`d30be2c` — per-user configuration system + Preferences dialog (2026-07-12)

Effect: Per-user JSON config overlays shipped defaults at import; `settings.py`; `prefs_dialog.py` (Geometry/Paths/Materials/Calibrants/Menus/Algorithms); Settings menu; `DEFAULT_KERNEL/PIPELINE/OUTPUT_FORMAT/ERROR_MODEL/COLORMAP`; tab combos honor configured defaults. New `config_gui.md`, `config.example.json`, `tests/test_config.py`. **Files:** `settings.py` (new), `prefs_dialog.py` (new), `constants.py` (+165), `app.py`, `tab_batch.py`, `tab_calibrate.py`, `widgets.py`, docs, tests. **Roll back: high impact** — the config overlay + `DEFAULT_*` and the Preferences dialog are extended by the modular-tabs commit (`590b410`). Reverting this would also break `ui.visible_tabs` and the Tabs preferences section.

`399f3d7` — Stability: clean worker shutdown, no terminate(), stdout guard (2026-07-12)

Effect: `closeEvent` stops all tab QThreads; removed every `QThread.terminate()` (cooperative interrupt + orphan instead); `CalibrationWorker` restores stdout only if still its own; Data Viewer projection moved to `ProjectionWorker` (off GUI thread). **Files:** `app.py`, `tab_batch.py`, `tab_calibrate.py`, `tab_view.py`, `workers.py`. **Roll back:** reverting reintroduces the exit hard-crash risk and UI-freeze on projection — not recommended. Phase 1 of the 3-part review.

`4462ee1` — Performance: waterfall O(N), debounce, caching (2026-07-12)

Effect: Waterfall pre-allocated buffer + 100 ms coalesced redraw (O(N) not O(N^2)); 60 ms frame-slider debounce; cached radial bin-index grid; skip all-frame stats on frame-only changes. **Files:** `tab_view.py`, `widgets.py`. **Roll back:** reverting slows large scans / scrubbing; behaviour otherwise same. Phase 2 of the review.

`8846619` — Consistency: dedupe maps/writer, align lattices (2026-07-12)

Effect: `helpers._PARAMSTEST_DISTORTION` derived from `constants._V2_TO_V1` (removed duplicated `_V2V1` dicts); shared `helpers.write_standalone_paramstest` used by Calibrate + Export; LaB6/Si `_LATT/_LC` aligned to `MATERIALS`; texture colormap honors `DEFAULT_COLORMAP`. **Files:** `constants.py`, `helpers.py`, `tab_calibrate.py`, `tab_export.py`, `tab_texture.py`. **Roll back:** reverting reintroduces duplicated code paths; low functional risk. Phase 3 of the review.

`590b410` — Modular tabs + Pump Probe (TR-XRD) tab + plot quality (2026-07-12)

Effect: (1) **Modular tab visibility** — Data Viewer/Mask/Calibrate/Batch always shown, the rest toggled in Settings ▸ Preferences ▸ Tabs (`ui.visible_tabs`, applied live); `app.apply_tab_visibility()`. (2) **New Pump Probe tab** (`tab_pumpprobe.py`) — TRR filename pooling, MIDAS-engine integration via new `PumpProbeWorker` (reuses `build_integration_context`/`integrate_frame`), ΔI(q,delay), three-panel layout, four pyqtgraph views; ships a converted MIDAS `paramstest` as the default calibration. (3) **Plot quality** — white publication theme, black axes/labels/titles, readable legends, ΔI colorbar, aligned reference panel, shared draw-mode + line/point/font-size toolbar, bounded pan/zoom. **Files:** `tab_pumpprobe.py` (new), `workers.py` (+`PumpProbeWorker`), `app.py`, `constants.py`, `prefs_dialog.py`, `tests/test_smoke.py`, docs (`gui_documentation`, `config_gui`, `config.example.json`). **Roll back:** to remove **only Pump Probe**, delete

`tab_pumpprobe.py` and its wiring in `app.py`/`workers.py`; to remove **only modular tabs**, revert the `app.apply_tab_visibility` / `prefs_dialog` Tabs section and `constants` tab lists. A full `git revert 590b410` removes both features together. Depends on `aa9395e` (DataLoaderPanel), `1149afc` (build_integration_context), `2385f75` (`_convert_radial`/`_UnitAxis`), `d30be2c` (config/prefs).

`6d6f3fb` — docs: add development_history.md/.pdf (2026-07-13)

Effect: Added this per-commit change log + change→commit index + rollback guide. **Files:** `documentation/development_history.md`, `documentation/development_history.pdf`. **Roll back:** docs only.

`d5fafd8` — UI scaling: user-adjustable whole-interface zoom (HiDPI / 4K) (2026-07-13)

Effect: Whole-application scale (layout + fonts) for HiDPI / 4K monitors. `constants.DEFAULT_UI_SCALE` (config `ui.ui_scale`); `app.main()` applies it via `QT_SCALE_FACTOR` **before** the QApplication is created (so fixed widths, splitters, pyqtgraph and stylesheet px all scale uniformly) + `AA_UseHighDpiPixmaps`. Manual control in **Preferences ▸ Display** (spin + 100/125/150/200 % presets) and **Settings ▸ Interface scaling…**; both persist `ui.ui_scale` and offer an immediate self-restart (`MainWindow.restart_app` via `QProcess`) since the factor is read only at startup. **Files:** `constants.py`, `app.py`, `prefs_dialog.py`, docs (`gui_documentation`, `config_gui`, `config.example.json`). **Roll back:** `git revert d5fafd8`. Self-contained (depends only on the config system `d30be2c`); reverting removes the scale control and the app renders at 1.0×. To keep the feature but disable it, set `ui.ui_scale` to 1.0.

`c4e1c12` — Display Lsd in mm (calculations & files stay in μm) (2026-07-13)

Effect: The sample-to-detector distance is entered/shown in **mm** in the Data Viewer, the Calibrate seed, and Preferences ▸ Geometry; conversion to/from μm happens only at the display boundary (`DataViewerTab._lsd_um()`, ×1000 on read, ÷1000 on set). `get_geometry`/`apply_geometry`/`manual_seed`/prefs `_assemble` still emit μm; the config key stays `lsd_um` (μm); calibration-file writers (`paramstest.txt`, `calibration.json`) are unchanged (always μm). Batch drift-status label → mm. **Files:** `tab_view.py`, `tab_calibrate.py`, `prefs_dialog.py`, `tab_batch.py`, docs. **Roll back:** `git revert c4e1c12` to return every Lsd field to a μm display. Purely a display-layer change — no stored/file/calculation values are affected either way.

`d6df1f8` — Fix "Populating font family aliases" startup warning (2026-07-13)

Effect: Removed the `qt.qpa.fonts: Populating font family aliases` warning (and its ~40 ms startup cost) that Qt emitted because the code named the nonexistent family "Monospace" — both via `QFont("Monospace", …)` and the CSS generic `font-family:monospace`. Now names real per-platform families (Menlo/Consolas/DejaVu Sans Mono/Courier New) via `style.MONO_FAMILIES` / `MONO_CSS` and `widgets._mono_font()` (QFont `setFamilies` + Monospace style hint). **Files:** `style.py`, `widgets.py`, `tab_export.py`, `tab_view.py`. **Roll back:** `git revert d6df1f8` (cosmetic; only affects fixed-width font selection and re-introduces the harmless warning).

`455ca4e` — Calibrate: multi-column results readout + Send-to-Data-Viewer (2026-07-13)

Effect: (1) The Calibrate **Results** tab shows the full parameter set exactly as written to `paramstest.txt` (Lsd, BC, tx/ty/tz, p0–p14, Parallax, Wavelength, px, NrPixelsY/Z, RhoD, SpaceGroup, LatticeConstant) in a 3-column, column-major grid (`_paramstest_pairs` via the shared writer + `_populate_param_grid`), with a strain/timing line and the named-distortion table below. (2) New **→ Send to Data Viewer** button (`sendGeometryToViewer` signal + `_send_to_viewer`) pushes the calibrated λ/pixel/Lsd/beam-centre into the Data Viewer through new `DataViewerTab.set_geometry` (μm internal, Lsd shown mm, auto-BC off); wired in

app.py as the reverse of the Viewer→Calibrate hand-off. **Files:** tab_calibrate.py, tab_view.py, app.py, docs. **Roll back:** git revert 455ca4e. Self-contained; depends on the mm-display change (c4e1c12) for the Viewer Lsd conversion.

7e157e0 — Ship test_data; hide four optional tabs by default (2026-07-14)

Effect: (1) test_data/ sample data (calibrant_ceria.tif/h5, nickel_stack.h5, nickel_tifs/, make_test_data.py) is now committed so the GUI's default paths work on a fresh clone — .gitignore re-includes it past the global *.tif/*.h5 ignores via trailing negations, while test_data/output/, out_temp.txt and .DS_Store stay ignored. (2) DEFAULT_VISIBLE_TABS = ["Calib. Refinement", "Pump Probe"] — Corrections, PDF Analysis, Texture and Results & Export ship hidden (enable in Preferences ▸ Tabs). **Files:** .gitignore, test_data/** (added), constants.py, config.example.json, tests/test_smoke.py, docs. **Roll back:** git revert 7e157e0 restores the all-optional-tabs default and removes the shipped data + re-ignores test_data/. Note: a per-user config with its own ui.visible_tabs overrides the shipped default regardless.

6332b3d — Fix launch crash on fresh Linux/Windows (colormap w/o matplotlib) (2026-07-14)

Effect: Fixes the GUI failing to start on a clean env where every always-on tab's image viewer crashed with 'NoneType' object has no attribute 'getColors'. Cause: DEFAULT_COLORMAP "hot" (and the whole COLORMAPS list) are matplotlib colormaps; pg.colormap.get("hot") returns None without matplotlib, and that None was passed into pyqtgraph. New widgets._resolve_cmap() resolves native → matplotlib → native fallback → grayscale ramp and never returns None (used by ImageViewer, WaterfallViewer, Texture); matplotlib added as a declared dependency (pyproject + environment.yml matplotlib-base) so the named maps actually resolve. The downstream "Geometry hand-off wiring failed / QWidget has no attribute pushGeometry" log lines were only a symptom of the tabs becoming placeholders and disappear with this fix. **Files:** widgets.py, tab_texture.py, pyproject.toml, environment.yml, tests/test_smoke.py. **Roll back:** git revert 6332b3d (reintroduces the crash on matplotlib-less envs).

72a1770 — env: drop midas_suite (unblocks conda env create) (2026-07-14)

Effect: conda env create -f environment.yml was aborting because the pip section's midas_suite meta-package pulls midas-index 0.7.3, whose sdist fails to build against scikit-build-core>=0.8 (Use cmake.version instead of cmake.minimum-version). The GUI never uses midas_suite's extras; the backends it actually imports (midas-calibrate-v2/-integrate-v2/-calibrate/-hkls/-distortion) are declared in pyproject.toml and pulled by the editable -e . install. Removed the redundant midas_suite line; README updated. (Unrelated to the repo: the libarchive.19.dylib / conda-libmamba-solver errors in that log are a broken base-Anaconda mamba, worked around with --solver=classic.) **Files:** environment.yml, README.md. **Roll back:** git revert 72a1770 (re-adds midas_suite and its build failure).

68313f8 — Pin environment.yml to a verified NumPy-1 set + README recreate steps (2026-07-14)

Effect: Makes conda env create reproduce a known-good environment. Fresh installs previously pulled the newest backends (numba 0.66 → NumPy 2.x) against conda torch 2.2.2, which can't interop with NumPy 2 (torch.from_numpy: Numpy is not available). Pinned every package to the versions from the user's working base Anaconda — the **NumPy-1 family**: numpy=1.26.4, scipy=1.13.1, h5py=3.11.0, tifffile=2023.4.12, pyqtgraph=0.14.0, matplotlib-base=3.8.4 (conda); PyQt5==5.15.10, torch==2.4.0, numba==0.59.1, and the NumPy-1-era MIDAS backends (calibrate-v2 0.3.3, integrate-v2 0.1.0, calibrate 0.2.3, hkls 0.4.1, distortion 0.2.0) via pip; then -e .. Dropped the conda pytorch/cpuonly lines + pytorch channel (torch now via pip). pyproject.toml: lowered midas-calibrate floor 0.2.7 → 0.2.3 so the proven base version satisfies -e .. README gains a "Recreating the environment" section (remove + create), a conda-solver/libarchive note, and

the CPU-only torch install note. Verified end-to-end: fresh env resolves clean; GUI + a real nickel-frame integration run (numpy 1.26.4 + torch 2.4.0). **Files:** `environment.yml`, `pyproject.toml`, `README.md`. **Roll back:** `git revert 68313f8` (returns to ranged deps; a fresh env would again risk the NumPy-2/torch mismatch). To move forward to a NumPy-2 stack instead, bump torch>=2.3 and the backends to their numba-0.66 releases together.

`3a6ecb9` — Calibrate Results: all-text multi-column readout (drop distortion table) (2026-07-14)

Effect: The Calibrate **Results** tab is now entirely text. Removed the distortion `QTableWidget`; the distortion coefficients appear in the same multi-column parameter grid, with the paramstest `p0`–`p14` slots relabelled to their names (iso_R2, a1, phi1, …) via `helpers._PARAMSTEST_DISTORTION`. Larger font (12 pt) and roomier row/column spacing for readability. Removed the `DistortionTable` import + `set_distortion` call. **Files:** `tab_calibrate.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 3a6ecb9` (restores the named-distortion table widget below the grid). `DistortionTable` still exists in `widgets.py` (used only here), so a revert is clean.

`b1642fa` — Fix calibration TypeError (initial_BC_y) + add scikit-image (2026-07-14)

Effect: Fixes calibration failing with `calibrate() got an unexpected keyword argument 'initial_BC_y'` on the pinned (base-matching) midas-calibrate-v2 0.3.3, whose `calibrate()` has no beam-centre seed parameter (only newer backends add it). New `calib._supported_kwargs(fn, kwargs)` filters kwargs to the installed callable's signature (keeps `initial_Lsd`, drops the unsupported `initial_BC_y/z`, logs it), so the GUI tolerates backend-version signature drift. Also pins **scikit-image=0.23.2** in `environment.yml` — midas-calibrate-v2's `auto_seed_calibrant` needs it; without it, seeding degrades to the coarse arc fallback. Verified: manual-seed one_shot calibration of the shipped ceria runs end-to-end (numpy 1.26.4 + torch 2.4.0). **Files:** `calib.py`, `environment.yml`. **Roll back:** `git revert b1642fa` (reintroduces the TypeError on backends lacking the BC-seed args, and removes the scikit-image pin).

`df544d2` — Data Viewer: tilt/distortion-aware radial integration (2026-07-14)

Effect: When a loaded calibration file carries the full geometry (tilts + distortion), the Data Viewer's radial integration goes through the MIDAS engine (`build_integration_context` + `integrate_frame`) instead of concentric-circle binning, so pixels map through the calibrated tilts/distortion. `_load_calibration` now also reads the full geometry via `geometry_fields_from_file` (falls back to scalar/circle mode if absent) and shows the active mode. `_midas_radial` caches the binning geometry per (geometry, R-bin, image shape, static mask) and reuses it across frames (fast `hard` kernel; loader's static composite mask so the cache holds); `_radial_integrate` branches to it with a guarded fallback to circle binning on error. **Files:** `tab_view.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert df544d2` (reverts to circle-only radial integration in the Data Viewer). Self-contained; the MIDAS-engine primitives it reuses are unchanged.

`bc86d74` — Data Viewer: Top-N pixels + mask-aware stats + full-geometry receive (2026-07-19)

Effect: Adds a "Top-N pixels" toolbar toggle that marks the N brightest pixels (crosshair + circle) and feeds their values to the intensity-statistics panel, re-ranking live on frame/mask changes. Masked pixels (mask file + intensity-range mask) are excluded from the Top-N ranking and its stats/histogram. `_midas_radial` now unions the static mask with the per-frame intensity-range mask and fingerprints the mask by content so the cached binning context rebuilds when the mask changes. The tab also accepts a *full* geometry dict (tilts + distortion + detector size) from Calibrate and routes integration through the MIDAS engine when it arrives. The stats panel readout box auto-sizes to its content (no inner scrollbar) with the histogram as the single flexible child;

`DataLoaderPanel` is now a `QWidget` hosting a scroll area + stats panel in a draggable vertical splitter. **Files:** `tab_view.py`, `widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert bc86d74`.

`b0a5cc3` — Calibrate: per-coefficient distortion, frame averaging, seed feedback (2026-07-19)

Effect: Adds `DistortionRefineDialog` (behind the "…" next to the Distortion checkbox) to pick any of the 15 harmonics or use η-fold presets; `calib.py` maps each selected v2 harmonic to its v1 p-slot (via `DISTORTION_ISO`/`DISTORTION_PRESETS` in constants). Adds an "Average frames" card (start:end:skip, streamed via `DataLoaderPanel.average_frames`), seed tilts (tx/ty/tz) carried into the v1 params, a "Feed result back to seed" option, and a "→ Send to Data Viewer" that now sends the full geometry (tilts + distortion + detector size). Drops the 310px bottom-panel cap and adds a non-collapsible splitter. **Files:** `calib.py`, `tab_calibrate.py`, `dialogs.py`, `constants.py`, `widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert b0a5cc3`.

`b6f1681` — Batch: read-only "Calibration values" card (2026-07-19)

Effect: Adds a Calibration values card to the Batch parameter column showing the geometry actually driving integration (λ, Lsd, BC, tilts, pixel sizes, detector size, non-zero distortion count), resolved from the live Tab-2 result or a calibration file and refreshed on source/path change. Also lets the Log panel fill its splitter space. **Files:** `tab_batch.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert b6f1681`.

`49623ae` — Pump Probe: publication plotting, delay colour-bar, default mask (2026-07-19)

Effect: Publication-quality plot defaults (clean 2.0px lines, 13pt fonts, dashed grid, refactored `_rainbow_cmap`); a continuous delay→colour bar (`GradientLegend`) replaces the many-row legend past a threshold, with left/right legend anchoring; a log-delay x-axis option for kinetics; and a default detector mask wired via new `DataLoaderPanel.add_mask_file` / `MaskSelector.add_file_source`. TR-XRD defaults now point at a local-only `test_data_pump_probe/` folder kept out of git. **Files:** `tab_pumpprobe.py`, `constants.py`, `widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 49623ae`.

`b251ca8` — chore: gitignore context, internal docs, large sample sets (2026-07-19)

Effect: Ignores the local `.context/` scaffold and `CLAUDE.md`, the internal-only PDF calculation write-up, and the large local sample dirs (`test_data/17BM/`, `test_data/test_s25ide/`) via directory-level ignores that override the `test_data/**` re-includes. **Files:** `.gitignore`. **Roll back:** `git revert b251ca8`.

`1769f69` — Data Viewer: live PV stream (Live Data card) (2026-07-20)

Effect: Subscribes to an EPICS PVA image PV via `pvapy` and pushes frames straight through the existing image/corrections/ring/radial-integration pipeline — no separate viewer window. "Live Data" is a collapsible card (own title-bar checkbox) above the Data card; unchecking stops an active stream. Data card gets a small ⟳ reload button to restore the static file/folder/HDF5 source after stopping a stream. `pvapy==5.4.1` is now a required, pinned dependency (chosen over latest to avoid upgrading the numpy/pyqtgraph pins). `MainWindow` calls a generic `tab.shutdown()` hook on close so a live stream stops cleanly on app exit. **Files:** `widgets.py`, `tab_view.py`, `app.py`, `pyproject.toml`, `environment.yml`, `documentation/gui_documentation.md`, `tests/test_live_stream.py`. **Roll back:** `git revert 1769f69`.

`0b4326d` — Fix: cap native thread pools to prevent calibration crash (2026-07-22)

Effect: Clicking "Run Calibration" crashed the app outright — a native, uncatchable `Bus error`, not a Python exception. Root cause: `CalibrationWorker` (a `QThread`) triggers a multi-threaded `numpy.linalg.inv()` deep

inside `midas-calibrate-v2`'s HKL/ring generation (likely surfaced by the recent 0.3.3 → 0.5.2 bump); running from a `QThread` while the Qt event loop is alive, that races against PyTorch's own OpenMP pool and corrupts memory. Reproduced deterministically at two call sites (`numpy.linalg.inv` during ring generation, `scipy.ndimage.median_filter` during seeding); both disappear once native thread pools are capped to 1. Every heavy operation in this app runs inside a `QThread` (17 worker classes in `workers.py`), so the fix caps `OPENBLAS_NUM_THREADS` / `OMP_NUM_THREADS` / `MKL_NUM_THREADS` / `VECLIB_MAXIMUM_THREADS` / `NUMEXPR_NUM_THREADS` to 1 process-wide via `os.environ.setdefault` in `_paths.py` — already home to the sibling `KMP_DUPLICATE_LIB_OK` workaround for the same underlying duplicate-OpenMP-runtime issue — rather than patching each worker individually. Verified: offscreen run of the full calibration pipeline to convergence with no crash; `pytest` 15/15 green. **Files:** `midas_gui/_paths.py`. **Roll back:** `git revert 0b4326d` (restores multi-threaded BLAS; calibration/other workers become exposed to this native-crash class again on macOS).

`234ded9` — Live viewer: preserve manual colormap/level changes across frames (2026-07-24)

Effect: `ImageViewer._redisplay` recomputed vmin%/vmax% percentile levels on every incoming frame (live or otherwise), silently overwriting a level range the user had dragged on the pyqtgraph histogram mid-acquisition. Now tracks manual histogram edits via `sigLevelsChanged` (guarded against feedback from our own programmatic `setImage` calls with a suspend flag) and keeps pinning to the last manual levels until the user explicitly changes the vmin%/vmax% spin boxes, which resets back to auto. **Files:** `widgets.py`. **Roll back:** `git revert 234ded9`.

`69f470c` — Data Viewer: widen the image/radial-plot splitter handle (2026-07-24)

Effect: The vertical splitter between the image view and the radial-integration panel had no explicit handle width (unlike the outer horizontal splitter), making it thin and hard to grab. Set `setHandleWidth(8)` to match. **Files:** `tab_view.py`. **Roll back:** `git revert 69f470c`.

`96f8add` — Radial profile: stop forcing Y-axis min to -1000 on every update (2026-07-24)

Effect: `ProfileViewer._replot` unconditionally called `setYRange()` with a hardcoded -1000 floor on every redraw, clobbering any manual Y-range the user set. The auto lower bound is now `0.9 * current data minimum`; once the user manually changes the Y range (drag, wheel-zoom, or the axis context menu's numeric entry — detected via `sigYRangeChanged` with the same suspend-flag pattern as `234ded9`), that value is kept for the rest of the session instead of being recomputed. Also dropped the hardcoded `setLimits(yMin=-1000/1)` hard floors. **Files:** `widgets.py`. **Roll back:** `git revert 96f8add`.

`3eb4a44` — Calibrate: recolor Pick-Ring fit overlay to blue (2026-07-24)

Effect: `PickableImageViewer`'s Pick-Ring points, fitted circle, and fitted-center marker used the same amber (`#f0c060`) as the Data Viewer's Simulate-rings overlay, making the two indistinguishable when both were visible on the same image. Pick-Ring's fit overlay is now blue (`#2a7fd4`); the separate Pick-BC "+" click marker (already `#00aaff`) and Simulate-rings amber were left unchanged. **Files:** `widgets.py`. **Roll back:** `git revert 3eb4a44`.

`ecf6d01` — Data Viewer: make Simulate rings a live toggle instead of one-shot (2026-07-24)

Effect: "Simulate rings" is now a checkable toggle rather than a single-click action. While on, it resimulates automatically whenever material, lattice constants, space group, or geometry (wavelength/Lsd/pixel size/max 2θ) change — mirroring the existing `_rad_auto`-style "recompute on change" idiom already used for radial integration. Beam-centre edits still just reposition the existing rings (unchanged; ring radii don't depend on BC), so no wiring was added there. **Files:** `tab_view.py`. **Roll back:** `git revert ecf6d01`.

`aa82cb6` — Preferences: add Devices tab; Data Viewer Live PV becomes a device dropdown (2026-07-24)

Effect: New **Preferences ▸ Devices** tab (add/remove/edit rows of name + prefix + PVA suffix), following the existing Materials/Calibrants table-tab pattern; persisted as a new `"devices"` list in the JSON config (replaces wholesale when present, same as materials/calibrants). Ships pre-filled with the 20-ID-D detectors extracted from the beamline's `make_det(...)` blueprint — `oryx20idd` (`20iddOR1:`), `s20idPil` (`20idPil`), `pg4` (`1idPG4:`), `gh1s` (`20idGH1s:`), `s20varex1` (`20IDFF:`) — all with PVA suffix `Pva1:Image`. The Data Viewer's Live Data "Live PV" field is now an editable combo box (`_NoScrollComboBox`) populated from this list: picking a device by name fills in its full PV (`prefix + PVA suffix`); typing any other PV by hand still works unchanged, with the same example placeholder text. **Files:** `constants.py`, `prefs_dialog.py`, `widgets.py`, `tests/test_live_stream.py`. **Roll back:** `git revert aa82cb6` (Devices tab and the Live PV dropdown both go; the Live PV field reverts to a plain text box).

`c156c62` — Fix image-view autorange drift; bound pan/zoom to image (2026-07-24)

Effect: Two bugs in the shared `ImageViewer` (Data Viewer / Mask Builder / Calibrate all use it): (1) the crosshair `InfiniteLines` were added to the pyqtgraph `ViewBox` without `ignoreBounds=True`, so their position — which follows the mouse on every hover event — fed the auto-range bounds calculation. Once the bottom-left **A** (auto-range) button enabled *continuous* auto-range, the view kept re-fitting itself to wherever the cursor was, and only a right-click ▸ View All (which disables continuous auto-range as a side effect) made it static again. Fixed by excluding the crosshairs from bounds via `ignoreBounds=True`. (2) Pan/zoom had no limits, so scroll/drag could wander arbitrarily far from the image or zoom out indefinitely into empty space. Added `ViewBox.setLimits()` (computed from the image shape in `set_image()`): pan is bounded to roughly half an image-width/height of margin past each edge, and min/max zoom range are tied to the image size. **Files:** `widgets.py` (`ImageViewer.set_image`, new `_apply_view_limits`). **Roll back:** `git revert c156c62` (crosshair can drift the view again under continuous auto-range; pan/zoom become unbounded again).

`cde4bb2` — Radial profile plot: bound pan/zoom to the data extent (2026-07-24)

Effect: Same class of issue as `c156c62`, applied to the shared `ProfileViewer` (the radial-integration plot on the Data Viewer and Calibrate tabs): it had no `ViewBox` limits, so the plot could be scrolled/dragged arbitrarily far from the actual profile. `_replot()` now calls a new `_apply_view_limits(xmin, xmax, ymin, ymax)` on every redraw — computed from the current profile's X range (in whichever unit is selected: R (px) / 2θ / Q) and Y range (respecting the existing sticky-manual-Y-min behavior) plus a margin (15% X, 25% Y), via `ViewBox.setLimits(...)`. Recomputing per-replot means the bound tracks new profiles and axis-unit switches automatically. **Files:** `widgets.py` (`ProfileViewer._replot`, new `ProfileViewer._apply_view_limits`). **Roll back:** `git revert cde4bb2` (radial profile pan/zoom becomes unbounded again).

`5b8e3b3` — User profiles, full GUI state save/load, Calibrate tilt-ring rework (2026-07-27)

Effect: Three pieces of work landed in one commit: 1. **User profiles.** `settings.py` now keeps one config file per named profile under `<config dir>/profiles/<name>.json` (`profile_meta.json` tracks which is active), transparently migrating an existing single `config.json` into a profile named "Default" the first time it runs — no data lost, no explicit migration step. `load_config()`/`save_user_config()`/`reset_user_config()`/`user_config_path()` keep their existing signatures and now resolve to the active profile, so every existing call site (`constants.py`, `prefs_dialog.py`, `app.py`) needed no changes. New `constants.reload_from_config()` resets every `DEFAULT_*` global to its shipped value then re-applies the active profile's config on top (a plain re-`_apply` alone would leave a previous profile's override in place if the new profile doesn't mention that key). Preferences gets a **Profile row** — combo + New…/Duplicate…/Rename…/Delete — wired to this API. 2. **Full GUI state save/load.** New **File** menu, **Save GUI State…** (`Ctrl+S`) / **Load GUI State…** (`Ctrl+O`), dumps/restores every tab's fields to one JSON file via new `widgets_to_dict()`/`apply_dict_to_widgets()` dispatch helpers (`helpers.py`) and a

`get_state()`/`set_state()` pair added to all 10 tabs plus the shared `DataLoaderPanel`/`FieldSelector`/`MaskSelector`/`CorrectionFlagsWidget` widgets. Loading a state re-triggers each tab's own file-loading for path-backed fields (images, masks, dark/bright/background, HDF5 datasets) but deliberately does **not** re-run long pipelines (Fit, Batch Integrate, PDF transform, Refinement) — those need one manual click of the tab's own action button afterward. `MaskTab`/`CalibrationTab` additionally write small sidecar files (`<stem>_mask.tif`, `<stem>_calibration.json`) next to the state file so an in-progress mask or fit result that hasn't been exported anywhere else isn't silently lost. 3. **Calibrate tilt-ring rework** (previously uncommitted, folded in here): the "Corrected" ring overlay is now drawn synchronously from the fitted ty/tz tilt (`_draw_corrected_rings`) instead of a background `CorrectedRingsWorker` forward-model thread (removed from `workers.py`). Data Viewer gained matching `ty`/`tz` tilt fields for its ring simulation, and every Ring-simulation spin-box's step size (λ, max 2θ, Lsd, pixel, BC, tilt) is now configurable via Preferences ▸ Data Viewer / the `viewer_steps` config key instead of hardcoded. **Files:** `settings.py`, `constants.py`, `prefs_dialog.py`, `helpers.py`, `widgets.py`, `app.py`, `tab_view.py`, `tab_calibrate.py`, `tab_mask.py`, `tab_batch.py`, `tab_refine.py`, `tab_pumpprobe.py`, `tab_corrections.py`, `tab_pdf.py`, `tab_texture.py`, `tab_export.py`, `workers.py`, `tests/test_config.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 5b8e3b3` — reverts all three pieces together (they share edits in `tab_view.py`/`tab_calibrate.py`, so a partial revert needs a hand-picked patch, not a plain `git checkout <path>`). Existing single-profile configs are unaffected either way since `profiles/Default.json` is a copy, not a move, of the original `config.json`.

`e14c1ea` — Data Viewer: tilt-aware profile, ring fixes, calibration export, plot zoom persistence (2026-07-28)

Effect: Seven requested fixes/features plus two follow-up plot bugs found while testing them, all in the Data Viewer tab: 1. **Tilt-corrected radial profile without a calibration file.** New `DataViewerTab._effective_calib_geom()` returns `self._calib_geom` if a calibration is loaded, otherwise synthesizes a geometry from the Ring-simulation card's own `ty`/`tz`/BC/Lsd/pixel fields whenever they're non-zero. `_radial_integrate()`/`_midas_radial()` route through this instead of only checking `_calib_geom`, so the profile's R/2θ/Q axis stays tilt-consistent with the already-correct 2D ring overlay even before any calibration file is loaded. `_midas_radial`'s integration-context cache key changed from `id(g)` to a value tuple (the synthesized geometry is a fresh dict each call, so `id()` never hit the cache). 2. **Ring 2θ-range cutoff bug.** `_redraw_rings` capped ring radius at the image's pixel diagonal (`math.hypot(NY, NZ)`), silently dropping any ring past that regardless of the configured "max 2θ" — for far-detector/ fine-pixel geometries this fell around 35-40°. Replaced with a bare `rad > 0 and math.isfinite(rad)` guard; pyqtgraph's ViewBox already clips anything drawn outside the visible image. 3. **Configurable ring thickness.** New `DEFAULT_RING_WIDTH = 2.0` constant and a spin box in the Ring-simulation card, wired into `_redraw_rings`'s pen width and into `_state_widgets()` (round-trips through Save/Load GUI State). 4. **"Simulate rings" turns green while live.** New `QPushButton#primary:checked` rule in `style.py` — confirmed via grep that the Data Viewer's live-toggle button is the only checkable `primary_btn` in the app, so this can't affect any other tab. 5. **Save calibration from the Data Viewer.** Three new buttons (Save JSON / Save params (.txt) / Save PONI) export whichever geometry is currently in effect (`_export_geom()`, mirroring `_effective_calib_geom()`). JSON uses the same bare-key shape `geometry_fields_from_file()` reads back; `.txt` reuses the existing `write_standalone_paramstest()`; `.poni` uses a new `helpers.write_poni()` that inverts the existing PONI reader's convention (`Poni1/2 = BC_y/z * pxY/Z`, `Distance = Lsd`, SI units) — tx/ty/tz tilts have no PONI Rot1-3 equivalent and are documented as dropped on export, matching the reader's existing documented limitation. 6. **`Ctrl+S` overwrites a loaded/saved GUI-state file.** `MainWindow` tracks `_gui_state_path`, set on every successful save or load; a plain `Ctrl+S` now writes straight to that path with no dialog once one exists. New `Ctrl+Shift+S` ("Save GUI State As…") always prompts, and becomes the new target for subsequent plain saves. 7. **Radial-profile X-axis floors at 0.** Both `ProfileViewer._replot`'s `setXRange` call and `_apply_view_limits`'s pan/zoom `xMin` bound now clamp to `max(0.0, ...)`.

Testing these surfaced two more bugs in `ProfileViewer`, fixed in the same commit: - **Y-range could get permanently stuck.** The old `_manual_ymin` latch recorded *any* Y-range-changed signal, including ones pyqtgraph fires internally (e.g. autorange during `curve.setData()`) well before the method's own suspend-flag window started — once latched, every future replot used that stale value regardless of new data. Fixed by

wrapping the entire `_replot()` body (curve/band updates, limit-setting, everything) in one suspend flag, so only genuine mouse-driven pan/zoom is ever recorded. - **Zoom reset on every parameter change.** Replaced the unconditional `setXRange`/`setYRange` calls with tracked `_user_xrange`/`_user_yrange`: the plot still auto-fits fresh data when the user hasn't manually zoomed, but a manual zoom/pan is now restored after every replot instead of being wiped by the next profile update. The remembered range is only dropped when it would be meaningless in the new context — switching the R/2θ/Q axis unit clears the X range, toggling Log Y clears the Y range. **Files:** `midas_gui/app.py`, `midas_gui/constants.py`, `midas_gui/helpers.py`, `midas_gui/style.py`, `midas_gui/tab_view.py`, `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert e14c1ea`. All pieces are additive/isolated (new widgets, a new geometry-resolution helper, a new writer function, and a `ProfileViewer` range-tracking rewrite) — no other commit builds on this one.

`3c15ae7` — Data Viewer: native-menu-backed manual axis limits, Top-N intensity floor (2026-07-29)

Effect: Reworks the radial-integration plot's and intensity-histogram's Auto/Manual toggle to defer axis-value entry to pyqtgraph's own existing right-click axis "Manual" min/max fields, instead of a separate custom field row (an earlier same-day attempt at this feature used custom fields and was corrected): 1. New `_add_auto_manual_buttons(plot_widget, on_auto, on_manual)` in `widgets.py` hides pyqtgraph's native auto-range corner button and replaces it with a small "A"/"M" `QPushButton` pair in the same spot, wired to `clicked` (not `toggled`) so that **reclicking the already-active button** still fires its handler — "A" re-fits immediately to current data, "M" snaps back to the held manual range, discarding any pan/zoom drift in either mode. 2. New `_install_manual_axis_capture(plot_widget, callback)` hooks each axis's native `ViewBoxMenu` `minText`/`maxText` `editingFinished` signals (pyqtgraph already applies the typed value to the view itself) and mirrors the committed range into a `self._manual_range` tuple tracked independently of `ViewBox.state['targetRange']` (which changes on drag/ pan too, not just explicit edits — needed for reclick-to-reset to mean something different from "wherever the mouse left it"). 3. `ProfileViewer`/`IntensityStatsPanel` `_replot`/`_redraw_hist` skip their auto-fit/clamp logic entirely while `_manual_mode` is set and call the new `_apply_manual_range()` instead (clears any `setLimits()` pan/zoom bound, then `setXRange`/`setYRange` with `padding=0` from the held tuple) — this is what makes an exact typed value (e.g. `0`) survive every live- acquisition redraw instead of being clamped/padded by that frame's own `_apply_view_limits()` recompute. 4. Top-N pixel marking (`DataViewerTab._show_topn`, `tab_view.py`) gains an "I >" checkbox + threshold field next to the N spin box, matching the existing `_imask_on`/`_fspin` convention; pixels at/below the threshold are excluded from ranking before `argpartition`, so fewer than N (or zero) markers show when fewer pixels clear the floor. 5. Unrelated: `constants.DEFAULT_DEVICES` PVA device name/prefix updates (`oryx20idd→20iddNF`, `20iddOR1:→20idOR1:`, `20idPil→20idPil:`, `gh1s→20iddTomo`, `s20varex1→20iddFF`), bundled into this commit at the user's request. **Files:** `midas_gui/constants.py`, `midas_gui/tab_view.py`, `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 3c15ae7`. Self-contained — no later commit depends on the new helpers or the Top-N threshold field.

`53f8f19` — Widen non-data-derived numeric caps across tabs (2026-07-29)

Effect: Several `QSpinBox`/`QDoubleSpinBox` range caps had no physical or data-derived justification (arbitrary round numbers picked when the field was first added) and were clamping legitimate inputs; raised them: 1. Data Viewer's simulated-ring `ty`/`tz` tilt fields and Calibrate's seed `tx`/`ty`/`tz` fields: `±10°` → `±180°`, matching the full range other angle fields in the app already allow. 2. Data Viewer max-2θ: `1–90°` → `0.001–180°`. 3. Calibrate E-M/LM iteration counts and multi-panel geometry (panel counts, size, gap): capped at 20/2000/50/10000/1000 → `1_000_000`. 4. Corrections empty-frame/Compton scale, absorption μR, gain-training steps/ lr/unity-weight/smooth-weight: capped at 1–20 → effectively unbounded (`1e6`–`1e9`). 5. Mask hot/dead factor, frozen-fraction, stride, and learnable-mask steps/ lr/sparsity: capped at 1–2000 → effectively unbounded. 6. Refine optimizer lr and iteration count: capped at 10/2000 → effectively unbounded. 7. Batch drift `n_knots`: capped at 20 → `1_000_000`. **Files:** `midas_gui/tab_batch.py`, `midas_gui/tab_calibrate.py`, `midas_gui/tab_corrections.py`, `midas_gui/tab_mask.py`, `midas_gui/tab_refine.py`,

`midas_gui/tab_view.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 53f8f19`. Self-contained — no later commit depends on the widened ranges.

`05ce224` — Data Viewer: mask enable checkboxes, calibration card reposition + ty/tz sync, intensity-range title checkbox (2026-07-30)

Effect: Three independent Data Viewer tweaks: 1. `MaskSelector` (`widgets.py`) rows gain a checkbox to include/exclude a mask from the composite without deleting it; `composite_mask()` now OR's only *checked* sources. The "Tab 1 mask" row (auto-populated from the Mask Builder tab) is always inserted/kept at the top of the list. `get_state()`/`set_state()` persist each source's `enabled` flag. 2. The Load calibration card moved below the Ring simulation card's Simulate button, and loading a calibration file now also fills the Ring-simulation card's ty/tz tilt fields (previously only λ/Lsd/px/BC). 3. The Intensity range card's title bar is now the on/off checkbox itself ("Exclude out-of-range pixels") instead of a separate checkbox line inside the card. **Files:** `midas_gui/tab_view.py`, `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 05ce224`. Self-contained — no later commit depends on the mask `enabled` flag, the card reposition, or the title-bar checkbox.

`0e9ed21` — Data Viewer: support multiple ring-simulation materials at once (2026-07-29)

Effect: Ring simulation card holds a list of materials instead of one, so a sample phase can be overlaid on a calibrant (or any N materials) at the same time: 1. New `MaterialDialog` (factored out of the old inline lattice fields) edits one material's name/preset/lattice/space-group/cubic-checkbox. Opened by clicking a material row's underlined name button. 2. Each material row is a checkbox (show/hide that material's rings), a clickable color swatch (`QColorDialog`, drives that material's ring lines + hkl labels on both the image overlay and radial-profile plot), the clickable name, and a ✕ delete button (disabled when it's the last remaining row). **+ Add material** appends a new row with the next color from a 10-entry cycling palette (`_MATERIAL_COLORS`); a single default **Ni (FCC)** row is seeded at startup, matching prior behavior. 3. Geometry fields (λ, max 2θ, Lsd, pixel size, beam centre) stay shared across all materials — only lattice/SG/color/visibility are per-material. 4. `_simulate()` now loops materials, simulating rings per enabled one and collecting per-material errors instead of aborting on the first exception; `_redraw_rings()`/`_refresh_profile_markers()` draw each material's rings in its own color. 5. `ProfileViewer.set_ring_markers()` (`widgets.py`) takes a list of `{"radii": [...], "color": "#hex"}` groups instead of a flat radii list, so the radial-profile plot can show multiple colored ring sets; `tab_calibrate.py` updated to pass its single ring set as a one-entry group list. 6. `DataViewerTab.get_state()`/`set_state()` persist/restore the materials list (replacing the old single-material `mat`/`a..sg`/`cubic` state fields) so GUI-state save/load round-trips multi-material setups. **Files:** `midas_gui/tab_view.py`, `midas_gui/tab_calibrate.py`, `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 0e9ed21`. Self-contained — no later commit depends on the multi-material materials list or the `set_ring_markers()` groups signature.

`911f8ac` — Data Viewer: Live Data "Use Buffer" ring buffer for stack analysis on live streams (2026-07-30)

Effect: Live Data card gains a **Use Buffer** toggle + **N** spin box (`DataLoaderPanel`, `widgets.py`) that captures the last N live frames into an in-memory `deque` ring buffer, so Projection and every other stack-based analysis become usable on live data instead of only on loaded HDF5/folder stacks: 1. Clicking **Use Buffer** arms it: turns yellow (`Buffering… (n/N)`) and appends each incoming live frame to the buffer as it streams. 2. When no new frame arrives for ~2 s (streaming paused, or **Stop** clicked), a single-shot `QTimer` (`_buffer_stall_timer`) freezes the buffer into a navigable stack: `_nframes`/`_setup_navigator()` update so the frame slider/spin/prev-next and **Project stack** work exactly as they would on a loaded stack; the button turns green (`Buffer Ready (N)`). 3. `_get_frame()`/`full_stack()` read from the frozen buffer when active, otherwise fall through to the existing stack/paths/h5 sources unchanged. 4. Starting a new live stream or loading static data calls `_reset_buffer()`, discarding any existing buffer and turning the toggle off. **Files:** `midas_gui/widgets.py`,

documentation/gui_documentation.md. **Roll back:** `git revert 911f8ac`. Self-contained — no later commit depends on the ring-buffer fields or `_get_frame()`/`full_stack()` buffer branch.

`a88ba1f` — Data Viewer: fix live ring-buffer race between GUI and worker threads (2026-07-30)

Effect: `_on_live_frame()` appends to the "Use Buffer" ring buffer (`DataLoaderPanel._buffer`, `widgets.py`) on the GUI thread while background `QThread`s (e.g. `ProjectionWorker.run()`, reached via `full_stack()`) read the same `deque` with no synchronization — a live frame arriving mid-read could raise "deque mutated during iteration" or hand analysis a torn frame set. Added a `threading.Lock` (`_buffer_lock`) guarding every read/write of `self._buffer` and the `_buffer_frozen` flag checked alongside it, in `_get_frame()`, `full_stack()`, `_on_live_frame()`, `_on_buffer_toggled()`, `_on_buffer_stalled()`, and `_reset_buffer()`. **Files:** `midas_gui/widgets.py`. **Roll back:** `git revert a88ba1f`. Self-contained — the lock only wraps existing buffer accesses, no signature/state-shape changes.

`2cfd9cd` — Data Viewer: fix refresh-timer starvation during fast live streaming (2026-07-30)

Effect: `dataChanged` was connected to `self._refresh_timer.start()` directly. `QTimer.start()` on an already-armed `setSingleShot(True)` timer restarts its countdown rather than queuing a fire, so a continuous burst of events faster than the 60ms interval (e.g. live frames streaming quickly) could defer the refresh indefinitely — the displayed image/stats/rings would appear frozen mid-burst even though data kept arriving. Added `_on_data_changed()`, connected to `dataChanged` in place of the old lambda, which only calls `._refresh_timer.start()` while the timer is idle (`not isActive()`), turning it into a throttle (fires at least once per interval) instead of a restart-on-every-event debounce that can starve. **Files:** `midas_gui/tab_view.py`. **Roll back:** `git revert 2cfd9cd`. Self-contained — restores the direct lambda connection.

`839770d` — Data Viewer: cap live buffer ("Use Buffer") to 100 frames (2026-07-30)

Effect: The ring-buffer **N** spinbox (`widgets.py`) allowed up to 2000 frames, each stored as a full float32 array — for a large-format detector this could allocate tens of GB with no warning. Lowered `setRange(2, 2000)` to `setRange(2, 100)`; tooltip and `gui_documentation.md` updated to note the cap. **Files:** `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 839770d`. Self-contained — restores the old range only.

`08392c6` — Data Viewer: move "All frames" stats computation off the GUI thread (2026-07-30)

Effect: `_all_frame_values()` (reached from `_update_stats()` when the stats scope is "All frames") read the entire stack/live buffer and applied field corrections synchronously on the GUI thread — unlike every other heavy operation in this file, which already runs in a background `QThread`. For a large HDF5/folder stack, or a full live ring buffer, this could visibly freeze the UI. 1. Added `AllFrameStatsWorker` (`workers.py`), following the same pattern as `ProjectionWorker`: GUI-derived inputs (dark/bright/background arrays, composite mask, intensity-range thresholds) are snapshotted on the GUI thread before the worker starts, so `run()` only touches numpy data. 2. `_update_stats_all_frames()` (`tab_view.py`, replaces `_all_frame_values()`) launches the worker and, if a new request arrives while one is already running, coalesces it into a single re-run afterward via `_stats_all_frames_dirty` instead of queuing redundant work. **Files:** `midas_gui/tab_view.py`, `midas_gui/workers.py`. **Roll back:** `git revert 08392c6`. Self-contained — no later commit depends on `AllFrameStatsWorker` or the removal of `_all_frame_values()`.

`57ced2f` — Data Viewer: debounce intensity-mask spinbox edits (2026-07-30)

Effect: The intensity-mask "pixel ≤" / "pixel >" spinboxes triggered `_update_stats()` / `_radial_integrate()` (and Top-N re-ranking, if active) on every `valueChanged` signal — i.e. once per keystroke or arrow-click — doing potentially expensive recomputation while the user was still mid-edit. 1. Added `_imask_debounce_timer`, a single-shot `QTimer` (120ms), alongside the existing `_refresh_timer` in `__init__`. 2. `_imask_lo`/`_imask_hi` now connect to a new `_on_imask_value_changed()`, which updates the (cheap) mask overlay immediately for visual feedback but only (re)starts the debounce timer for the heavier recompute; the timer's `timeout` fires the existing `_on_imask_changed()` once edits settle. The on/off toggle (`_imask_on`) is unaffected — it still calls `_on_imask_changed()` directly. **Files:** `midas_gui/tab_view.py`. **Roll back:** `git revert 57ced2f`. Self-contained — restores the direct `valueChanged.connect(self._on_imask_changed)` wiring.

`81b8ea8` — Data Viewer: warn when composite mask drops a source (2026-07-30)

Effect: `MaskSelector.composite_mask()` OR's every enabled mask source together, but silently skipped any source that failed to load or whose shape didn't match the union built so far — a user could enable a mask file believing it was in effect while it was quietly excluded. 1. `composite_mask()` now collects the label (and, for shape mismatches, the offending shape) of every dropped source into `self._mask_warning`. 2. `_refresh()` shows the warning in the existing status label (amber `#e0a030`) alongside the normal source count; mutating the source list (add/remove/toggle/`set_tab1_mask`) clears the stale warning until the next `composite_mask()` call recomputes it. **Files:** `midas_gui/widgets.py`. **Roll back:** `git revert 81b8ea8`. Self-contained — restores the silent drop and the plain "N mask source(s)" status text.

`3d96cb1` — Data Viewer: add hardware-free Sim Detector for Live Data testing (2026-07-31)

Effect: Testing the Live Data card required a real EPICS PVA detector stream from beamline hardware — no way to exercise it standalone. 1. New `midas_gui/sim_detector.py`: `SimDetectorServer` runs a real `pvapy.PvaServer` publishing genuine NTNDArray frames (same plumbing a real detector's PVA image plugin uses via `AdImageUtility`), so it's indistinguishable from a real stream to `PvaLiveSource`. Defaults to an Eiger2 500K shape (1030×514 px, matching the repo's existing `DEFAULT_PIXEL_UM` comment), counts in `[0, 60000]`, 5 Hz — all constructor parameters. `ensure_running()`/`stop_all()` give a process-wide registry keyed by channel name. 2. `constants.DEFAULT_DEVICES` gains a "Sim Detector" entry (PV `midasSim:Pva1:Image`); the config-overlay system only round-trips `name`/`prefix`/`pva_suffix` for devices, so the sim device is identified purely by its resolved PV string, not a custom marker key. 3. `DataLoaderPanel._start_live()` (`widgets.py`) lazily starts the sim server via `ensure_running()` the first time that PV is connected to. 4. `DataViewerTab.shutdown()` (`tab_view.py`) calls `sim_detector.stop_all()` alongside the existing `stop_live()`. **Files:** `midas_gui/sim_detector.py` (new), `midas_gui/constants.py`, `midas_gui/widgets.py`, `midas_gui/tab_view.py`. **Roll back:** `git revert 3d96cb1`. Self-contained — removes the sim device/file and its two call sites; no later commit depends on it.

`3b12fbf` — Image viewer: persist manual color-scale window across live frames (2026-07-31)

Effect: `234ded9` (2026-07-29) made a manually-dragged histogram LUT level range survive across incoming frames, but two gaps remained: the histogram's own zoom/pan window still reset on every redraw, and toggling Log/Linear reinterpreted a manual level's raw numbers in the new scale instead of converting them — both made a deliberately-set color window jump around unexpectedly. 1. `ImageViewer` gains `_manual_hist_range` (mirrors `_manual_levels` but for the histogram's own axis zoom) and `_suspend_hist_range_track`, tracked via a new `sigRangeChanged` connection and `_on_hist_range_changed()`. 2. `set_image()` gains a `reset_levels` flag: `True` (default) drops both manual windows and returns to the vmin%/vmax% percentile defaults; `False` — passed for a live-streaming frame update — leaves them in place. `DataLoaderPanel.is_live_frame_update()` reports whether the most recent `dataChanged` came from a live PVA frame, so `tab_view.py` and `tab_calibrate.py` can pass the right value. 3. `_redisplay()` now also re-zooms the histogram to the percentile window by default (`setHistogramRange(lo, hi, padding=0.1)`) so a single bad pixel can't stretch it into an

unreadable sliver. 4. New `_on_log_toggled()` converts `_manual_levels` through log10/10**x when the Log checkbox flips (clamped to ±300 before exponentiating, since a manual level can sit far outside real data range and float `10**x` raises `OverflowError` rather than saturating), and drops `_manual_hist_range` so the view reframes tightly around the converted levels instead of carrying a now-mismatched zoom through the transform.** Files:** `midas_gui/widgets.py`, `midas_gui/tab_view.py`, `midas_gui/tab_calibrate.py`. **Roll back:** `git revert 3b12fbf`. Self-contained — restores the pre-`234ded9`-successor behavior (LUT levels still persist per `234ded9`, but histogram zoom resets and Log/Linear toggle no longer converts levels).

`f3a1b91` — Data Viewer: preserve armed "Use Buffer" state when Start is clicked (2026-07-31)

Effect: `_start_live()` unconditionally called `_reset_buffer()`, which discarded the buffer and flipped the **Use Buffer** button back to its off/gray state even if the user had already armed it (yellow `Buffering…`/green `Buffer Ready`) before clicking **Start**. That forced a redundant second click on **Use Buffer** after every Start. `_start_live()` now only calls `_reset_buffer()` when `_buffer_active` is `False`; when it's already `True`, it re-arms a fresh empty deque (same as `_on_buffer_toggled`'s checked branch) and restyles to `"filling"`, so buffering continues into the new stream without user intervention. **Files:** `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert f3a1b91`. Self-contained — restores the unconditional `_reset_buffer()` call on Start.

`2525277` — Data Viewer: unrestrict wavelength/Lsd/pixel-size ranges, tighten ring row (2026-07-31)

Effect: Ring simulation card's λ/Lsd/pixel-size spin boxes were capped at arbitrary limits (pixel size 1–5000 μm, λ 0.001–10 Å, Lsd 0.001–100000 mm); they now accept any positive value (λ 0.0001–1e6 Å, Lsd 0.001–1e6 mm, pixel 0.1–1e6 μm). Display precision was also tightened per field: λ 5→4 decimals, Lsd 4→3, pixel size/BC_y/BC_z 2→1, ty/tz tilts 4→2. "Show rings" was renamed **Rings** and, together with **Labels** and a shrunk ring-thickness field (max width 60px), now sits on one row instead of thickness alone on the row below. **Files:** `midas_gui/tab_view.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 2525277`. Self-contained — restores the previous ranges/decimals and the two-row Rings/Labels + Ring-thickness layout.

`b7a1518` — Data Viewer: add "?" help button explaining radial-integration calculation (2026-07-31)

Effect: A small circular **"?"** button now sits next to the radial-profile toolbar's `Radial` control; clicking it opens a message box explaining how the R-bin profile is computed — full-geometry (η, R) binning (pixel-count-weighted mean across η) when a calibration is loaded or a tilt is set, versus the circle-binning fallback (plain per-bin mean, no tilt correction) otherwise, and that a failed full-geometry integration falls back to circle binning with a warning above the calibration card. Also widens the ring-width spinbox (60px → 80px) so its value isn't clipped. **Files:** `midas_gui/tab_view.py`. **Roll back:** `git revert b7a1518`. Self-contained — removes the help button and reverts the ring-width spinbox to 60px.

`de15d57` — Data Viewer: move exclude-range controls into radial-plot toolbar, int-safe bounds (2026-07-31)

Effect: The "Exclude out-of-range pixels" controls leave their own left-panel card and move into the radial-integration plot's toolbar, pinned at the far right (past `X`/`Log Y`/`R bin`/`Auto`/`Integrate`); the `R bin` field is narrowed and the `N bins | max=...` stats printout is hidden to make room. The pixel-</> bound spin boxes

widen 84px→112px and switch to integer display (no decimals), with range extended to -1e9..5e9 so 32-bit detector overflow sentinels (e.g. `2**32-1`) fit without a plain `QSpinBox`'s int32 overflow. The lower-bound comparison changes from <= to < (a pixel exactly at the bound is no longer masked). "Load calibration" card renamed **Load/save calibration**. **Files:** `midas_gui/tab_view.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert de15d57`. Self-contained — restores the separate "Exclude out-of-range pixels" card, the float spin boxes, and the <= bound comparison.

`1d45c40` — Data Viewer: pixel-size field allows a second decimal place (2026-07-31)

Effect: The Data Viewer's pixel-size spin box (Ring simulation card) goes from 1 decimal to 2, so pitches like `0.65` μm can be entered exactly instead of rounding to `0.7`. Needed for near-field detector geometries, which use much finer pixel pitches than typical far-field panels. Step size (0.1 μm) is unchanged. **Files:** `midas_gui/tab_view.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 1d45c40`. Self-contained — returns the field to 1 decimal.

`ecfbf36` — Data Viewer: add Box/Circle/Line ROI tool with live floating stats popups (2026-08-01)

Effect: A new **ROI: Box / Circle / Line** row on the image toolbar (alongside **Clear ROIs**) shares the same click-drag gesture as Pick BC/Pick Ring — arming one disarms the others. Click a shape button, then click-drag on the image to draw it (drags shorter than a few pixels are treated as a cancelled attempt, mode stays armed). Drawing a shape opens a small floating, freely-draggable stats popup next to it, color/label-matched to the shape (color cycled from a fixed palette, shared by the on-image shape, its label, and the popup). Box/Circle popups show the full intensity statistics readout (N, percentiles, histogram) scoped to the enclosed pixels; Line popups show a live intensity-vs-distance profile with a direction arrow and a Flip-direction button. Popups recompute on drag/resize and on every frame-navigation/live-frame/correction-change refresh, but their on-screen position is independent of the shape — set once at creation (monitor-aware), never moved automatically afterward. Closing a popup, right-click → Remove ROI, or Clear ROIs removes a shape; ROIs are session-only (not saved with tab state). **Files:** `midas_gui/roi_tools.py` (new — `ROIImageViewer`, `ROIStatsPopup`), `midas_gui/tab_view.py` (Data Viewer's image viewer switched from `PickableImageViewer` to `ROIImageViewer`), `documentation/gui_documentation.md`. **Roll back:** `git revert ecfbf36`. Self-contained — restores the plain `PickableImageViewer` (no ROI tool) and deletes `roi_tools.py`.

`d5654c1` — Data Viewer: ROI tool drops Circle, Clear ROIs resets numbering, Pick Clear also removes BC marker (2026-08-01)

Effect: Three follow-ups to the `ecfbf36` ROI tool: (1) the Circle option is removed from the **ROI:** row — only Box and Line remain; (2) **Clear ROIs** now resets the ROI counter, so the next ROI drawn after a clear starts back at "ROI 1" instead of continuing the prior session's numbering; (3) the **Clear** button shared by **Pick BC**/**Pick Ring** now also removes the Pick BC crosshair marker — previously it only cleared Pick Ring's points/fit-circle, and stayed disabled if only a BC point (no ring points) had been picked, so a lone BC marker couldn't be cleared at all. **Files:** `midas_gui/roi_tools.py`, `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert d5654c1`. Self-contained — restores the Circle ROI option and the old Clear behavior (ring points only).

`bb78c9a` — Data Viewer: line ROI drawn as single arrow shape, no separate arrowhead item (2026-08-01)

Effect: A line ROI's shaft and arrowhead are now built as one `QPainterPath` (`_build_arrow_path`) recomputed from the two endpoints on every drag, instead of a plain shaft line plus a separately positioned/ rotated `pg.ArrowItem`. The old approach could show the arrowhead at an odd angle as the endpoint moved; the new path always points cleanly from one endpoint to the other, with the head size held constant in screen pixels. The ROI's own connecting line is hidden (`roi.setPen(None)`) so it isn't drawn a second time underneath the arrow. **Files:** `midas_gui/roi_tools.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert bb78c9a`. Self-contained — restores the `pg.ArrowItem`-based line-direction indicator.

`786c94c` — Data Viewer: box-ROI popup's zoomed crop image now resizes with the popup (2026-08-01)

Effect: The **Box** ROI popup's zoomed crop `PlotWidget` was `setFixedSize(110, 110)`; it now has a 90x90 minimum size with an Expanding size policy and the same layout stretch factor as the histogram column, so dragging the popup dialog's edge to resize it grows or shrinks the crop image along with the histogram instead of leaving it pinned at its old fixed footprint. **Files:** `midas_gui/roi_tools.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 786c94c`. Self-contained — restores the fixed 110x110 crop image size.

`468b417` — Data Viewer: Projection card gains N-frames cap; λ menu gains energy-to-wavelength entry (2026-08-03)

Effect: The Projection card's **Axis** field (always 0, i.e. across the stack of frames) is removed; a new **N frames** field caps how many frames after **Skip frames** are included in the Max/Sum/Average projection (0 = use all remaining frames) — `ProjectionWorker` gains an `nframes` param and slices `data[:nframes]` after the skip slice, and its info string now reports the frame count used instead of the (always-0) axis. The clickable **λ** label's popup menu (`make_kedge_label`, used on Data Viewer/Calibrate/ PDF) is rebuilt as a `QToolButton` + `QMenu` with an **Energy (keV)** `QWidgetAction` row at the top — typing a value and pressing Enter (or clicking ↵) sets λ = 12.398420/E — followed by the existing K-edge foil list, instead of the plain label-with-menu built by `_clickable_menu_label`. **Files:** `midas_gui/helpers.py`, `midas_gui/tab_view.py`, `midas_gui/workers.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 468b417`. Self-contained — restores the Axis spinbox (dropping N frames) and the plain K-edge-only λ menu.

`c336778` — Data Viewer: B-PILOT bridge auto-starts Live Data on scan dispatch (2026-08-09)

Effect: New `midas_gui/bridge_server.py` opens a `QLocalServer` (`BridgeServer`, socket name `midas_gui_live_bridge_v1`) on app launch — `MainWindow.__init__` starts it and `closeEvent` stops it. B-PILOT (a separate Bluesky plan-runner GUI) connects as a `QLocalSocket` and sends one JSON line per scan dispatch, e.g. `{"type": "live_pv", "version": 1, "prefix": "20IDFF:"}`; `MainWindow._resolve_and_start_live` resolves the prefix against `constants.DEVICES` (`bridge_server.resolve_pv`, matching `prefix` + that device's `pva_suffix`) and, on a match, calls the new `DataViewerTab.start_live_pv` / `DataLoaderPanel.start_live_pv`, which check/ expand the Live Data card, switch streams if a different PV is already running, fill in the PV combo, and click **Start** — no manual clicks needed in MIDAS GUI. Unmatched prefixes or a B-PILOT that never connects are a silent no-op (logged, not raised). Malformed/wrong-version messages are ignored. A stale socket file from a prior unclean shutdown is removed before `listen()` so restarts don't fail with "address already in use". **Files:** `midas_gui/bridge_server.py` (new), `midas_gui/app.py`, `midas_gui/tab_view.py`, `midas_gui/widgets.py`, `tests/test_bridge_server.py` (new), `documentation/gui_documentation.md`. **Roll back:** `git revert`

`c336778`. Self-contained — removes the bridge server and the two `start_live_pv` methods; no other code calls them.

`6c79f13` — Cross-tab data sharing; ROI popups always-on-top/minimizable; Project-stack highlight (2026-08-10)

Effect: New `midas_gui/data_bridge.py` `DataSourceRegistry`, owned by `MainWindow`, is bound by every Data Loader panel (Data Viewer, Calibrate, Calib. Refinement, Batch Integrate, Pump Probe) and by Mask Builder. Each gains an **Import from…** submenu on its Data/Image/Stack browse button, built fresh on open, listing whatever's currently loaded in every *other* bound tab: a file/folder path, or (if that tab's Live Data **Use Buffer** ring buffer is frozen) a **Buffer (N frames)** entry. Picking a path loads it like a normal browse. Picking a buffer either **delegates** live — no copy, tracks the source buffer, clears itself if the source resets (`DataLoaderPanel.use_external_buffer`/`bufferInvalidated`) — for panels that keep an in-memory stack, or **snapshots once** to a temp HDF5 file (new `helpers.new_temp_h5_path`/`save_stack_h5`, dataset `buffer/data`) for panels that only ever read paths (Batch Integrate, Pump Probe stream mode, Mask Builder's Stack field). `FieldSelector` (Dark/Bright/Background) gets the same menu, scoped to its own field type via a new `field` tag on each registry descriptor. `DataLoaderPanel`'s **Use Buffer** row also gains a 💾 button to save the frozen buffer to a user-chosen HDF5 file. Unrelated, bundled from the same session: `ROIStatsPopup` is now `WindowStaysOnTopHint` (so it can't get buried behind other windows) and gains a "–" minimize button that hides the popup and adds an entry to a new `ROIRibbon` strip along the image viewer's left edge (`roi_tools.py`, wired in `tab_view.py`'s `set_ribbon`); clicking the ribbon entry restores it. Data Viewer's **Project stack** button turns green while a projection is displayed (`_apply_project_style`), reverting on **Back to frames** or new data. **Files:** `midas_gui/data_bridge.py` (new), `midas_gui/app.py`, `midas_gui/helpers.py`, `midas_gui/roi_tools.py`, `midas_gui/tab_mask.py`, `midas_gui/tab_view.py`, `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 6c79f13`. Self-contained — removes the registry, all "Import from…" menus, the buffer-save button, ROI always-on-top/ minimize-to-ribbon, and the Project-stack green highlight.

`acb43c1` — Dependencies: upgrade midas-hkls/midas-calibrate-v2, switch midas-pdf to PyPI (2026-08-10)

Effect: `midas-hkls` 0.5.0→0.7.0, `midas-calibrate-v2` 0.5.2→0.5.3. `midas_pdf` is now the real PyPI package (`midas-pdf==0.1.1`) instead of the vendored `midas_gui/_vendor/` copy, so `pdf_backend.py` drops the `midas_hkls.absorption` compatibility shim (needed only while `midas-hkls` lacked that submodule) and the vendored-path `sys.path` insert, replacing both with a plain `import midas_pdf`. `midas_gui/_vendor/` and its package-data block in `pyproject.toml` are removed entirely. Transitive MIDAS deps pulled in by `midas-calibrate-v2`/`midas-pdf` but not imported directly (`midas-integrate`, `midas-peakfit`, `midas-zipper`, `hdf5plugin`, `psutil`) and previously-implicit deps (`numba`, `scikit-image`) are now pinned explicitly in `pyproject.toml`'s `dependencies`, so a plain `pip install .` reproduces the verified-working set without relying on `environment.yml` or another package's `install_requires`. Verified via an isolated venv: pytest 25/25 green plus a full synthetic-image E2E smoke test (auto-seed → one_shot calibration → integration → PDF G(r)) through the GUI's own code paths. **Files:** `environment.yml`, `pyproject.toml`, `midas_gui/pdf_backend.py`, `documentation/gui_documentation.md`; deletes all of `midas_gui/_vendor/`. **Roll back:** `git revert acb43c1`. Restores the vendored `midas_pdf` tree, the `midas_hkls.absorption` shim, and the prior dependency pins — but note `midas-hkls`/`midas-calibrate-v2` would need re-pinning to their older verified versions too if reverting for compatibility reasons, not just to undo the vendoring.

`3058b0c` — Bundle 20-ID-D/20-ID-E/1-ID-E beamline device profiles (2026-08-10)

Effect: The Live Data PV dropdown's device list was hardcoded for 20-ID-D. Ships three bundled `Preferences ▸ Profile` presets — **20-ID-D**, **20-ID-E**, **1-ID-E** — so switching beamlines is a dropdown pick, reusing the existing per-user Profile system (`constants._apply()` already replaces `DEVICES` wholesale when a profile's JSON has a `"devices"` key) with **zero changes** to `prefs_dialog.py` or `widgets.py`. `settings.py` gains a `BUNDLED_PROFILES` literal (one device list per beamline, each entry `{"name", "prefix", "pva_suffix": "Pva1:Image"}`) and `_ensure_profiles()` seeds any not-yet-seen bundled profile once per machine, tracked in a new `profile_meta.json` key `"bundled_seeded"` so a profile a user later deletes isn't silently recreated. 20-ID-D's list is unchanged (already confirmed working on the beamline); 20-ID-E (`pimega`, `spl1`, `s20varex2`, `pg6`, `gh2`) and 1-ID-E (`ge1-ge5`, `pixirad`, `gh1`, `pg1`, `pg5`, `s1varex1`) were extracted from B-PILOT's `instrument/devices/{s20ide,s1id}_devices/*_area_detectors.py` `make_det(...)` blueprints — every device with `pva1_exists=True`. All three profiles also carry a `Sim Detector` entry for hardware-free testing. Device names use B-PILOT's raw variable names (per user decision) rather than invented descriptive labels, so entries stay traceable back to source. Verified: new `test_bundled_beamline_profiles_seeded` plus the full suite (26/26) pass; fresh installs still default to `Default` (same devices as 20-ID-D); an isolated-HOME subprocess check confirmed switching the active profile to each of the three beamlines drives `constants.DEVICES` correctly with no other code changes. **Files:** `midas_gui/settings.py`, `tests/test_config.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 3058b0c`. Self-contained — removes the three bundled profiles and their seeding logic; any profile files a user already has on disk from this feature are untouched by the revert itself (delete them by hand from `<config dir>/profiles/` if desired).

`97ae1ea` — Dependencies: switch PyQt5 to conda-forge (fixes silent startup hang) (2026-08-10)

Effect: `environment.yml` pip-installed `PyQt5==5.15.10`. That wheel bundles its own Qt5 libs but `dlopen`s xcb-side system libraries (e.g. `libxcb-cursor.so.0`, required since PyQt5-Qt5 5.15.9+) at runtime instead of vendoring them. On a beamline workstation this produced a silent startup hang: `python launch.py` never returned to the shell prompt, no traceback, `~/midas_gui_error.log` stopped right after the "starting" / UI-scale lines (i.e. past `_install_diagnostics()` but before `app.exec_()`), and Ctrl+C did nothing — the signature of a native deadlock inside Qt's xcb platform plugin, not a catchable Python exception, so `launch.py`'s own crash-diagnostics wrapper couldn't see it. Same failure class B-PILOT hit and fixed the same way (`bpilot_mpe_dev.yml`, 2026-08-10): moved Qt bindings to conda-forge's `pyqt=5`/`qt=5`, which vendor a self-consistent `libxcb`/`xcb-util-cursor`/`libxkbcommon-x11` stack inside the env instead of relying on the OS. Also drops the `-e .` editable install from `environment.yml`: it wasn't needed (`launch.py` imports `midas_gui` straight off its own directory, no install required), and leaving it in would have reintroduced the bug — `pyproject.toml` still pip-pins `PyQt5==5.15.10`, so `pip install -e .` in this env would re-check that pin against the new conda-forge `pyqt` and could reinstall the pip wheel over it. **Files:** `environment.yml`. **Roll back:** `git revert 97ae1ea`. Restores the pip `PyQt5==5.15.10` pin and the `-e .` editable install — only do this if conda-forge's `pyqt`/`qt` turn out to be unavailable or broken on a target machine, since the pip wheel is what caused the original hang.

`23cd55e` — PDF: rebuild tab for full Stage 2-3 workflow (2026-08-12)

Effect: The PDF tab moves from Stage-1-only (composition-weighted Faber-Ziman S(Q)→G(r)) to the full Stage 2-3 workflow: background/ Paalman-Pings empty-cell absorption subtraction, detector-efficiency correction, absolute normalization, differentiable multiple scattering, a fluorescence diagnostic, CIF-driven structure refinement, and Δ-PDF significance testing, behind a new 4-tab left/right panel layout. Deliberately scoped to exclude Bayesian SVI/NUTS, RMC, SAXS/SANS joint refinement, multi-phase/core-shell, anisotropic ADP, and directional strain-PDF (kept out of `pdf_backend.py`'s re-exports on purpose). Ships a dedicated `test_data/test_pdf/` dataset (raw Varex frames, calibration, a rasterized mask, pre-integrated I(Q) for Ni/CeO_2/IPA/Kapton/air-scatter, and an authored `Ni.cif`), gitignored like `test_data_pump_probe/` (~320 MB) so `constants.py`'s `DEFAULT_PDF_*` point to local-only data — a fresh checkout elsewhere just won't have

the tab preloaded. Every optional stage uses an explicit `QCheckBox` "Enable" toggle rather than a checkable `QGroupBox`, since `widgets_to_dict/apply_dict_to_widgets` (`helpers.py`) don't persist `QGroupBox` checked state. Verified via 26/26 offscreen pytest plus a manual functional pass through the actual `PDFTab` widgets, including a structure fit recovering `a=3.5247 Å` vs. expected `3.524 Å`. **Files:** `midas_gui/constants.py`, `midas_gui/pdf_backend.py`, `midas_gui/tab_pdf.py`, `midas_gui/workers.py`, `.gitignore`, `documentation/gui_documentation.md`. **Roll back:** `git revert 23cd55e`. Self-contained — returns the PDF tab to the Stage-1-only pipeline from `b381d8d`; any `test_data/test_pdf/` already on disk is untouched by the revert itself.

`cc63d5a` — Mask tab: allow multi-file selection for stack source (2026-08-12)

Effect: The Stack browse menu gains a "Files (multi-select)…" entry (`_browse_stack_files`) that opens a multi-select `QFileDialog`, letting users hand-pick exactly which frames feed the temporal-stack methods (spatial-outlier temporal median, temporal constancy, cosmic-ray rejection) instead of only a whole folder or a single file/glob. The chosen list (sorted for a deterministic frame order) is stored in a new `self._stack_files` attribute that takes priority in `_collect_stack_paths()`; the stride spinner still applies to it. Picking Folder/File or typing a path clears `_stack_files` via the existing `_on_stack_path_changed` handler, so the two input modes can't conflict. Persisted across GUI-state save/load. Verified via a full offscreen pytest run (pre-existing `test_app_builds_offscreen` flakiness and a full-suite-only pyqtgraph teardown segfault in the unrelated PDF tab both reproduce identically on unmodified `main`, confirming neither is caused by this change). **Files:** `midas_gui/tab_mask.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert cc63d5a`. Self-contained — the "Files (multi-select)…" menu entry and `_stack_files` state disappear; stack input reverts to folder/file/glob only.

`429d41a` — Mask tab: add configurable bad-pixel dilation (2026-08-12)

Effect: A new "5 · Post-processing" card adds a **Dilation (px)** spin box (default 0, range 0-50). `_set_mask()` — the single point all mask-producing paths (threshold-only short-circuit, `MaskComputeWorker` result, mask loaded from disk) converge through before merging with hand-drawn shapes — now runs `scipy.ndimage.binary_dilation` with `iterations=N` on the *computed* mask when $N > 0$, before it's OR'd with `self._drawn_mask` in `_emit_final()`. The default 4-connected structuring element with N iterations gives exactly the requested semantics: dilation=1 marks every pixel directly touching a bad pixel, dilation=2 adds one further ring, etc.; hand-drawn shapes are never grown. `scipy` was already a pinned dependency and `scipy.ndimage` already used the same way in `workers.py`. Persisted via `_state_widgets()`. Verified with a targeted script instantiating `MaskTab` offscreen and asserting exact pixel counts at dilation 0/1/2 plus that a hand-drawn pixel is excluded from growth at dilation=5. **Files:** `midas_gui/tab_mask.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 429d41a`. Self-contained — removes the Post-processing card and the dilation step in `_set_mask()`.

`188ea77` — Mask tab: switch dilation to 8-neighbor (full-block) growth (2026-08-12)

Effect: `_set_mask()`'s `binary_dilation` call now passes an explicit 3×3 full structuring element (`np.ones((3,3), dtype=bool)`) instead of relying on scipy's default 4-connected (diamond) structure. With `iterations=N`, this changes the growth semantics from "N rings of 4-connected neighbors" to "the full $(2N+1)\times(2N+1)$ square centered on each bad pixel" — dilation=1 now marks the entire 3×3 block around a bad pixel, dilation=2 the entire 5×5 block, matching the requested 8-neighbor behavior instead of the original commit's 4-connected one. Hand-drawn shapes are still never grown (same insertion point as `429d41a`). Verified with a targeted offscreen script asserting exact pixel counts (9 at dilation=1, 25 at dilation=2 on an isolated bad pixel) and that a hand-drawn pixel outside the dilation radius is unaffected. **Files:** `midas_gui/tab_mask.py`,

documentation/gui_documentation.md. **Roll back:** git revert 188ea77. Self-contained — restores the 4-connected binary_dilation default from 429d41a.

---

b2616c6 — Data Viewer: ROI tool usability improvements (2026-08-13)

Effect: Five rough edges in roi_tools.py's ROI drawing/popup tooling, all requested together as a Data Viewer ROI cleanup pass: 1. ROIStatsPopup's custom "–" minimize QToolButton is removed; a new changeEvent/_reject_native_minimize pair intercepts the OS-level (title-bar) minimize via QEvent.WindowStateChange + Qt.WindowMinimized, deferred through QTimer.singleShot(0, ...) to avoid re-entering Qt's window-state machinery, and routes it into the same "tuck into the ROIRibbon" behavior the old button had. 2. Box ROIs get three new pg.TextItems (coord_item/width_item/ height_item) showing the top-left (x, y) corner, w =, and h = in the ROI's color, repositioned/retexted on every sigRegionChanged (move *and* resize) via a new _update_roi_annotations(). Line ROIs are unaffected. 3. New shared _make_roi_text_item() helper gives every on-image ROI text item (the existing "ROI N" label included) a translucent background via pg.TextItem(fill=...) and a font ~20% larger than the app default (QFont.pointSize(), confirmed reliable for pg.TextItem's internal QGraphicsTextItem — unlike real QWidgets under this app's QSS stylesheet, which report -1 there). 4. ROIImageViewer.set_bad_mask() (wired from tab_view.py's existing _update_intensity_overlay() — the sole choke point already covering initial load, live frames, mask toggle, threshold edits, and autofill) feeds the active bad-pixel mask into _refresh_roi_stats(): box-ROI histogram/crop stats now exclude bad pixels (intersected at full resolution) and the popup's crop image gains a _bad_overlay pg.ImageItem marking them translucent red, matching the main viewer's set_mask_overlay() convention; line-ROI stats null out masked samples (set_line_profile shows "(all pixels masked)" if every sample is excluded, instead of raw NaNs). 5. Dragging a new line ROI previously rendered as a QRubberBand.Line, which can only paint an axis-aligned bounding box (the .Line shape only changes pen style, not the geometry it's constrained to) — replaced with a real QGraphicsLineItem dropped into the ViewBox via self._iv.addItem(..., ignoreBounds=True), the same pattern already used for the line-ROI's persistent arrow overlay, so the live preview is a true diagonal. A shared _map_drag_to_view() helper replaces the old inline scene-mapping in _finish_roi_draw; a new _abort_roi_drag() cleans up whichever preview item is live if a draw mode is turned off mid-drag. Box-mode's QRubberBand is unchanged. Verified with targeted offscreen scripts: box annotation text/position at known geometry, bad-mask-aware histogram/crop-overlay shape, an all-masked line producing the new stats string, and a simulated line drag producing a QGraphicsLineItem with non-degenerate dx/dy (a true diagonal) that's cleaned up on release. Full pytest suite: 24/24 pass excluding the two known pre-existing, unrelated issues already logged in .context/STATE.md (test_smoke.py::test_app_builds_offscreen config flakiness and a full-suite-only pyqtgraph teardown segfault in tab_pdf.py), both of which reproduced identically and unchanged in this session. **Files:** midas_gui/roi_tools.py, midas_gui/tab_view.py, documentation/gui_documentation.md. **Roll back:** git revert b2616c6. Self-contained — restores the custom minimize button, drops the corner/width/height annotations and translucent/larger text styling, drops mask-awareness from ROI stats (ROIImageViewer.set_bad_mask/the tab_view.py wiring), and restores the QRubberBand.Line drag preview.

a1a0b09 — Data Viewer: ROI usability follow-ups (2026-08-13)

Effect: Real-usage follow-up fixes on top of b2616c6, all in roi_tools.py plus one existing shared panel in widgets.py: 1. Fixed the OS-level minimize "pops back open" glitch: _reject_native_minimize() previously called setWindowState (clear Qt.WindowMinimized) *before* hide(), which asks the OS to reverse the minimize and starts a native un-minimize animation that hide() can lose a race against — once the animation completed the popup ended up visible again despite the ribbon entry already existing (fixed by the user clicking minimize a second time). Now hide() runs first (unconditionally takes the window off-screen) and the state bit is cleared afterward (invisible, since the window is already hidden); a changeEvent isVisible() guard and a defensive state-clear before _on_roi_restore's show() round it out. Native minimize/animation behavior can't be exercised under offscreen QPA — flagged for a real windowed manual check. 2. Box ROI (x, y)/w =/h = on-

image annotations (_update_roi_annotations) now use round(...) instead of f"{v:.1f}" — whole pixels only, no fractional-pixel values. 3. Line ROIs get a new length_item pg.TextItem (created in _register_roi, alongside the existing arrow), positioned at the line's midpoint and retexted (f"{round(length)} px") on every sigRegionChanged inside _update_line_arrow. 4. Fixed the line-ROI "ROI N" label always appearing at the image's top-left corner: pg.LineSegmentROI.pos() is always (0, 0) (its geometry lives in listPoints(), a local-frame handle list, not an item-level offset the way RectROI works) — _on_roi_geom_changed's generic entry["label_item"].setPos(entry["roi"].pos()) therefore parked every line label at the origin. Box ROIs keep that path unchanged; line ROIs now get their label positioned inside _update_line_arrow instead, reusing the already-computed, flip-aware view_p0 (the line's actual mapped-to-parent start point). 5. New shared _apply_view_limits(plot_widget, xmin, xmax, ymin, ymax, clamp_xmin_zero=, clamp_ymin_zero=) helper (same formula as ProfileViewer._apply_view_limits in widgets.py, the radial- integration plot) applied to the box-ROI histogram (_redraw_hist) and the line-ROI profile plot (set_line_profile), so neither can be zoomed/panned arbitrarily far from the current data. widgets.py's IntensityStatsPanel._redraw_hist — which already had a partial, pan-only version (xMin/yMin only) — was brought to parity with xMax/maxXRange/maxYRange. 6. Gitignored test_data_gitignore/ (large local-only functional-test dataset the user keeps on disk for manual GUI testing but never wants pushed). Verified with targeted offscreen scripts (line label anchored at the true start point and swapping correctly on flip, length text/position matching a known line geometry, box coord rounding) plus the full pytest suite — no new failures beyond the same two known pre-existing issues logged in .context/STATE.md (test_app_builds_offscreen config flakiness, and a full-suite-only pyqtgraph teardown segfault in tab_pdf.py), both reproduced identically and unchanged. **Files:** midas_gui/roi_tools.py, midas_gui/widgets.py, .gitignore, documentation/gui_documentation.md. **Roll back:** git revert a1a0b09. Self-contained — restores the old minimize-then-hide ordering (glitch returns), the one-decimal box annotations, drops the line-length text item, restores the old roi.pos()-based line-label positioning (top-left-corner bug returns), drops the popup-plot/Intensity-histogram zoom-out caps, and un-ignores test_data_gitignore/.

1181b58 — Data Viewer: bump ROI/histogram axis label font size (9pt -> 12pt) (2026-08-13)

Effect: Readability tweak — the axis labels ("distance (px)"/"intensity" on the ROI stats popup's line-profile plot, "intensity"/"count" or "log(count+1)" on the ROI popup's histogram, and "intensity"/"log(count+1)" on IntensityStatsPanel's histogram) go from 9pt to 12pt. No layout, behavior, or data changes. **Files:** midas_gui/roi_tools.py, midas_gui/widgets.py. **Roll back:** git revert 1181b58. Self-contained — restores the 9pt axis label font size.

cc6e90e — Calibrate tab: honor partial distortion-coefficient selection, live-update dark/bright/background preview (2026-08-16)

Effect: Two Calibrate-tab (Tab 2) fixes: 1. The **Distortion (n/15)** "…" per-coefficient dialog previously only affected the Four-stage/advanced pipelines — One-shot silently ignored a partial selection and refined all 15 coefficients regardless. calib.py's run_pipeline() now detects a strict subset selection on the one_shot path and routes it through midas_calibrate_v2.pipelines.single. autocalibrate (the same lower-level, per-p# Refine-dict routine the other pipelines already use) instead of the high-level calibrate(), which only exposes an all-or-nothing refine_distortion bool. normalize_result() gained a matching branch (raw.unpacked present but no raw.stage2) to normalize the resulting CalibrationResult the same way first_time's pv.unpacked case already does. Selecting "All (15)" or leaving Distortion unchecked still runs the normal One-shot path. 2. The Results tab's parameter grid now lists only the distortion coefficients actually selected for the run that produced the displayed result (_last_dist_coeffs, captured when the run starts), instead of always showing all 15 p0–p14 slots — so the display matches what was asked to refine. The saved paramstest.txt/.json exports are unaffected and still carry every slot's real value (including any legitimately held-fixed nonzero value carried over from a prior calibration). 3. Unrelated same-commit fix: picking or changing a Dark, Bright, or Background field in the shared Data Loader panel now immediately refreshes the calibration image preview to the corrected frame (DataLoaderPanel.fieldsChanged wired to a new

_on_fields_changed, mirroring the Data Viewer tab) — it previously kept showing the raw, uncorrected frame until the next full data load. The raw image (self._image) is untouched; only the on-screen render changes, since CalibrationWorker still needs the raw frame for the actual pipeline run (it and the backend apply bright/background/dark themselves). Verified via the full pytest suite — no new failures beyond the same two known pre-existing issues logged in .context/STATE.md (test_app_builds_offscreen config flakiness, and a full-suite-only pyqtgraph teardown segfault in tab_pdf.py), both reproduced identically and unchanged. No dedicated automated test covers the LM-fit coefficient routing or the live preview refresh (no offscreen harness drives an actual calibration run or a Data Loader field pick in this suite). **Files:** midas_gui/calib.py, midas_gui/tab_calibrate.py, documentation/gui_documentation.md. **Roll back:** git revert cc6e90e. Self-contained — restores One-shot's all-or-none distortion refinement, the always-show-all-15 Results grid, and the raw (uncorrected) calibration preview until the next full data load.

068bd0d — Add MIDAS ImTransOpt (image flip/transpose) support to Data Viewer + Mask tabs, persist in calibration files (2026-08-17)

Effect: MIDAS's ImTransOpt image-transform parameter (repeatable integer code in .txt parameter files — 1=flip-Y, 2=flip-Z, 3=transpose, applied in file order, interpreted *before* geometry/BC/Lsd) was previously only half-wired in: the Calibrate tab had three checkboxes ("Flip Y" / "Flip Z" / "Transpose") that built an in-memory codes list applied to the calibration image/dark/bright/background via the shared _apply_im_trans(), but the codes were never written to or read back from any saved calibration file, and the Data Viewer and Mask Builder tabs had no transform controls at all. 1. **midas_gui/helpers.py**: added parse_im_trans(text) (reads repeatable ImTransOpt <code> lines from a paramstest, dropping an explicit 0 no-op) and im_trans_codes_from_checkboxes(flip_y, flip_z, transp) (the fixed flips-then-transpose ordering, shared by all three tabs — replaces two previously-duplicated implementations in tab_calibrate.py). read_geometry() and geometry_fields_from_file() both now return an im_trans key ([] if absent) for all three supported formats (paramstest, .json, .poni — the latter always [], no pyFAI equivalent). write_standalone_paramstest() now appends one ImTransOpt <code> line per code from getattr(result, "im_trans", None) after the main file write (same append-after-write pattern already used for PanelShiftsFile). 2. **Data Viewer** (tab_view.py): new "Transforms:" checkbox row in the Ring-simulation/geometry card. Applied at all three points the tab computes its current working frame (_on_loader_data, _on_fields_changed, _on_projection_done — the latter caches the untransformed projection output in a new self._proj_raw so toggling a checkbox mid-projection recomputes from the untransformed source instead of compounding transforms onto an already-transformed image). get_geometry()/set_geometry() (the Data Viewer <-> Calibrate push/pull) and _export_geom()/_save_calibration() (JSON/paramstest export) all carry im_trans now; _load_calibration() restores the checkboxes from a loaded file's ImTransOpt/im_trans. 3. **Mask Builder** (tab_mask.py): same "Transforms:" row under the Image card, applied in the single-image _load_image() and passed into MaskComputeWorker (new im_trans constructor kwarg) for its multi-file and HDF5 stack-source loading (midas_gui/workers.py, applied per-frame in both branches). set_calibration() (the Calibrate -> Mask hand-off) pre-checks Mask's boxes to match the incoming result.im_trans (still user-overridable). 4. **Calibrate tab** (tab_calibrate.py): the two duplicated checkbox-to-codes blocks now call the shared helper; result.im_trans is set when a run completes (_on_done) so it flows into _save_json() (already dumps all non-underscore vars(result), so no change needed there), _save_paramstest()'s standalone path (via the write_standalone_paramstest() change above) and its template path (ff_paramstest_from_auto_result() is external and untouched — ImTransOpt lines are appended manually afterward, the same way PanelShiftsFile already was), and the on-screen paramstest preview grid (_paramstest_pairs(), unchanged — it already renders whatever write_standalone_paramstest() produces). _load_calib_file() and apply_geometry() (Data Viewer -> Calibrate) restore the checkboxes from a loaded/pushed im_trans, logging a note if a file's ImTransOpt order doesn't match the fixed checkbox order (only matters when transpose is combined with a flip, since flips commute with each other but not with transpose — no real example file in the MIDAS repo does this). 5. All three tabs' _state_widgets() (Save/Load GUI State) gained flip_y/flip_z/transp entries (Calibrate already had them). **Verified:** new tests/test_im_trans.py (19 cases — parsing, write/read round-trip, the checkbox-to-codes helper for all 8 combinations, _apply_im_trans

composition order) plus targeted offscreen scripts confirming Data Viewer/Mask checkbox toggles actually flip/transpose the loaded array, and a full Calibrate-tab save-paramstest -> reload -> checkbox-state round trip. Full `pytest` suite: 43 passed, 2 deselected (the same pre-existing `test_app_builds_offscreen` config flakiness and full-suite-only `tab_pdf.py` pyqtgraph-teardown segfault logged in `.context/STATE.md`, both reconfirmed identical on unmodified `main` before this change). **Files:** `midas_gui/helpers.py`, `midas_gui/tab_calibrate.py`, `midas_gui/tab_view.py`, `midas_gui/tab_mask.py`, `midas_gui/workers.py`, `tests/test_im_trans.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 068bd0d`. Self-contained — restores the Calibrate-tab-only, in-memory-only `ImTransOpt` behavior and removes the Data Viewer/Mask transform controls.

`73a7a2a` — Calibrate tab: Transforms checkboxes now live-update preview and drive the actual calibration run (2026-08-17)

Effect: `068bd0d` made the Calibrate tab's Transforms checkboxes round-trip through saved/loaded calibration files, but toggling one still did nothing to the on-screen preview or the actual calibration run — the displayed/picked-on image and the array handed to the pipeline stayed untransformed regardless of the checkboxes. Two fixes: 1. **Live preview + picks.** The three checkboxes now call `_on_im_trans_changed()` on toggle, which re-renders `_show_calib_image()` through the same `_apply_im_trans()` helper Data Viewer/Mask use. Since Pick BC / Pick Ring read coordinates straight off the displayed `PickableImageViewer` array, picks now automatically land in the transformed coordinate space instead of the untransformed one — matching Data Viewer and Mask Builder's behavior. 2. **Actual calibration run.** `CalibrationWorker.run()` previously passed `cfg["im_trans"]` straight to `calib.run_pipeline()`, which only applies it internally on some code paths — Four-stage, Bayesian, Joint-cake, panel-layout, and partial-distortion-coefficient runs (`cc6e90e`) all silently ignored it, so the fitted beam centre/detector dimensions never matched a transformed image. The worker now applies `_apply_im_trans()` to the calibrant image and to Dark itself, once, right before the pipeline call, then clears `cfg["im_trans"]` so `run_pipeline`'s own internal transform (meant for callers handing it a raw image directly) doesn't double-apply it. 3. **Send to Data Viewer** now also carries `im_trans` in the geometry dict handed to Data Viewer, so its own Flip/Transpose checkboxes sync to match the calibration that produced the sent geometry (previously only λ/pixel size/Lsd/BC/tilts/distortion made the trip). 4. Unrelated same-commit cleanup: the Average frames card's **skip** (stride) spinbox was removed — it added a rarely-used degree of freedom that made the averaged-frame count harder to reason about; **start/ end (0=all)** remain. **Verified:** full `pytest` suite — 43 passed, 2 deselected (the same two pre-existing issues logged in `.context/STATE.md`: `test_app_builds_offscreen` config flakiness and the full-suite-only `tab_pdf.py` pyqtgraph-teardown segfault). No dedicated automated test covers the live preview refresh or the worker's transform-before-pipeline change (no offscreen harness drives an actual calibration run in this suite). **Files:** `midas_gui/tab_calibrate.py`, `midas_gui/workers.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 73a7a2a`. Self-contained — restores the untransformed live preview/picks, `run_pipeline`'s own (partial) internal transform handling, the Send-to-Data-Viewer payload without `im_trans`, and the Average frames skip/stride control.

`683d150` — Add pip requirements.txt mirroring environment.yml + README pip install path (2026-08-21)

Effect: `midas-gui` previously only documented a conda install path (`environment.yml`); users without conda had no supported way to reproduce the pinned dependency set with plain pip. Added `requirements.txt` at the repo root, generated from `environment.yml`'s pinned versions (matches the same versions already declared in `pyproject.toml`'s `dependencies`), and a new "Alternative: pip only (no conda)" section in the README with the `venv` + `pip install -r requirements.txt` commands. Carried over both of `environment.yml`'s load-bearing warnings as comments: the numpy 1.26.4 / numba 0.59.1 / torch 2.4.0 NumPy-1 compatibility pin, and the PyQt5 pip wheel's known xcb-library silent-startup-hang risk on Linux (with the `apt install libxcb-cursor0` workaround) versus conda-forge's self-contained Qt build. **Verified:** `pip install -r`

`requirements.txt` was not run against a fresh environment in this session (no isolated network/venv sandbox available here); version pins were cross-checked line-by-line against `environment.yml` and `pyproject.toml`'s `dependencies` list instead, which already match. **Files:** `requirements.txt` (new), `README.md`. **Roll back:** `git revert 683d150`. Purely additive/documentation — safe to revert standalone; removes the pip install path and leaves conda as the only documented route.

`c8d98f1` — Test data: reorganize test_data/ into per-dataset subfolders (2026-08-23)

Effect: `test_data/` mixed the shipped synthetic sample set with several local-only large datasets scattered across differently-named top-level dirs (`17BM/`, `test_s25ide/`, and the sibling `test_data_pump_probe/` outside `test_data/` entirely), making it unclear from the name alone which folders ship with the repo and which are git-ignored local assets. Renamed the shipped synthetic data (`calibrant_ceria.*`, `nickel_stack.h5`, `nickel_tifs/`, `make_test_data.py`) into `test_data/gui_synthetic/`, and the large local-only sets into `test_data/s17bm/`, `test_data/trr_s25ide/`, and `test_data/trr_s7id/` (the latter absorbing what was `test_data_pump_probe/` at the repo root). `.gitignore`'s shipped-data re-includes now scope to `test_data/gui_synthetic/**` only, so every other `test_data/` subfolder stays git-ignored by default; the standalone `/test_data_pump_probe/` and `/test_data_gitignore/` ignore rules were removed since those paths no longer exist. `constants.py`'s Pump Probe tab default path (`DEFAULT_TRXRD_DIR`) and `gui_documentation.md`'s reference to it were updated to `test_data/trr_s7id/pump_probe_BTO/detimages/`. **Verified:** offscreen 9-tab app build + full pytest suite still pass (same pre-existing `test_app_builds_offscreen` config flakiness as noted in `.context/STATE.md`, unrelated to this change); confirmed no other source file referenced the old `test_data_pump_probe/`, `test_data/17BM/`, or `test_data/test_s25ide/` paths before renaming. **Files:** `.gitignore`, `midas_gui/constants.py`, `documentation/gui_documentation.md`, `test_data/**` (renames only, no content changes). **Roll back:** `git revert c8d98f1`. Self-contained rename/path change — restores the old top-level `test_data_pump_probe/`, `test_data/17BM/`, and `test_data/test_s25ide/` layout and the matching `.gitignore`/`constants.py` paths.

`246dba7` — Viewers: flip detector-image origin to bottom-left (0,0), per MIDAS convention (2026-08-23)

Effect: Every `pg.ImageView`-based viewer (Data Viewer, Mask Builder, Calibrate) was drawing pixel `(0, 0)` at the top-left, because `pg.ImageView.__init__` calls `view.invertY()` internally. MIDAS's own convention places detector `(0, 0)` at the bottom-left, so the on-screen image now matches the physical world view of the detector looking downstream from the sample along the beam. Fixed with a single `vb.invertY(False)` override in `ImageViewer.__init__` (`widgets.py`), which propagates to `PickableImageViewer`/`ROIImageViewer` since both subclass it — covering all three viewers in one place. `roi_tools.py`'s ROI crop-preview popup (`ROIStatsPopup`) and the Data Viewer box-ROI corner-coordinate annotation/docstrings were updated in lockstep, since both were explicitly compensating for (or documenting) the old top-left-origin default. This is a pure display-orientation flip: it does not change click-to-pixel math (`mapSceneToView`/`mapFromScene` are invert-aware), Calibrate's ring/tilt geometry math, or the separate `ImTransOpt` data-level transform (the Transforms: Flip Y/Flip Z/Transpose checkboxes), which still operate on the underlying pixel data independent of which corner is rendered as the origin. **Verified:** confirmed via an offscreen render-and-inspect-pixmap script (not just coordinate round-trips, which can't prove the visual direction) across all three viewers plus the ROI crop popup, and a full pytest re-run (44 passed, 1 known pre-existing failure per `.context/STATE.md`). Not yet confirmed with a real windowed manual pass — recommended as a follow-up, especially Calibrate's ring overlay and the Data Viewer's ROI crop popup. **Files:** `midas_gui/widgets.py`, `midas_gui/roi_tools.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 246dba7`. Self-contained — restores the top-left-origin display convention and the matching ROI-popup/box-corner- annotation compensation.

`3b785c9` — Fix stale DEFAULT_* test-data paths after gui_synthetic/ reorg (2026-08-23)

Effect: The `246dba7`/`c8d98f1` reorg session moved `calibrant_ceria.tif`, `calibrant_ceria.h5`, `nickel_stack.h5`, `nickel_tifs/`, and `calibration_synthetic.json` into `test_data/gui_synthetic/`, but `constants.py`'s six `DEFAULT_*` constants (`DEFAULT_CALIBRANT_TIF`, `DEFAULT_CALIBRANT_H5`, `DEFAULT_NICKEL_H5`, `DEFAULT_NICKEL_DIR`, `DEFAULT_NICKEL_FRAME0`, `DEFAULT_CALIB_FILE`) still pointed at the old repo-root locations — silently breaking every tab's default-data preload on a fresh checkout (Data Viewer, Calibrate, Mask Builder, Batch Integrate, Corrections, PDF, Texture all reference one or more of these). Added a `_GUI_SYNTH = _TEST_DATA / "gui_synthetic"` base and repointed the six constants at it. Found while verifying the new Hydra-mode `DetectorGeometryCard` extraction (see the following commit) — unrelated to that work, fixed as its own commit per project convention. **Note:** a locally-*cached* profile (e.g. `~/Library/Application Support/midas_gui/profiles/<name>.json`, seeded from the old shipped defaults before this fix) can still override these with the stale paths at runtime via `constants.reload_from_config()`'s config-overlay step — that's local machine state, not a repo file, and isn't touched by this commit. A user hitting this after upgrading should reset/re-seed that profile (or manually fix its `paths` section) to pick up the corrected defaults. **Verified:** confirmed all six paths resolve and exist under `test_data/gui_synthetic/` with the fix; full pytest re-run (same single pre-existing `test_app_builds_offscreen` flakiness, unrelated). **Files:** `midas_gui/constants.py`. **Roll back:** `git revert 3b785c9`. Self-contained — restores the stale repo-root paths (breaking default-data preload again).

`3d2d99d` — Data Viewer: add Hydra-mode ribbon scaffold; extract DetectorGeometryCard (2026-08-23)

Effect: First step of the Hydra (4-panel GE detector) viewer feature. Added a leftmost vertical mode ribbon (`HydraModeRibbon`, `hydra_widgets.py`) to the Data Viewer tab with two exclusive modes: **Single detector** (today's existing view — now "page 0" of a new `QStackedWidget`, behavior unchanged) and **Hydra** (a placeholder "coming soon" page). Also extracted the ring-simulation, calibration load/save, and radial-integration logic (~700 lines: materials list, `MaterialDialog`, tilted-ring drawing, beam-centre pick/ring-fit handling, the MIDAS-engine and circle-binning radial integration paths, calibration file read/write) out of `DataViewerTab` into a new reusable `DetectorGeometryCard` widget (`hydra_geometry_card.py`). The card is bindable rather than hard-wired — `set_viewer`/`set_profile_view`/`set_image_source`/ `set_radial_controls` let it attach to whichever image viewer, radial plot, and frame source it should currently act on, and `set_viewer` cleanly clears its overlays off a previous viewer before attaching to a new one — so the same class can later be instantiated once per Hydra panel (ge1-4 + composite) without duplicating this logic five times. `DataViewerTab` keeps its external API (`get_geometry`/`set_geometry`/`pushGeometry`) as thin delegates to the card, so `app.py`'s Data-Viewer-to-Calibrate geometry hand-off wiring needed no changes. Saved-GUI-state JSON shape is unchanged (the card's fields are merged into the same flat `fields` dict Save/Load GUI State already used) for backward compatibility with existing saved sessions. **Verified:** the extraction (page 0) was checked against the pre-refactor code with an offscreen pixel-diff of the rendered tab (ring simulation + calibration load + radial integration all exercised) — only a 6px-wide cosmetic sliver at the very left edge differs (a margin artifact from the new ribbon itself, not a content regression); full pytest re-run (same single pre-existing, unrelated `test_smoke.py` `visible_tabs` flake). The Hydra page itself is not yet functional — no visual/behavioral change there beyond the placeholder label. **Files:** `midas_gui/tab_view.py`, `midas_gui/hydra_geometry_card.py` (new), `midas_gui/hydra_widgets.py` (new), `documentation/gui_documentation.md`. **Roll back:** `git revert 3d2d99d`. Self-contained — restores the single-page Data Viewer tab with the ring-sim/calibration/radial logic inline.

`dd44f38` — Hydra: geometry-compositing engine + sibling auto-discovery + tests (2026-08-23)

Effect: Adds `midas_gui/hydra.py`, the windmill-compositing math for the 1-ID-E Hydra detector (4x GE panels): `compute_inv_coords`/ `remap_to_composite`/`composite`/`autopick_big_det_size`/`DetectorState`/ `build_windmill_composite`, ported from a sibling project's own port of JSP's `multidet.py`.

DetectorState.load_from_geometry_dict consumes this repo's own helpers.geometry_fields_from_file (already used by the Calibrate/Data-Viewer calibration-load UI) instead of a bespoke per-panel parser — the ported files' bundled param-file parser was intentionally not carried over. Bundled default geometry (hydra_default_geometry/ps_ge{1..4}.txt, a generic 1-ID-E-style example windmill layout) seeds a panel with no calibration loaded yet. Adds helpers.hydra_panel_index/hydra_siblings — given any one GE panel's file path, finds the other 3 by substituting the geN token (handles both a geN/ folder segment and a .geN. filename infix, this beamline's actual naming convention). Fixed a latent staleness bug in build_windmill_composite's BigDetSize cache (found during testing): it was keyed only by which panel numbers were present, so it kept returning a stale canvas size after a panel's geometry changed (e.g. a new calibration file) for the same panel set — now keyed by each panel's actual BC/tx/px/size too. **Verified:** cross-checked against real local (uncommitted, gitignored) test_data/s1ide CeO2 Hydra data — the composite matches the reference project's own reported result (6656px canvas, 0% NaN, correct pinwheel arrangement), and rendering with the real fitted per-panel calibration shows Debye-Scherrer rings stitching into continuous arcs across all 4 panel boundaries — the physically-grounded confirmation that no chirality/mirroring error exists. This resolved an open question from the reference project (whose own custom viewer needed an X-mirror for its composite) — this codebase's pg.ImageView-based viewers need no such correction; full reasoning in .context/DECISIONS.md. Added a committed synthetic fixture (test_data/gui_synthetic/hydra/, 4 small 256x256 panels with a known idealized geometry, tx values deliberately not exact multiples of 90° so a sign error can't alias one panel's marker onto another's) and two new permanent test files: tests/test_hydra_geometry.py (12 tests: sibling discovery, DetectorState, bundled-default parsing, an independently-hand-derived-math marker-position check) and tests/test_hydra_chirality.py (the same marker check rendered through the real ROIImageViewer via widget.grab() — a permanent regression guard, not just a one-off manual check). Full pytest suite: same single pre-existing, unrelated test_smoke.py flake. **Files:** midas_gui/hydra.py (new), midas_gui/hydra_default_geometry/* (new), midas_gui/helpers.py, test_data/gui_synthetic/hydra/* (new), tests/test_hydra_geometry.py (new), tests/test_hydra_chirality.py (new). **Roll back:** git revert dd44f38. Self-contained — no other code references hydra.py yet (the Hydra page UI that will consume it lands in a follow-up commit).

778a2f3 — Commit .context/ per global policy (2026-08-23)

Effect: .context/ (STATE/DECISIONS/ARCHITECTURE/DOMAIN/ROADMAP + the midas_pdf reference stack) had been listed in .gitignore since this repo's early history, contradicting the standing global-CLAUDE.md rule that this folder always travels with the repo (so accumulated project knowledge/ decisions survive across machines and clones). Removed the .gitignore rule and committed the folder as-is. Also formalized /test_data/s1ide/ (real, local-only CeO2 Hydra calibration data, used read-only this session with explicit user approval for the Hydra compositing verification — see dd44f38) as gitignored, matching how it had already been treated all along (never committed, never a pytest dependency). **Files:** .gitignore, .context/* (new). **Roll back:** git revert 778a2f3. Self-contained, though reverting would re-orphan .context/ from version control going forward.

1cc4dab — Hydra: wire loader/toolbar/per-panel calibration UI + composite into the tab (2026-08-23)

Effect: Replaces the Hydra page placeholder (from 3d2d99d) with a working HydraViewerPage. New HydraLoaderPanel (one path field auto-discovers all 4 GE panels via helpers.hydra_siblings, plus a shared frame navigator) and HydraDetectorToolbar (ge1-4/composite buttons, enabling only buttons for panels actually found) in hydra_widgets.py. HydraViewerPage (hydra_page.py) owns one hydra.DetectorState and one DetectorGeometryCard per panel, plus a DetectorGeometryCard for the composite. Switching the toolbar rebinds the shared ROIImageViewer/ ProfileViewer to the newly-active card (via set_viewer/ set_profile_view — exactly what the 3d2d99d extraction was built to support) and displays that panel's raw frame or the geometry-based composite. Loading a calibration file (or editing/picking BC) on a ge-card now

syncs into its `DetectorState` and invalidates the composite so it rebuilds with the new geometry; the composite's own card auto-seeds its beam centre at the canvas centre (`BigDetSize/2`) the first time a given canvas size appears, so it has a working ring overlay for free. Added `DetectorGeometryCard.geometryChanged` signal + `get_full_geometry()` accessor for this syncing. `DataViewerTab.get_state()`/`set_state()` now folds in `HydraViewerPage`'s own state (loader path, active panel, every card's fields/materials). **Scope cuts for this version** (documented, not oversights): radial integration reuses the existing single-curve `ProfileViewer` (shows whichever panel/composite is active, one shared R-bin/Auto/Integrate control set across all 5 cards) rather than the eventual 4-curves + toggleable-composite plot; no dark/bright/mask corrections for Hydra frames yet (raw frames only). **Verified:** end-to-end with the synthetic fixture (`test_data/gui_synthetic/hydra/`) — loading matched per-panel calibration into all 4 cards rebuilds the composite at the expected canvas size with the composite card's BC correctly auto-seeded at centre; every ge1-4/ composite button displays the right frame with independent ring simulation and radial integration; an offscreen screenshot confirms the full page renders and functions together. New `tests/test_hydra_ui.py` (4 tests). Full pytest suite: same single pre-existing, unrelated `test_smoke.py` flake. **Files:** `midas_gui/hydra_page.py` (new), `midas_gui/hydra_widgets.py`, `midas_gui/hydra_geometry_card.py`, `midas_gui/tab_view.py`, `tests/test_hydra_ui.py` (new). **Roll back:** `git revert 1cc4dab`. Self-contained back to the `3d2d99d` placeholder state.

`8707372` — Hydra: multi-curve radial profile plot with toggleable composite curve (2026-08-23)

Effect: Adds `HydraProfileViewer` (`hydra_widgets.py`): one fixed-color curve per Hydra panel (ge1-4) plus a toggleable "Composite" curve, sharing an R/2θ/Q axis-unit selector and Log-Y toggle — replacing the single-curve `ProfileViewer` placeholder from `1cc4dab`. Each ge-card's own `DetectorGeometryCard.set_profile_view` is now bound once, permanently, via a small `_ProfileSinkAdapter` (`hydra_page.py`) into that panel's named curve on the shared plot, since (unlike the image viewer) all 4 profiles need to stay live regardless of which panel is currently active in the toolbar. The **Composite** curve is deliberately *not* the composite card's own `radial_integrate()` (which would integrate the already-composited image, double-counting any panel overlap and mixing registration error into the profile — exactly the approach the plan review flagged and the user asked to avoid). Instead `HydraViewerPage._refresh_composite_curve` converts each available panel's own profile to a shared 2θ axis, resamples all onto one common grid, and takes a NaN-aware `nansum`, then round-trips through the normal `set_curve(r_px, ...)` API using one contributing panel's geometry as a consistent reference axis (so X-axis unit switching still works). The composite card's own profile/radial-control bindings are deliberately left unset so it can never write into that same curve slot. New `_refresh_profile_curves()`/`_refresh_composite_curve()` orchestration: frame/sibling-list changes reload+reintegrate every available panel (not just the active one); "Integrate" force-refreshes all of them regardless of Auto; R-bin edits refresh the derived Composite curve after each card's own pre-existing auto-radial wiring runs. **Also found and documented** (not silently worked around): adding these tests exposed a rare, pre-existing pyqtgraph interpreter-crash risk (`Segmentation fault`/`Bus error` in pyqtgraph's own global ViewBox/ WidgetGroup teardown internals) that surfaces more easily under a large test suite with many `ImageView`/`ViewBox` instances. An autouse `app.processEvents()` teardown fixture in `tests/test_hydra_ui.py` measurably reduced (did not guarantee-eliminate) it; `gc.collect()` there made it *worse*. Full detail — including what not to try next — in `.context/DECISIONS.md`. **Verified:** end-to-end with the synthetic fixture — all 4 curves populate independently, the Composite toggle correctly shows/hides and recomputes, the Integrate button refreshes everything with Auto off; an offscreen screenshot confirms the legend/colors/curves render correctly together. 2 new tests in `tests/test_hydra_ui.py` (6 total in that file). Full pytest suite: 3 consecutive clean runs post-mitigation, same single pre-existing `test_smoke.py` flake each time. **Files:** `midas_gui/hydra_widgets.py`, `midas_gui/hydra_page.py`, `tests/test_hydra_ui.py`, `.context/DECISIONS.md`. **Roll back:** `git revert 8707372`. Self-contained back to the `1cc4dab` single-curve-plot state.

`0aa5feb` — Hydra: fix 5 bugs found in the real windowed Phase 6 pass (2026-08-24)

Effect: The first real windowed (not offscreen) manual pass over Hydra mode surfaced 5 real bugs that headless tests/screenshots never exercised, all fixed: 1. **λ/max 2θ/pixel size shared across ge1-4 + Composite.** `DetectorGeometryCard` gained `get_shared_fields()`/ `apply_shared_fields()`; `HydraViewerPage._sync_shared_fields()` mirrors an edit on any one of the 5 geometry cards onto the other 4 (same X-ray beam and GE detector model, so these three are always physically identical), guarded by a `_syncing_shared` re-entrancy flag. 2. **Dark/bright/background correction added for Hydra**, closing a documented scope cut. New `HydraFieldSelector` (`hydra_widgets.py`) reuses the single-detector tab's `helpers.apply_field_corrections`/ `workers.FieldAverageWorker` as-is and auto-discovers the other 3 panels' field files via `helpers.hydra_siblings`. 3. **Composite windmill orientation.** Two rounds: (a) an earlier same-day attempt to "fix" the rotation to clockwise was itself wrong and reverted back to the original counterclockwise convention; (b) even after that revert, ge2/ge4 were still on the wrong sides of the canvas, fixed by adding a vertical-axis mirror in `hydra.py::compute_inv_coords` (`Y_lab = (half − Yo) ∗ px`) — scoped to the composite build only, independent of the rotation-direction math, and does not touch the per-panel ge1-4 raw displays. Both fixes confirmed against real `park_may26_bc` calibration + real `park_may26/ge{1..4}` frames, not just offscreen tests. 4. **Stale radial-integration geometry fixed.** `_effective_calib_geom()` was returning a frozen snapshot from calibration-load time, so a post-load BC/λ/Lsd/px/tilt edit moved the ring overlay (reads live widgets) but not the radial profile. Fixed to always override wavelength/Lsd/BC/pxY/pxZ/ty/tz from the live widgets. 5. **`ImageViewer` vmin% auto-level now excludes exact-zero pixels** app-wide — fixes a washed-out Composite view whose mostly-empty `BigDet` canvas previously dominated the percentile calculation. **Verified:** full Hydra geometry/chirality test suite (14 tests) passes against the reverted rotation + new mirror; orientation and stale-geometry fixes additionally confirmed by rendering the composite from real `park_may26_bc`/`park_may26/ge{1..4}` data, not offscreen fixtures alone. Full reasoning for all 5 bugs, including the wrong-then-reverted clockwise attempt, in `.context/DECISIONS.md`. **Files:** `midas_gui/hydra.py`, `midas_gui/hydra_geometry_card.py`, `midas_gui/hydra_page.py`, `midas_gui/hydra_widgets.py`, `midas_gui/widgets.py`, `tests/test_hydra_chirality.py`, `tests/test_hydra_geometry.py`, `tests/test_hydra_ui.py`, `.context/DECISIONS.md`. **Roll back:** `git revert 0aa5feb`. Self-contained back to the `8707372` state (Phase 6 scope cuts — dark/bright/background, shared fields — would return; the still-wrong pre-revert orientation is not restored, since that never existed on `main`).

`eadb14d` — Hydra: fix Pick BC/Pick Ring point leakage across panels (2026-08-24)

Effect: Continuing the Phase 6 manual pass, found that the single `ROIImageViewer` shared by all 5 Hydra panel cards (ge1-4 + Composite) keeps its Pick BC/Pick Ring click state (`_ring_pts` etc.) on the viewer itself, not per-card. `DetectorGeometryCard.bind_viewer()` already cleared each card's own ring/label overlay items off the previous viewer on rebind, but never cleared the viewer's in-progress pick points — so points clicked while one panel was active could survive a panel switch and mix into another panel's circle fit. `bind_viewer()` now also calls `old._clear_ring_points()` when rebinding away from a viewer. **Verified:** new regression test `test_hydra_pick_ring_points_dont_leak_across_panels` (picks 2 points on ge1, switches to ge2, asserts the viewer's pick state is empty, then picks 3 fresh points and asserts only those 3 are present). **Files:** `midas_gui/hydra_geometry_card.py`, `tests/test_hydra_ui.py`. **Roll back:** `git revert eadb14d`. Self-contained; no dependents.

`93dafa2` — Hydra: Calibrate tab split into Single-detector/Hydra modes + Data Viewer refinements (2026-08-24)

Effect: Calibrate tab (Tab 2) gains a leftmost mode ribbon (`HydraModeRibbon`, reused as-is) splitting **Single detector** / **Hydra**, mirroring the Data Viewer tab's existing split: 1. New `hydra_calib_widgets.py` (`HydraCalibPanelCard` — one GE panel's Transforms + seed + fitted result/rings/residuals/results-grid, all independent per panel) and `hydra_calib_page.py` (`HydraCalibrationPage` — shared Pipeline/Detector&Calibrant/Threshold/Average-frames/Refine -parameters/Advanced cards applied to all 4 panels' fits, a `HydraLoaderPanel` reused verbatim, a shared image viewer + ge1-4 toolbar with no Composite, and bottom tabs for a shared multi-curve Radial Profile, per-panel Ring Residuals/Results, and a shared Log prefixed `[ge{n}]`). 2. **Sequential** (one panel at a time, full log capture) or **Parallel** (all present panels fit at once)

run modes. `CalibrationWorker` gained a `capture_stdout` flag (default `True`); Parallel mode passes `False` since several workers would otherwise race on the process-global `sys.stdout`/`sys.stderr` redirect. 3. Geometry hand-off with the Data Viewer's Hydra page: **← Data Viewer** imports loaded panels + fitted geometry (`HydraViewerPage.export_for_calibration` / `DataViewerTab.get_hydra_export`); **→ Send to Data Viewer** pushes a fitted panel's geometry back (`DataViewerTab.set_hydra_panel_geometry`). 4. `helpers.source_kind` and `helpers.paramstest_pairs` promoted out of `HydraFieldSelector._kind_of` / `CalibrationTab._paramstest_pairs` so both single-detector and Hydra Results grids share one implementation. 5. Bundled Data Viewer Hydra-page refinements from the same session: Transforms (Flip Y/Flip Z/Transpose[/Rotate]) is now its own boxed Transforms card shared by every geometry card; Hydra ge1-4 gain a per-panel-only Rotate field (`hydra.apply_panel_rotation`, excluded from the Composite view); a per-panel Projection card (Max/Sum/Average stack reduction); a fix for Flip Y/Flip Z/Transpose not refreshing the displayed Hydra image; the Hydra radial plot's pan/zoom bounded to the combined data range of the visible curves; and the Material dialog's Preset dropdown now also fills the Name field. **Verified:** new `tests/test_hydra_calib_ui.py` (pick-state cross-panel isolation; Sequential/Parallel run orchestration + Results/Ring-Residuals panel switching, against a stubbed `CalibrationWorker`/`IntegrationWorker`). Full suite run clean apart from the pre-existing `test_app_builds_offscreen` "visible_tabs" local-config flake and the known pyqtgraph interpreter -teardown noise (both pre-existing, see `.context/DECISIONS.md`). Not yet exercised with a real windowed pass (Pick BC/Pick Ring mouse clicks, real calibration convergence on real Hydra data) — offscreen/stubbed-worker tests only so far. **Files:** `midas_gui/app.py`, `midas_gui/helpers.py`, `midas_gui/hydra.py`, `midas_gui/hydra_calib_page.py` (new), `midas_gui/hydra_calib_widgets.py` (new), `midas_gui/hydra_geometry_card.py`, `midas_gui/hydra_page.py`, `midas_gui/hydra_widgets.py`, `midas_gui/tab_calibrate.py`, `midas_gui/tab_view.py`, `midas_gui/workers.py`, `tests/test_hydra_calib_ui.py` (new), `.context/DECISIONS.md`. **Roll back:** `git revert 93dafa2`. Self-contained back to the `eadb14d` state; no dependents yet.

`6b961d4` — Hydra: Batch Integrate tab split into Single-detector/Hydra modes (2026-08-24)

Effect: Batch Integrate tab (Tab 4) gains a leftmost mode ribbon (`HydraModeRibbon`, reused as-is) splitting **Single detector** / **Hydra**, mirroring the Calibrate tab's own split — each of the 4 GE panels is integrated with its own independently fitted geometry: 1. New `hydra_batch_widgets.py` (`HydraBatchPanelCard` — one panel's Calibration source [From Calibrate tab / From file] + read-only values grid + compact progress bar) and `hydra_batch_page.py` (`HydraBatchPage` — shared Integration/Corrections/Monitor-normalisation/Output cards applied to all 4 panels, a `HydraLoaderPanel(mode="stream")`, a `QStackedWidget` of 4 per-panel Waterfall+Stacked-profile viewer pairs switched by a `HydraDetectorToolbar` panel toggle, shared Log prefixed `[ge{n}]`). 2. **Sequential** (one panel at a time) or **Parallel** (all panels at once) run modes, `BatchWorker`-per-panel orchestration copied from `HydraCalibrationPage`'s pattern. Each panel writes to its own `<out_dir>/ge{n}/` subfolder. 3. Automatic hand-off: a Hydra panel's Calibrate-tab fit auto-populates that panel's Batch Integrate calibration source the moment it finishes (`HydraCalibrationPage.panelCalibrationDone` → `CalibrationTab.hydraPanelCalibrationDone` → `BatchTab.set_hydra_panel_calibration`), mirroring the existing single-detector `calibrationDone→set_calibration` wiring. Each panel keeps an independent manual "From file" fallback. 4. Masks are wired per panel for Hydra Integrate — a deliberate departure from Hydra Calibrate's `mask=None` scope-cut: `HydraLoaderPanel` gains a `mode="nav"|"stream"` switch; `"stream"` mode adds a shared frame-range+stride row and one independent `MaskSelector` per panel (no sibling auto-discovery, since mask files are physically panel-specific). 5. Drift correction and live MONITOR mode are deferred for Hydra v1 (both exist on the single-detector Batch tab). 6. `HydraBatchPage` is built lazily on `BatchTab` (only on first switch to Hydra mode, on the auto hand-off, or on a saved-session restore) since it owns 8 pyqtgraph widgets and most sessions never open it. 7. `helpers.py` gains `resolve_calibration_fields`/`render_calib_value_grid`, hoisted out of `BatchTab`'s own calibration-value-grid logic so `HydraBatchPanelCard` doesn't duplicate it; `BatchTab` now calls the same two functions. **Verified:** new `tests/test_hydra_batch_ui.py` (calibration hand-off, per-panel mask independence, output subfolder-per-panel naming, Sequential/Parallel orchestration + per-panel viewer-stack switching) against a stubbed `BatchWorker`. `release.sh`'s test step now runs the two Hydra UI test files (`test_hydra_calib_ui.py`,

test_hydra_batch_ui.py) in their own separate pytest processes instead of one combined run — A/B testing found their cumulative pyqtgraph widget churn pushed the pre-existing, documented interpreter-teardown segfault (see .context/DECISIONS.md) from a ~40% baseline rate to ~100%; isolated runs eliminate it for those files while leaving the rest of the suite at baseline. Not yet exercised with a live windowed pass (real Hydra data, real per-panel BatchWorker runs) — offscreen/stubbed-worker tests only so far. **Files:** midas_gui/app.py, midas_gui/helpers.py, midas_gui/hydra_batch_page.py (new), midas_gui/hydra_batch_widgets.py (new), midas_gui/hydra_calib_page.py, midas_gui/hydra_widgets.py, midas_gui/tab_batch.py, midas_gui/tab_calibrate.py, tests/test_hydra_batch_ui.py (new), release.sh. **Roll back:** git revert 6b961d4. Self-contained back to the 93dafa2 state; no dependents yet.

e8dea6b — Add File > Project: opt-in FAIR provenance record (HDF5) (2026-08-24)

Effect: New midas_gui/project.py module and File ▸ New/Open/Close Project… menu actions add a long-lived, opt-in provenance record separate from GUI State (which snapshots widget *configuration*; a Project records what actually *happened*): 1. A Project is a single HDF5 file (project.create_project/open_project, marked with a PROJECT_MARKER attr + schema version). While one is open (tracked via a shared ProjectContext handed to CalibrationTab and BatchTab, which forward it to their Hydra pages), every completed Calibrate run (single-detector or each of the 4 Hydra panels) and every completed Batch Integrate run appends one append-only attempt_NNNN record under /single/... or /ge{n}/... — never overwritten, so re-running keeps the full history. 2. Each record embeds: full params/result as a JSON metadata blob (_sanitize_result_dict drops only torch-tensor fields, duck-typed via .numpy); the mask/dark/bright/background actually applied, as small embedded HDF5 datasets (masks drawn in Mask Builder have no file of their own to reference); and an environment_snapshot() (midas-gui version + best-effort git commit, package versions, PyQt/Qt versions). Raw scan/calibrant file paths are referenced, not duplicated — hashed via sha256_file (full SHA-256 under 200 MB, else a fast head/tail fingerprint so logging never stalls on a huge multi-frame dataset). 3. A Batch Integrate record embeds the exact calibration values used (calibration_snapshot) and links back to the specific Calibrate attempt that produced them via calib_attempt_ref (the calibration result object carries its own _project_attempt_ref after being logged), when that run was itself logged to the same project. 4. Logging is always best-effort: _log_to_project (one per tab/page) is a no-op when no project is open, and any write failure is caught, logged to the tab's own Log panel, and never blocks or alters the on-screen result — a full Calibrate/Integrate run must never fail because of a provenance-write problem (e.g. disk full). 5. Status bar gains a permanent "Project: …" label; File menu gains New/Open/Close Project actions (Close only enabled while one is open). **Verified:** new tests/test_project.py (create/open roundtrip, refuses to overwrite, rejects a non-project .h5, sha256_file full vs. partial, attempt numbering + latest pointer, mask/cfg field filtering, integration -to-calibration linking, and CalibrationTab/BatchTab's own _log_to_project methods including the open-project no-op case) plus new project-logging assertions added to tests/test_hydra_calib_ui.py and tests/test_hydra_batch_ui.py (per-panel attempts land under /ge{n}/..., calib_attempt_ref round-trips end to end). Full suite run clean (main suite in one process, the two Hydra UI files each in their own process per release.sh's existing isolation) apart from the pre-existing test_app_builds_offscreen local-config flake and the known pyqtgraph interpreter-teardown noise (both pre-existing, see .context/DECISIONS.md). **Files:** midas_gui/app.py, midas_gui/project.py (new), midas_gui/hydra_batch_page.py, midas_gui/hydra_calib_page.py, midas_gui/tab_batch.py, midas_gui/tab_calibrate.py, tests/test_project.py (new), tests/test_hydra_batch_ui.py, tests/test_hydra_calib_ui.py. **Roll back:** git revert e8dea6b. Self-contained back to the 6b961d4 state; no dependents yet.

162fef1 — Calibrate: link Hydra seed-mode across panels; add Eta vs R Cake tab (2026-08-24)

Effect: Two Calibrate-tab changes, requested independently of each other: 1. Hydra Calibrate's **Use manual seed** / **Feed result back to seed** checkboxes (HydraCalibPanelCard._manual_seed_check/_feedback_check) are now one shared choice mirrored across all 4 GE panels — toggling either on any one panel's card (whether by hand, or programmatically via Pick BC/Pick Ring/Load calibration file/seed-from-result) sets the same state on the other three (HydraCalibrationPage._sync_seed_checkbox, wired per-card at construction). Only the

mode is linked; the seed BC/Lsd/tilt *values* stay fully independent per panel, since each GE module's beam centre and mounting genuinely differ. Single-detector Calibrate is unaffected (only one detector, nothing to link). 2. Both Single-detector and Hydra Calibrate gain a new **Eta vs R Cake** tab alongside Radial Profile: a 2-D (η, R) intensity heatmap — the routine azimuthal-integration "cake" visualization for area-detector diffraction data, R (px) on the X-axis and η (°) on the Y-axis. New `midas_gui.widgets.CakeViewer` (a `pg.ImageView`-based widget, following the app's existing 2-D-heatmap convention, but positioned via `ImageItem.setRect()` to the actual R/η bin-centre extent rather than assumed to start at pixel (0,0) like the detector `ImageViewer`) with its own Log/colormap/vmin%/vmax% toolbar and a hover readout in R/η/ intensity. `IntegrationWorker.run()` (`workers.py`, shared by both tabs) now passes `return_cake=True` to the existing `integrate_frame()` (the 2-D cake was already computed there and previously discarded at this call site) and emits `cake_2d`/ `eta_axis_deg` alongside the existing 1-D `profile`; both tabs' `_on_int_done` feed the new field into the viewer(s) when present (`.get()`-guarded, so the existing stubbed-`IntegrationWorker` tests — which don't emit it — are unaffected). Hydra mode uses a per-panel `CakeViewer` stack (`self._cake_views[n]`), switched by the same toolbar that already drives the Results/Ring-Residuals stacks. **Verified:** new assertions added to `tests/test_hydra_calib_ui.py` — checkbox-state linking across all 4 panels with seed-value independence proven by giving each panel a distinct BC first; per-panel cake population after a run and cake-tab switching with the active panel (extends `_FakeIntegrationWorker` to emit synthetic `cake_2d`/`eta_axis_deg`). Also manually verified end-to-end outside the stubbed tests: a real (non-stubbed) `IntegrationWorker.run()` against a synthetic image produces a correctly-shaped `(n_eta, n_r)` cake and eta axis via the real `midas_integrate_v2` backend, and `CakeViewer.set_cake()`/log-toggle/ percentile-change/hover-readout all render without error offscreen; a full `MainWindow()` construction confirms both the single-detector and lazily-built Hydra `CakeViewer`(s) wire up cleanly. Full suite run clean (main suite in one process, the two Hydra UI files each in their own process per `release.sh`'s existing isolation) apart from the two known pre-existing issues (`test_app_builds_offscreen` local-config flake, pyqtgraph interpreter-teardown noise — see `.context/DECISIONS.md`). **Files:** `midas_gui/hydra_calib_page.py`, `midas_gui/tab_calibrate.py`, `midas_gui/widgets.py`, `midas_gui/workers.py`, `tests/test_hydra_calib_ui.py`. **Roll back:** `git revert 162fef1`. Self-contained back to the `e8dea6b` state; no dependents yet.

`e693316` — File > Open Project now populates Calibrate/Batch Integrate from recorded attempts (2026-08-24)

Effect: Opening a project (`e8dea6b`) previously only wired it up for *future* provenance logging — every tab's fields were left exactly as they were, so a saved project's data paths, geometry, and settings could never be picked back up. **File ▸ Open Project…** now follows a successful open with a new `ProjectLoadDialog` (one row per panel found in the file — single- detector, or each present Hydra ge1–ge4) offering two independent checkboxes per row, **Calibrate** and **Batch Integrate**, each paired with an attempt picker defaulting to that panel's latest record; a checkbox is disabled if that panel has no attempt of that kind. Accepting applies the checked rows and switches each tab's mode ribbon (Single-detector/Hydra) to match. `project.py` gains a read-side API — `discover_panels`/`list_attempts`/ `read_attempt` — plus pure (no-Qt) mapping functions (`calib_attempt_gui_fields`/`calib_attempt_loader_state`/ `integrate_attempt_gui_fields`/`integrate_attempt_loader_state`/ `calibration_namespace`) that translate a stored attempt's `cfg`/`result`/ `inputs` into the *same widget-keyed field dicts* `CalibrationTab`'s and `BatchTab`'s existing `get_state()`/`set_state()` GUI-state machinery already know how to apply — the single-detector tab's fields, the Hydra page's shared "recipe" fields, and one Hydra panel card's seed fields turn out to be three non-overlapping subsets of the same key vocabulary, so one field dict serves all three (`helpers.apply_dict_to_widgets` just ignores keys a given widget map doesn't define). `CalibrationTab .apply_project_calibration()`/`BatchTab.apply_project_integration()` build one combined state dict per tab and call `set_state()` once, so a project with attempts for several Hydra panels lands in a single restore rather than one mode-ribbon flip per panel. Batch Integrate additionally gets a *live, usable* calibration (not just the read-only display fields `set_state()` alone would give it): the attempt's stored `calibration_snapshot` is turned into a duck-typed result object (`project.calibration_namespace`, mirroring `helpers .result_ns_from_geometry_file`'s shape) and installed via the tab's own

`set_calibration()`/`HydraBatchPage.set_panel_calibration()` — so **Run** reproduces the recorded integration immediately, without first re-running Tab 2. Known limitation (documented, not fixed here): dark/bright/background/mask sources that were embedded directly in the project (no file of their own — e.g. a mask hand-drawn in Mask Builder) are not restored, since `FieldSelector`/`MaskSelector` only ever supported file-path-backed sources, even for plain GUI-State save/load. **Verified:** new pure-logic tests in `tests/test_project.py` for every new `project.py` helper (panel discovery, attempt listing/reading, both attempt→field mappings including the μm→mm Lsd conversion and the unbounded-frame-range `None→0` guard, and the calibration namespace shape), plus two Qt-level end-to-end tests building a real `CalibrationTab`/`BatchTab` and asserting the actual widget values after `apply_project_calibration`/`apply_project_integration` — one exercising Hydra (ge1/ge2, mode switch + per-panel seed/calibration values distinct per panel) and one single-detector. Also manually exercised against the project file this bug was filed against (`test_data/midas_project.h5`, a real Hydra ge1–ge4 project) end-to-end through `MainWindow`: mode switched to Hydra, each panel's seed BC/Lsd and shared wavelength/calibrant populated correctly, and each Batch panel got a live calibration with the recorded Lsd — confirming the exact complaint ("data paths and settings are not autopopulated") is fixed. Full suite run clean (main suite in one process, the two Hydra UI files each in their own process per `release.sh`'s existing isolation) apart from the two known pre-existing issues (`test_app_builds_offscreen` local-config flake, pyqtgraph interpreter-teardown noise — see `.context/DECISIONS.md`). **Files:** `midas_gui/project.py`, `midas_gui/dialogs.py`, `midas_gui/app.py`, `midas_gui/tab_calibrate.py`, `midas_gui/tab_batch.py`, `tests/test_project.py`. **Roll back:** `git revert e693316`. Self-contained back to the `162fef1` state; no dependents yet.

`81056e4` — Profiles: header Profile combo for instant switching; fix stale option lists on profile change (2026-08-25)

Effect: Two related complaints fixed together. First, switching profiles required opening Settings ▸ Preferences every time — a **Profile: [combo ▼]** control now sits at the main window's top-left corner, in the same row as the tab bar (`QTabWidget.setCornerWidget`), wired straight to `settings.set_active_profile()` + `constants.reload_from_config()`; the Preferences dialog's own Profile row still works the same as before and stays in sync either way — both now funnel through one new `MainWindow.on_profile_changed()` instead of the dialog calling `apply_tab_visibility()` directly. Second, Hydra mode (the 1-ID-E 4-panel GE detector view) is now hidden on Data Viewer/Calibrate/Batch Integrate's mode ribbons unless the active profile is **1-ID-E** — the only beamline with that detector — falling back to Single detector automatically if Hydra was the active mode when it's hidden (`set_hydra_available`/ `set_hydra_enabled`, driven by `MainWindow.apply_hydra_visibility()`). Third, the actual reported bug: profile-scoped *option lists* — the Data Viewer's Live PV device dropdown, the Calibrate tab's Calibrant dropdown (single-detector and every Hydra panel), and the pixel-size-preset/ K-edge-foil popup menus — were only ever populated once at tab-construction time, so after a profile switch they kept showing the *previous* profile's choices until the app was restarted. Fixed with new `refresh_devices()` (`DataViewerTab`/`DataLoaderPanel`) and `refresh_calibrants()` (`CalibrationTab`/`HydraCalibrationPage`) methods, both called from `on_profile_changed()`, sharing a new `helpers.refresh_combo_items()` that repopulates a combo box while preserving the current selection if it still exists; the pixel/K-edge popup menus (`make_pixel_label`/`make_kedge_label`) now rebuild their entries from live `constants.*` module attributes on every `QMenu.aboutToShow` instead of freezing a list at construction. Seeded default *values* (wavelength, pixel size, Lsd, beam-centre, default file paths) are deliberately left alone by a profile switch — only fields built after the switch pick those up — since there's no way to tell a seeded default apart from the user's own in-progress edit once a field exists; see `.context/DECISIONS.md` (2026-08-25 entry) for the full reasoning. **Verified:** new tests in `tests/test_helpers.py` (combo/menu rebuild logic in isolation), `tests/test_live_stream.py` (`refresh_devices` — repopulates, preserves custom-typed text, no-ops without a Live Data card), and `tests/test_smoke.py` (`on_profile_changed` end-to-end through a real `MainWindow`, asserting the Live PV dropdown and both the single-detector and Hydra Calibrant dropdowns all show the newly-active profile's values). Full suite run clean apart from the two known pre-existing issues (`test_app_builds_offscreen` local-config flake, pyqtgraph interpreter-teardown noise — see `.context/DECISIONS.md`). **Files:** `midas_gui/app.py`, `midas_gui/helpers.py`,

midas_gui/hydra_calib_page.py, midas_gui/hydra_widgets.py, midas_gui/prefs_dialog.py, midas_gui/tab_batch.py, midas_gui/tab_calibrate.py, midas_gui/tab_view.py, midas_gui/widgets.py, tests/test_helpers.py, tests/test_live_stream.py, tests/test_smoke.py. **Roll back:** `git revert 81056e4`. Self-contained back to the `e693316` state; no dependents yet.

`9e4b5c0` — Data Viewer/Calibrate: auto-detect pixel size + wavelength on file load (2026-08-25)

Effect: Loading a Data file now auto-populates **pixel size** (from the detector tag in the filename — `.ge1`–`.ge5` → GE, 200 μm; `.vrx` → Varex, 150 μm; `.pxrd` is recognized as Pixirad but has no known pixel size to auto-fill) and, for an HDF5 frame file, **wavelength** (derived from its recorded beam energy at the `instrument/HEM/Energy` dataset, keV → Å via `HC_KEV_A`). Gated to the **1-ID-E**, **20-ID-D**, and **20-ID-E** profiles via `settings.active_profile()` since the filename/metadata conventions are beamline-specific; anything not detected is left at its previous/default value. New `helpers.detect_detector_from_filename()` / `detect_wavelength_from_h5()` / `detect_geometry_from_path()` do the detection (best-effort — any read error or missing dataset just yields `{}`, never blocks the actual file load). Wired into the Data Viewer (via a new `DataLoaderPanel.metadataDetected` signal feeding `DetectorGeometryCard. apply_shared_fields`) and Calibrate (`CalibrationTab._on_metadata_detected`) in single-detector mode, and into Hydra mode on both tabs via a new `HydraLoaderPanel.detected_geometry()` accessor consulted from `HydraViewerPage._on_siblings_changed`/`HydraCalibrationPage. _on_siblings_changed` off the loaded anchor-panel file. Batch Integrate is unaffected by design — it always gets pixel size/wavelength from the calibration it's handed, not the raw file. **Verified:** new tests in `tests/test_helpers.py` (filename/HDF5/combined detection, gating to unknown profiles) and `tests/test_smoke.py` (end-to-end through a real `MainWindow`/`CalibrationTab` load). Full suite run clean apart from the two known pre-existing issues (`test_app_builds_offscreen` local-config flake, pyqtgraph interpreter-teardown noise across many `MainWindow()` builds in one process — see `.context/DECISIONS.md`); each new test also verified individually in its own process. **Files:** `midas_gui/helpers.py`, `midas_gui/hydra_calib_page.py`, `midas_gui/hydra_page.py`, `midas_gui/hydra_widgets.py`, `midas_gui/tab_calibrate.py`, `midas_gui/tab_view.py`, `midas_gui/widgets.py`, `tests/test_helpers.py`, `tests/test_smoke.py`. **Roll back:** `git revert 9e4b5c0`. Self-contained; no dependents yet.

`653c832` — Open Project: auto-display rings/profile/cake + Hydra per-panel batch plots; header project-name indicator (2026-08-25)

Effect: Populating Calibrate/Batch Integrate from a project attempt (File ▸ Open Project…) previously only restored *input fields*, leaving every plot blank until the user pressed Fit/Run again. Now it also redraws the recorded **result**: `CalibrationTab._display_stored_result()` / `HydraCalibrationPage.display_stored_result()` redraw the predicted-ring overlay immediately from the stored geometry (pure geometry, no image needed) and, if the attempt's data file can still be loaded, re-run the existing `IntegrationWorker` for the Radial Profile / Eta-vs-R Cake tabs — mirroring exactly what a live Fit's `_on_done`/`_on_panel_done` already do. New `project.read_attempt_results()` reads the embedded `profiles/r_axis_px/frame_ids` arrays an integration attempt already stores (previously only the JSON metadata was read back by `Open Project`); `BatchTab._populate_plots_from_attempt()` / `HydraBatchPage. populate_panel_plots()` replay them into the Waterfall/Stacked-profiles views via the same widget calls a live run's per-frame streaming already uses. Each Hydra panel keeps its own independent viewer pair (`_viewer_pairs`), so the existing GE1–GE4 toolbar now shows genuinely panel-specific recorded results instead of nothing. Also: a new bold, high-contrast **"Project: <name>"** label at the tab bar's top-right corner (mirroring the Profile combo's top-left one) makes an open project hard to miss, alongside the existing quiet status-bar label. **Verified:** new/extended tests in `tests/test_project.py` (`test_read_attempt_results_roundtrip`, ring/plot assertions added to the existing `apply_project_calibration`/`apply_project_integration` tests) and `tests/test_smoke.py` (header label). Each relevant test file re-run individually in its own process; confirmed the pre-existing pyqtgraph interpreter-teardown segfault (`.context/DECISIONS.md`) reproduces on unmodified `main` too (3/3 runs) when running the full `tests/` directory in one process — unrelated to this change, not

attempted to fix (`gc.collect()` is known to make it worse). **Files:** `midas_gui/app.py`, `midas_gui/hydra_batch_page.py`, `midas_gui/hydra_calib_page.py`, `midas_gui/project.py`, `midas_gui/tab_batch.py`, `midas_gui/tab_calibrate.py`, `tests/test_project.py`, `tests/test_smoke.py`. **Roll back:** `git revert 653c832`. Self-contained; no dependents yet.

`08c917f` — Calibrate/Batch Integrate plots: bound pan/zoom like image viewers; Cake right-drag zooms Y only; Waterfall gains color-scale sidebar (2026-08-25)

Effect: Radial Profile and Eta vs R Cake (Calibrate) and Waterfall and Stacked profiles (Batch Integrate) could be zoomed/panned arbitrarily far from their actual data, unlike the main image viewers which are bounded via `ImageViewer._apply_view_limits()`. Added the same `setLimits()`-based bounding: `CakeViewer._apply_view_limits()` (R/η extent), `WaterfallViewer._apply_view_limits()` (R/frame# extent), and `StackedProfileViewer._apply_view_limits()` (combined data extent across all stacked curves, tracked incrementally as profiles stream in and recomputed on a spacing/x-unit change). `ProfileViewer`/`HydraProfileViewer` already had equivalent bounding — untouched. Also: the cake plot's right-click-drag zoomed **both** R and η together (stock pyqtgraph scales both axes on a diagonal right-drag); a new `_YZoomViewBox(pg.ViewBox)` overrides `mouseDragEvent` for the right button so the gesture scales η (Y) only — every other interaction (left-drag pan, wheel zoom, the context menu) is untouched, delegated to the base class. Separately, `WaterfallViewer` had no color-scale legend at all — unlike the `pg.ImageView`-based viewers (`ImageViewer`, `CakeViewer`), it only ever set a `cmap` name to an `ImageItem.setLookupTable()` call with no visible gradient/levels widget. Added a `pg.HistogramLUTWidget` wired via `setImageItem()`, placed in a new row alongside the plot; its gradient (not a direct `setLookupTable()` call) now drives color so a user's histogram level drag and the cmap dropdown stay consistent. `CakeViewer` already had this via `pg.ImageView`'s built-in histogram — unchanged. All three touched viewer classes are shared between single-detector and Hydra mode (`HydraCalibrationPage`/`HydraBatchPage` instantiate the same `CakeViewer`/`WaterfallViewer`/`StackedProfileViewer`), so the fix applies to both without per-mode changes. **Verified:** `tests/test_hydra_calib_ui.py`, `tests/test_hydra_batch_ui.py` and `tests/test_hydra_ui.py` each re-run individually in their own process — all pass (the only failures are the pre-existing pyqtgraph interpreter-teardown segfault/`QComboBox`-deleted noise at interpreter exit, reproduced identically on unmodified `main` via `git stash`); `tests/ test_smoke.py`'s `test_app_builds_offscreen` shows the same pre-existing local-config `visible_tabs` flake documented in `.context/STATE.md`. Also manually exercised `CakeViewer`/`WaterfallViewer`/`StackedProfileViewer` and the `_YZoomViewBox` drag math offscreen (`QT_QPA_PLATFORM=offscreen`) to confirm limits are applied, the histogram levels track cmap changes, and a simulated right-drag-up scales Y only while leaving X unchanged. **Files:** `midas_gui/widgets.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 08c917f`. Self-contained; no dependents yet.

`87d9df7` — Data Viewer: add lab-frame axes overlay (APS/MIDAS coordinate compass) (2026-08-25)

Effect: Ported the "Lab-frame axes" overlay from the sibling `midas-gui-swaxs` repo's Data Viewer (`gui_common.draw_lab_frame_axes` / `ff_asym_qt._draw_axes`, itself ported from JSP's tooling): a new **Lab-frame axes** checkbox on the Data Viewer's image toolbar (next to Top-N pixels) overlays the APS/MIDAS lab coordinate system on the displayed image, anchored at the beam centre — a red **X_Lab** arrow, a green **Y_Lab** arrow, a blue ⊗ beam-direction (Z_Lab) glyph, and an orange η-sweep arc (0°→45°) with a tick at η=0°. Lets a user visually confirm orientation/ImTransOpt is correct: a known feature should land in the quadrant the overlay predicts. All items are plain `pg.PlotDataItem`/ `pg.TextItems` added directly onto the viewer's `pg.ImageView` ViewBox, so pan/zoom transforms them for free — they're only rebuilt (`_clear_lab_axes` then `_draw_lab_axes`) when the image or geometry actually changes, wired via `DetectorGeometryCard.geometryChanged` (BC/tilt/calibration edits) plus an explicit call at the tail of every `self._cur`-reassigning method (`_on_loader_data`, `_on_fields_changed`, `_on_projection_done`, `_on_im_trans_changed`'s projection branch) and `set_state()` (session restore). The checkbox's state round-trips through Save/Load GUI State via the existing generic `_state_widgets()` dict, no dedicated setting needed. **Porting note:** the source repo's `pg.ImageView` keeps pyqtgraph's default `invertY()`, so its code needs a `V=-1.0`

flip to make the Y_Lab arrow still point up on screen; this repo's `ImageViewer` explicitly overrides that (`vb.invertY(False)`, so MIDAS pixel `(0,0)` renders bottom-left per `.context` convention) — the opposite convention — so `V=+1.0` here instead. Everything else (arrow/arc geometry, colors, the `y_sign=-1.0` MIDAS 'bl' convention, label placement math) ports verbatim, since neither repo inverts the X axis. **Verified:** offscreen (`QT_QPA_PLATFORM=offscreen`) construction of `DataViewerTab`, toggling the checkbox on/off and confirming exactly 9 pyqtgraph items are added/removed; confirmed the overlay redraws (new objects, not stale references) when `DetectorGeometryCard`'s BC spinbox changes; rendered a screenshot with the widget properly shown/sized first to confirm the green Y_Lab arrow points up and the red X_Lab arrow points left as intended, with the ⊗ glyph centered at BC. `tests/test_smoke.py` run 5x individually: the known pre-existing pyqtgraph interpreter-teardown segfault (`ColorMapMenu/GradientEditorItem` construction inside `tab_calibrate.py`, unrelated to this change) reproduced 1/5 times, identically reproducible on unmodified `main` via `git stash` — confirmed pre-existing, not introduced by this change (`.context/STATE.md`); the known local-config `visible_tabs` flake in `test_app_builds_offscreen` also reproduces on unmodified `main`. **Files:** `midas_gui/tab_view.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 87d9df7`. Self-contained; no dependents yet.

`a74b7d6` — Upgrade MIDAS backend packages to latest PyPI releases (2026-08-25)

Effect: Bumped all 8 MIDAS analysis-backend pins in `pyproject.toml`/ `environment.yml`/`requirements.txt` to their latest PyPI releases: midas-calibrate 0.2.7→0.4.3, midas-calibrate-v2 0.5.3→0.10.0, midas-hkls 0.7.0→0.9.0, midas-integrate 0.4.2→0.6.0, midas-integrate-v2 0.3.1→0.6.0, midas-peakfit 0.5.0→0.6.0, midas-zipper 0.1.5→0.2.1, midas-pdf 0.1.1→0.1.3 (midas-distortion stayed at 0.2.0, unchanged upstream). Added an explicit pin for `midas-params==0.11.1`, a new shared params-file registry/validator dependency first required (`>=0.9.0`) by several of the bumped packages. Also made `zarr==2.18.7`/`numcodecs==0.15.1` explicit pins — both were already being pulled in transitively by the old midas-integrate/-peakfit/ -zipper versions but were never pinned in this repo's env files; this just closes that gap. `numpy==1.26.4`/`scipy==1.13.1`/`torch==2.4.0`/ `numba==0.59.1`/`pvapy==5.4.1` and the rest of the core scientific/GUI stack are unchanged — none of the new backend releases require a newer floor. **Verified:** Three parallel research passes statically diffed every symbol `midas_gui` imports from these packages (all of `midas_calibrate_v2`'s `pipelines`/`compat`/`forward`/`seed` submodules, `midas_hkls.{SpaceGroup, Lattice,generate_hkls}`, all of `midas_integrate_v2`'s top-level `__all__`, and all of `midas_pdf`'s top-level + `corrections`/`cif`/`fluorescence` symbols) against old vs. new source — every one still exists at the same module path with an unchanged or backward-compatible signature (new kwargs are optional), so no `midas_gui` source changes were needed. Testing itself was done in a cloned conda env (`midas-gui-next`, `conda create --clone`) before touching the live env: `pip install` with the new backend pins alongside the held-fixed core stack resolved cleanly (no conflicts); the full test suite was run **per-file, isolated** (each `tests/test_*.py` in its own `pytest` process — a combined `pytest tests/` run is documented in `.context/STATE.md` as unreliable even on unmodified `main`) in both the clone and an untouched `midas-gui` baseline for comparison — identical results in both: no new failures, only the same pre-existing pyqtgraph interpreter-teardown crashes (`test_hydra_ui.py`, `test_project.py`) and the same `test_smoke.py` flake rate (3-4/5 runs) already documented as environment-level, not code-related. Manually smoke-tested `pdf_backend.py` end to end against real `test_data/test_pdf/` CSVs/CIF (Composition, faber_ziman_S, i_of_q_to_Gr, read_cif_to_crystal, paalman_pings_cell_only, fluorescence_report_sample_and_container) — identical numeric results old vs. new, with one new benign informational warning from midas-hkls 0.9.0's added ionic-form-factor self-consistency check (flags its own bundled Ce4+ coefficients as violating the electron-count sum rule and falls back to neutral-atom scattering — an upstream data-table note, not an error). `tests/test_hydra_calib_ui.py`'s real `CalibrationWorker` run against synthetic panel data (the deepest exercise of the `calibrate_v2` pipeline) passed identically. Only after all of this was green did the live `midas-gui` env get the same `pip install` applied in place (backed up via `pip freeze` to `/tmp/midas-gui-env-backup-2026-08-25.txt` first), and the stale editable-install `midas_gui.egg-info/requires.txt` (left over from an earlier `pip install -e .` in both envs, contrary to `environment.yml`'s comment that midas-gui isn't pip-installed there) was refreshed via `pip install --no-deps -e .` to stop pip's cosmetic "midas-gui requires midas-calibrate==0.2.7 but you have 0.4.3" conflict warnings. **Files:** `pyproject.toml`, `environment.yml`, `requirements.txt`. **Roll back:** `git revert a74b7d6`, then re-run the same `pip install` command from this

entry with the old pinned versions in both the `midas-gui` conda env and (if recreated) any clone, since this commit only touches the pin files — it does not itself change the installed env.

`40da952` — docs: add conda env update command to README (2026-08-25)

Effect: README's "Installation & Running" section previously only documented recreating the `midas-gui` conda environment from scratch (`conda env remove` + `conda env create`) when `environment.yml` changes. Added a lighter-weight "Updating the environment" subsection above it with `conda env update -n midas-gui -f environment.yml --prune`, which syncs an existing environment in place (adds/upgrades changed packages, and with `--prune` removes ones no longer listed) without the remove/recreate round-trip. **Files:** `README.md`. **Roll back:** `git revert 40da952`. Self-contained; no dependents.

`d0cd6ce` — Calibrate/Batch: fix Cake and Waterfall auto-level to exclude zero bins (2026-08-25)

Effect: `CakeViewer` (Calibrate's Eta vs R Cake) and `WaterfallViewer` (Batch Integrate) computed their vmin%/vmax% auto-level percentiles over all finite display values, including exact-zero bins/pixels from unfilled η/R bins or zero-padded rows — skewing the level window toward zero on a partially-empty cake/waterfall. Both now mask out exact zeros before the percentile calc (falling back to the unmasked set if nothing survives), matching the fix already in place for the main image viewer. Also raised the default vmin% from 1 to 30 for both, since the zero-exclusion alone made the old default too permissive for typical (mostly-empty) cakes. **Files:** `midas_gui/widgets.py` (`CakeViewer._redisplay`, `WaterfallViewer` level calc), `documentation/gui_documentation.md`. **Roll back:** `git revert d0cd6ce`. Self-contained; no dependents.

`2358ae4` — Batch/Pump Probe/Refinement/Data Viewer: fix ImTransOpt propagation to the MIDAS integration backend (2026-08-25)

Effect: Batch Integrate's lineout was wrong whenever the active calibration had a non-zero ImTransOpt (Flip Y / Flip Z / Transpose): `spec_from_calibration_result()` never copied `im_trans` onto the built `IntegrationSpec`, and no worker applied a flip of its own either, so raw (untransformed) frames were integrated against a geometry that had actually been fit on a transformed image — a coordinate-frame mismatch, not a double-flip. Confirmed with `test_data/test_ps.txt` (`ImTransOpt 2`) + a real CeO2 frame: unflipped integration piled signal at R≈1394-1677 px (a detector-corner edge artifact); the corrected path gives clean bands at R≈96-601 px. Fixed under a single rule for the whole app: **the MIDAS backend performs every pixel flip used in an actual calibration/integration computation; `midas_gui` never does** (the only exception, kept as-is, is a viewer's on-screen preview array). Concretely: `helpers._build_spec()` / `_spec_from_result_ns()` now set `spec.TransOpt` from `im_trans`, so `midas_integrate_v2`'s own `apply_trans_opt=True` (default) flips every raw frame internally, exactly once, at every integration call site (`CalibrationWorker`, `IntegrationWorker`, `BatchWorker`, `FolderMonitorWorker`, `PumpProbeWorker`, `RefinementWorker`, `RefineCompareWorker`, `DetectorGeometryCard._midas_radial`). `CalibrationWorker` stopped pre-flipping the image/dark and clearing `cfg["im_trans"]` before calling `calib.run_pipeline` — it now passes the raw arrays and the original `im_trans` straight through, since `run_pipeline` already branches correctly per pipeline (native `im_trans` kwarg for plain `calibrate()`; manual pre-flip only for the pipeline entry points that lack the parameter). Masks are the one exception that still get pre-flipped in Python — `*BinGeometry.from_spec()` has no `apply_trans_opt` hook — tracked as a package-side gap in ROADMAP.md, not fixed here. The Batch Integrate "Calibration values" card also gained a visible **ImTransOpt** row so the active transform is no longer invisible. Two other upstream backend gaps (most calibration pipelines don't accept `im_trans` natively; no `apply_trans_opt` hook on `BinGeometry.from_spec` for masks) and one pre-existing, unrelated ImTransOpt/mask bug in the Texture tab's `PoleFigureWorker` were found along the way and logged in ROADMAP.md, not fixed in this commit. **Files:** `midas_gui/helpers.py` (`_build_spec`, `_spec_from_result_ns`, `result_ns_from_geometry_file`,

`resolve_calibration_fields`, `render_calib_value_grid`), `midas_gui/workers.py` (`MaskComputeWorker`, `CalibrationWorker`, `IntegrationWorker`, `BatchWorker`, `FolderMonitorWorker`, `PumpProbeWorker`, `RefinementWorker`, `RefineCompareWorker`), `midas_gui/tab_batch.py`, `midas_gui/hydra_batch_page.py`, `midas_gui/hydra_batch_widgets.py`, `midas_gui/tab_pumpprobe.py`, `midas_gui/hydra_geometry_card.py`, `tests/test_hydra_batch_ui.py`, `documentation/gui_documentation.md`, `test_data/test_ps.txt` (new — the real repro params file). **Roll back:** `git revert 2358ae4`. Self-contained; no dependents. Reverting restores the pre-fix (broken) behavior for any non-zero-ImTransOpt geometry.

`6b1564b` — File menu: rename GUI State to Workspace, add recent-files, dirty-state indicator, autosave, and a Project History viewer (2026-08-26, branch `workspace_ux`)

Effect: user feedback that creating/saving/loading a "project" felt inconvenient traced to two functionally unrelated concepts sharing the "Project" mental-model space with no supporting affordances around either: "GUI State" (`Ctrl+S`/`Shift+S`/`Ctrl+O`, a mutable JSON widget snapshot) and "Project" (File ▸ New/Open/Close Project, an append-only HDF5 FAIR- provenance log, no shortcuts). Neither had a recent-files list, an unsaved-changes indicator, autosave/crash recovery, or a way to browse a project's own history without `h5dump`/HDFView. Reworked the File menu around a clearer model — **Workspace** (renamed from "GUI State": your editable draft) vs. **Project** (the permanent FAIR record it can optionally log into) — borrowing recent-files/dirty-indicator/autosave patterns from established desktop apps (VS Code, Photoshop, Blender), while leaving `project.py`'s HDF5 schema, its `create_project`/`open_project`/ `append_*_attempt` functions, and its opt-in/append-only guarantees completely untouched — every new feature either reuses `project.py`'s existing read-side API or lives in new, additive sidecar/config files. Concretely: (1) "GUI State" renamed to "Workspace" everywhere in the UI — same shortcuts, same JSON schema, existing saved files still load; (2) new global recent-files store (`settings.py`: `record_recent`/`get_recent`, capped at 10, MRU-ordered, stored in a new `recent_files.json` sibling to `profile_meta.json` — deliberately global, not per-Profile) backing new File ▸ Recent Projects / Recent Workspaces submenus; (3) window title now shows the active Workspace name plus Qt's native `[*]` unsaved-changes marker, driven by a periodic (~7s) cheap hash-diff of a shared `_serialize_workspace()` helper (refactored out of the old inline `save_gui_state` body) against the hash captured at the last save/load — reuses each tab's existing `get_state()`, no per-widget change signals wired, and passing `sidecar_stem=None` on the timer tick skips every tab's sidecar file I/O so the check has no side effects; (4) `closeEvent` now prompts Save/Discard/Cancel when dirty instead of closing silently; (5) a second, much less frequent (~5 min) timer autosaves a crash-recovery draft to `settings.autosave_draft_path()` while dirty (never touches a Project's `.h5`), and `MainWindow.maybe_offer_restore_autosave()` offers to restore it — wired *only* from `main()`, deliberately not from `__init__`/`_build_ui`, so plain `MainWindow()` construction (what every existing test does) can never block on a modal dialog because of a leftover draft from an earlier crashed/killed session; (6) File ▸ New Project… now offers to also save a Workspace linked to the new project (`<name>_workspace.json`) in the same step; (7) new read-only `ProjectHistoryDialog` (`dialogs.py`) — a `QTableWidget` (the app's first) listing every recorded attempt (panel/kind/name/timestamp) built entirely from `project.py`'s existing `discover_panels`/`list_attempts`/ `read_attempt`, with a detail pane showing the selected attempt's full metadata — wired to a new, project-open-only File ▸ Project History… menu item, so inspecting a project no longer requires an external HDF5 viewer. Landed on a separate `workspace_ux` branch per explicit request, so it can be reviewed/kept or discarded independently of `main`. **Files:** `midas_gui/app.py` (`_build_file_menu`, recent-menu population, `_serialize_workspace`/`_hash_workspace_state`/`_check_workspace_dirty`/ `_update_window_title`/`_autosave_tick`/`_clear_autosave_draft`/ `maybe_offer_restore_autosave`, `save_gui_state`/`load_gui_state`, `closeEvent`, `main()`), `midas_gui/settings.py` (`record_recent`/ `get_recent`/`autosave_draft_path`), `midas_gui/dialogs.py` (`ProjectHistoryDialog`), `tests/test_workspace_ux.py` (new — recent-files roundtrip/pruning/cap, dirty-flag hash-diff, autosave write/clear, restore-on-relaunch accept/decline, `ProjectHistoryDialog` against a fixture project), `documentation/gui_documentation.md` (§16 renamed/ rewritten, §17 gains Recent Projects/New-Project-pairing/Project History subsections). **Roll back:** `git revert 6b1564b` (or, since this is isolated to

workspace_ux, simply don't merge/delete the branch). Self-contained; no dependents — project.py itself is untouched by this commit.

943a91d — Project/Calibrate: self-contained project records, Hydra Overall Cake UI, cake-plot zoom fix, Profile persistence (2026-08-26)

Effect: bundles several related, already-implemented-but-uncommitted features found together when preparing to commit (see .context/DECISIONS.md 2026-08-26 entries for full rationale on each): (1) a calibration attempt logged to a Project now embeds its computed Radial Profile / Eta vs R Cake arrays (project.append_calibration_attempt(..., results=...), a new results HDF5 group, read_calib_attempt_results), so **File ▸ Open Project…**'s Populate step shows them instantly with no recompute and no need for the original raw image to still be on disk — required deferring the project-log call (_pending_log_result/_flush_pending_log) until the asynchronous post-Fit integration actually finishes, in both tab_calibrate.py (single-detector) and hydra_calib_page.py (per-panel); (2) a mask logged to a Project attempt is now embedded only when it includes something hand-drawn/computed in Mask Builder (MaskSelector.has_live_mask_source) — a mask assembled purely from file/folder sources is referenced by path+hash instead, like the raw calibrant image already was, threaded through as a mask_is_file_backed flag from tab_calibrate.py/hydra_calib_page.py/tab_batch.py/ hydra_batch_page.py; a new mask_present attr separates "was any mask applied" from mask_embedded's "is the array actually stored" so neither existing field's meaning changed; (3) Hydra Calibrate's **Overall** Radial Profile control changes from a checkbox to a green-when-active toggle button (HydraProfileViewer's new composite_as_button mode) that hides GE1-4's curves and shows their already-verified NaN-aware summed profile; the **Eta vs R Cake** tab gains matching GE1-4 checkboxes plus an Overall button that computes and displays the summed cake (_compose_overall_cake, verified correct via synthetic-data tests earlier in this session — no logic change, only wiring the existing function to a UI); a finished live Hydra run with Overall active also logs the summed profile as its own lightweight hydra_composite Project attempt (dialogs.py label "Hydra Overall"; app.py's discover_panels/Populate flow explicitly skips this pseudo-panel — no tab widget to restore an Overall result into, so it's visible only via **File ▸ Project History…**); (4) the Data Viewer's single-detector mode gains an **Eta vs R Cake** tab next to Radial Profile, reusing the same integrate_frame(..., return_cake=True) path Hydra Calibrate already used; (5) a saved Workspace and a newly-created Project both record the active beamline Profile, restored automatically on load/Open Project (app.py: _restore_active_profile, project.py: project_active_profile/active_profile_at_creation attr) through the same settings.set_active_profile/C.reload_from_config/ on_profile_changed path the header combo uses, if the recorded Profile still exists locally and differs from the one currently active; (6) Eta vs R Cake plots' right-click-drag now zooms only the axis actually dragged (horizontal → R, vertical → η, diagonal → both), matching Radial Profile's gesture, instead of always zooming η only — root cause was pg.ImageView.__init__ force-locking aspectLocked=True on any view it's given, which recouples X/Y on every right-drag regardless of a custom mouseDragEvent's math; fix is vb.setAspectLocked(False) (R-px and η-degrees have no physical aspect to preserve), which let the now-obsolete custom _YZoomViewBox class be removed entirely in favor of stock pyqtgraph behavior. All of (1)-(6) are additive to project.py's existing HDF5 schema — no field renamed or removed, so older project files keep reading exactly as before. Verified via synthetic-data tests and the existing test suite; not run against a live GUI session by a human this session. Found in this session: unlocking CakeViewer's ViewBox makes tests/test_hydra_calib_ui.py/tests/test_hydra_ui.py segfault on teardown noticeably more often than before — bisected extensively and confirmed this is the same pre-existing pyqtgraph/PyQt offscreen-teardown heisenbug already documented in .context/STATE.md (any change to that ViewBox object shifts crash probability, not a logic bug in this fix), not attempted to fix further. **Files:** midas_gui/project.py (append_calibration_attempt, append_integration_attempt, read_calib_attempt_results, project_active_profile, create_project, discover_panels, _PANEL_ORDER), midas_gui/tab_calibrate.py (_log_to_project, _pending_log_result/_flush_pending_log, _display_stored_result, _run_integration), midas_gui/hydra_calib_page.py (_compose_overall_cake wiring, _on_cake_panel_toggled/ _on_cake_overall_toggled, _maybe_log_composite_attempt, _refresh_composite_curve, _pending_log_results/_flush_pending_log), midas_gui/hydra_widgets.py

(`HydraProfileViewer.composite_as_button`), `midas_gui/widgets.py` (`CakeViewer` — removed `_YZoomViewBox`, `MaskSelector.has_live_mask_source`), `midas_gui/tab_view.py`, `midas_gui/hydra_geometry_card.py` (Data Viewer Cake tab), `midas_gui/ tab_batch.py`, `midas_gui/hydra_batch_page.py` (mask-embed flag, active-Profile logging), `midas_gui/app.py` (`_restore_active_profile`), `midas_gui/dialogs.py` ("Hydra Overall" panel label), `documentation/gui_documentation.md` (§5, §15-§17). **Roll back:** `git revert 943a91d`. Self-contained back to `6b1564b` (reverting drops the new `results`/mask-embed-flag/active-Profile project fields, the Overall Cake UI, the Data Viewer Cake tab, and the cake-zoom fix together — see `.context/DECISIONS.md` if only one piece needs undoing).

`5cf2e8c` — Fix app-wide error-dialog/log truncation that could hide the real exception (2026-08-27)

Effect: a Windows user's "Calibration failed" dialog showed only a stray `F` — `tab_calibrate.py`'s `_on_fail` did `QMessageBox.critical(self, "Calibration failed", msg[:400])`, and 400 characters happened to cut the traceback mid-word. The same `msg[:N]`/`traceback.format_exc()[:N]` truncation pattern ($N \in \{200,300,400,500,600\}$) was copy-pasted across ~15 files' worker-failure handlers, silently discarding whatever text fell past the cutoff on every tab, not just Calibrate. Added `dialogs.show_error( parent, title, full_text, log=None, log_prefix="")`: a one-line summary with Qt's native scrollable "Show Details…" panel holding the complete text, plus an optional full-text append to a tab's log widget/callback. Every truncated critical-dialog call was replaced with it — paired "log the cut text" + "show the cut dialog" call sites collapsed into one untruncated `show_error(...)` call; log-only sites with no paired dialog just had the `[:N]` slice dropped. `str(e)`-only dialogs (never truncated to begin with) were left untouched. One unrelated single-line status-label truncation (`widgets.py`'s `FieldAverageWidget`, no dialog/log involved) was found but deliberately left alone as a different UI affordance. **Files:** `midas_gui/dialogs.py` (new `show_error`), `midas_gui/ tab_calibrate.py`, `tab_refine.py`, `tab_pdf.py`, `tab_corrections.py`, `tab_batch.py`, `tab_pumpprobe.py`, `tab_texture.py`, `tab_view.py`, `tab_export.py`, `tab_mask.py`, `widgets.py`, `hydra_calib_widgets.py`, `hydra_geometry_card.py`, `hydra_batch_page.py`, `hydra_calib_page.py`, `documentation/gui_documentation.md` (§13). **Roll back:** `git revert 5cf2e8c`. Self-contained; no dependents. Note this only restores the truncation bug — it does not touch the still-open Windows `midas_calibrate_v2` import failure that prompted the fix (see `.context/STATE.md`/`DECISIONS.md`).

`08fe8f6` — Add README troubleshooting section for corrupted-cache import errors; gitignore local test_data/projects/ (2026-08-27)

Effect: documents a self-diagnosis fix for "Module could not be found" / DLL load / missing-submodule errors (e.g. `torch`'s `fbgemm.dll`, `ModuleNotFoundError: No module named 'mpmath.libmp'`) — these usually mean the package was installed from a corrupted/incomplete pip cache entry, not a real incompatibility. The README's new Troubleshooting section walks through `pip uninstall`/`pip install --no-cache-dir`/standalone `import` to isolate the problem from midas-gui before assuming a GUI bug. Also adds `/test_data/projects/` to `.gitignore` — a 66 MB local-only set of self-contained project `.h5`/workspace-JSON files used for manual testing, matching the existing pattern for `s17bm/`, `trr_s25ide/`, etc. (large local test datasets kept off the shared repo, not deleted). **Files:** `README.md`, `.gitignore`. **Roll back:** `git revert 08fe8f6`. Self-contained; no dependents.

`ccce056` — Calibrate: fix Flip Z ignored when Multi-panel detector is checked (2026-08-27)

Effect: With a Flip Z transform active, checking "Multi-panel detector" produced a beam centre still in the unflipped frame. Root cause: `calib.py`'s manual `im_trans` pre-flip workaround (needed because `autocalibrate_four_stage/_bayesian/_joint/ pipelines.single.autocalibrate` have no native `im_trans`

param) computed the auto-seed from the raw/untransformed image but ran the solve on the manually-flipped image, in all four affected `run_pipeline()` branches (one_shot+panel_layout, one_shot partial-distortion-refinement, four_stage, bayesian/joint) — seed and solve ran in two different frames, so the gradient-based refinement converged near the seed's original (wrong) position. `dark` was also passed untransformed to the solver whenever `image` had been flipped. Added `_prep_transformed(image, dark, im_trans)`, applying the transform to image+dark together once before seeding; all four branches now consume its output for both seed and solve. The `first_time` pipeline branch (ignores `im_trans` entirely — a separate, pre-existing gap) was explicitly left unfixed, per user agreement. **Verified:** the real bug-report image was too sparse for the installed auto-seeder (`make_seed()` raises `RuntimeError: no arcs detected`); verified instead with a synthetic ring image built around a known off-centre beam centre — confirmed `make_seed_safe()` returns a different BC depending on which frame (raw vs. flipped) it's given, then ran the fixed `run_pipeline("four_stage", ..., {"im_trans": [2]})` end-to-end and confirmed the recovered `BC_z` matches the flipped-frame truth, not the raw one. **Files:** `midas_gui/calib.py`. **Roll back:** `git revert ccce056`. Self-contained; no dependents. Restores the seed/solve frame mismatch for Flip-Z + Multi-panel-detector calibration (and the untransformed-dark bug in the same four branches).

`101558a` — Calibrate: feed Multi-panel detector results to downstream integration (2026-08-27)

Effect: Multi-panel calibration refined per-panel shifts, but the results never reached downstream integration in the format `midas_integrate_v2` needs. Three GUI-side gaps, all upstream of any package bug (panel numbering already matches between `PanelLayout.regular` and the v1 `DetectorMapper` convention): (1) `helpers._build_spec` (used by Calibrate's own Results-tab preview *and* Batch Integrate's "Use Tab 2 calibration") never patched panel fields onto the `IntegrationSpec` it builds — same pre-existing gap `TransOpt` already had and was already worked around for; (2) `geometry_fields_from_file` (the "Load calibration file" reader shared by Batch/Export/PDF/ Corrections/Hydra) never parsed `NPanelsY/NPanelsZ/PanelSizeY/ PanelSizeZ/PanelGapsY/PanelGapsZ/PanelShiftsFile` out of a paramstest, nor `panel_layout/panel_shifts_path` out of a calibration.json; (3) `tab_calibrate.py`'s "Save paramstest.txt" already wrote `panel_shifts.txt` + `PanelShiftsFile`, but never the panel grid keys — so even a correct shifts file pointed at `NPanelsY=0` downstream. `calib.py` now attaches `result.panel_layout/result.panel_shifts_path` (plain, JSON-safe attributes) the moment a Multi-panel run finishes (`_attach_panel_result`, called from `normalize_result`'s four_stage/ bayesian/joint branches); a shared `helpers._apply_panel_fields()` patches any `IntegrationSpec` built from a result or a loaded geometry file (`_build_spec`, `_spec_from_result_ns`); `_save_paramstest` writes the missing grid keys alongside the existing `PanelShiftsFile`. Only the single-detector Calibrate tab's `panel_layout` is affected — Hydra mode's 4 independent-detector "panels" are a different concept, untouched. **Verified:** built a synthetic `panel_u/panel_layout` (the real four-stage pipeline needs a physically convergent ring image, out of scope to fabricate here) and confirmed the full round-trip: `_attach_panel_result` writes a real, non-empty `panel_shifts.txt`; `_build_spec` on that result produces a spec with correct `NPanelsY/NPanelsZ/PanelSizeY/PanelSizeZ/ PanelGapsY/PanelGapsZ/PanelShiftsFile`; a standalone paramstest.txt saved with the panel keys, then reloaded via `geometry_fields_from_file` + `spec_from_geometry_file`, round-trips every panel field (including scalar-gap → per-gap-list expansion both ways). Ran `test_im_trans.py`, `test_hydra_geometry.py`, `test_hydra_chirality.py` (all pass) and `test_smoke.py::test_app_builds_offscreen` (passes); `test_hydra_calib_ui.py` hit the pre-existing, already-tracked pyqtgraph interpreter-teardown segfault after its one test passed — confirmed unrelated (see `.context/STATE.md`). **Files:** `midas_gui/calib.py`, `midas_gui/helpers.py`, `midas_gui/tab_calibrate.py`, `midas_gui/workers.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 101558a`. Self-contained; no dependents. Restores the pre-existing behavior where Multi-panel results never reach the Results-tab preview, Batch Integrate, or a fully-usable exported paramstest.txt.

`ac13797` — Data Loader: Browse… popup (multi-file/folder/name-stem) + Hydra gains real cross-tab Import from… (2026-08-27)

Effect: Every Data/Dark/Bright/Background field's ⋯ button — single- detector and Hydra alike — now opens a two-item menu, **Browse…** and **Import from…**, in place of the old flat File…/Folder… pair. New `dialogs.BrowseFilesDialog` offers up to 4 modes (radio buttons), passed in per-caller since three different consuming pipelines can't take every shape: **Single file**, **Multiple files** (arbitrary multi-select, not offered for Hydra fields or Batch Integrate's streamed Data field), **Full folder**, **Files sharing a name stem** (type-or-click-to-prefill a prefix; not offered for Hydra's main Data field, whose frame index comes from one anchor file's own internal frame count). HDF5 files are excluded from every mode but Single file. Separately, Hydra's Data Viewer/ Calibrate/Batch Integrate pages now bind their `HydraLoaderPanel` into the same `data_bridge.DataSourceRegistry` their single-detector counterparts already used (new `bind_hydra_registry()` on each tab, wired from `app.py`), so Hydra fields gain a real "Import from…" for the first time — labeled distinctly ("Data Viewer (Hydra)" etc.) so a Hydra anchor path is never confused with its single-detector counterpart. A confirmed Multiple- files/stem pick has no single string/glob form, so it's carried as a `list[str]` through `helpers.source_kind`/`_collect_frame_paths` and new `_explicit_paths` state on `FieldSelector`/`DataLoaderPanel` (save/restore included), alongside the existing plain-path-text case. **Verified:** per-file isolated `pytest tests/test_smoke.py::<each test>` all pass; full-file `test_live_stream.py` and `test_hydra_geometry.py` pass. `test_hydra_ui.py`/`test_project.py` each hit the pre-existing interpreter-teardown segfault/abort at process exit after every test in the file already passed — confirmed unrelated (module-scoped MainWindow + pyqtgraph ViewBox teardown, already tracked in `.context/STATE.md`). **Files:** `midas_gui/dialogs.py`, `midas_gui/widgets.py`, `midas_gui/hydra_widgets.py`, `midas_gui/helpers.py`, `midas_gui/app.py`, `midas_gui/tab_view.py`, `midas_gui/tab_calibrate.py`, `midas_gui/tab_batch.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert ac13797`. Self-contained; no dependents. Restores the old flat File…/Folder… menu and leaves Hydra fields without cross-tab "Import from…".

`a54f796` — Browse popup: show folder/file path (not a count) after multi-file picks; default to project root; wider name column (2026-08-27)

Effect: A Multiple-files/Filestem Browse… pick showed a bare "N files selected" count instead of a real path. New `helpers.display_text_for_paths()` shows the one file's own path if exactly one matched, else the shared parent folder of every matched file (`os.path.commonpath` if they're not all in the same directory). Used by `FieldSelector`/`DataLoaderPanel._set_explicit_paths` (Data/Dark/Bright/ Background, single-detector) and by Mask Builder's pre-existing Stack-field "Files (multi-select)…" picker (`tab_mask.py` — a separate, older, native-`QFileDialog` picker with the same "N files selected" pattern, predating the Browse… popup and unrelated to `ac13797`). Separately, `BrowseFilesDialog` now opens to new `constants.PROJECT_ROOT` (the `midas-gui` repo root, same derivation `_TEST_DATA` already used) instead of the user's home directory when no `start_dir` is given, and its file/ folder name column is now double Qt's stock 100px default. **Verified:** offscreen script confirmed `PROJECT_ROOT` resolves to the repo root, a fresh dialog's starting directory matches it, and `columnWidth(0)` reads 200 (was 100); `display_text_for_paths` spot-checked for 1 file / N files in one directory / N files across directories. Per-file isolated `test_smoke.py` (all 10 tests) and `test_live_stream.py` green. **Files:** `midas_gui/constants.py`, `midas_gui/dialogs.py`, `midas_gui/helpers.py`, `midas_gui/widgets.py`, `midas_gui/tab_mask.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert a54f796`. Self-contained; no dependents. Restores the "N files selected" count text, `Path.home()` as the popup's default folder, and the original 100px name column.

`ae3b665` — Project: merge Workspace and FAIR Project into one `.h5` file (2026-08-28)

Effect: Unified the two previously-independent persistence mechanisms — a JSON "Workspace" (`Ctrl+S`/`Ctrl+Shift+S`/`Ctrl+O`, every tab's live fields) and an HDF5 "Project" (append-only Calibrate/Batch-Integrate provenance) — into one `.h5`. `project.py` gained `write_workspace()`/ `read_workspace()`: a single

mutable `/workspace` slot (JSON state + optional sidecars), overwritten each save, alongside the existing append-only `attempt_NNNN` history (`SCHEMA_VERSION` bumped 1→2; old v1 files still open fine, `read_workspace` returns `({}, {})` when there's no `/workspace` group yet). `app.py`'s File menu collapsed to Save Project (`Ctrl+S`)/Save Project As…(`Ctrl+Shift+S`)/Open Project…(`Ctrl+O`) — `New Project…` is gone (Save-As to a new filename creates one); `Close Project` and `closeEvent` now prompt to save first if the session is dirty, since `Ctrl+S` now targets the same file. `save_project`/`_apply_workspace_state` replace `save_gui_state`/`load_gui_state`, harvesting the Mask-Builder/ Calibrate sidecar files (`get_state(sidecar_stem=...)`, unchanged) through a scratch `tempfile.TemporaryDirectory()` instead of leaving them next to a JSON file — no changes needed in `tab_mask.py`/`tab_calibrate.py`'s `get_state`/`set_state`. A `File ▸ Import Legacy Workspace (.json)…` action reads old standalone Workspace JSON files for backward compatibility. Per explicit decision, `append_calibration_attempt`/`append_integration_ attempt` dropped their `dark`/`bright`/`background` embedding entirely (already always file-backed — path+hash in `loader_state`/`inputs` already covers provenance); a live/drawn-in-tab mask with no file of its own remains the one embedded exception (had no `mask_is_file_backed` alternative). Fixed 4 call sites across `tab_calibrate.py`, `tab_batch.py`, `hydra_calib_page.py`, `hydra_batch_page.py` (plus removed now-dead `_last_fields`/`dark`/`bright`/`background` plumbing in the two Hydra pages). **Verified:** `tests/test_project.py` (20, incl. 2 new `write_workspace`/ `read_workspace` round-trip tests) and `tests/test_workspace_ux.py` (9, updated for the new API) pass per-file (the known pyqtgraph interpreter- teardown crash fires after all tests pass, unrelated). An offscreen end-to-end script confirmed: `Ctrl+S` with no project open → Save-As creates a fresh `.h5`; a second save overwrites `/workspace` in place leaving `attempt_NNNN` groups untouched; a logged calibration attempt has no `dark`/`bright`/`background` datasets; a fresh `MainWindow` opening that project restores an edited field exactly. **Files:** `midas_gui/app.py`, `midas_gui/project.py`, `midas_gui/tab_batch.py`, `midas_gui/tab_calibrate.py`, `midas_gui/hydra_batch_page.py`, `midas_gui/hydra_calib_page.py`, `tests/test_project.py`, `tests/test_workspace_ux.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert ae3b665`. Restores the separate JSON Workspace (`save_gui_state`/`load_gui_state`) and the old `New Project…` menu action; downstream `append_calibration_attempt`/`append_integration_attempt` callers would need their `dark`/`bright`/`background` args restored too if reverted in isolation.

`af8066f` — Batch Integrate: Browse… parity — Multiple files + filestem-filtered sources (2026-08-28)

Effect: Batch Integrate's streamed Data field previously only offered Single file/Full folder in its Browse… popup — Multiple files/Filestem were withheld because the streaming reader (`TIFFGlobSource`/`HDF5FrameSource`) could only consume a glob path, not an arbitrary file list. 1. `DataLoaderPanel._open_browse_dialog()` now always offers all four modes. A **Filestem** pick in stream mode is kept as a live `(folder, prefix)` filter (`_set_stem_filter`) rather than resolved to a frozen list: `_raw_source()` substitutes it into a `<folder>/<prefix>*` glob, so `source_cfg()`, `_load()`, and cross-tab "Import from…" all stay filestem-aware for free, and `FolderMonitorWorker` re-globs it on every poll like any other glob path — a new matching file dropped in later is picked up, a non-matching one is ignored. 2. A **Multiple files** pick becomes a new `"tiff_list"` source type; `workers._ExplicitTIFFSource` iterates the resolved paths in order via `helpers._load_image` (covers `.ge*` frames too). Not watchable by MONITOR (no single glob describes an arbitrary list) — `_start_monitor`'s existing `type != "tiff_glob"` guard already rejects it, with a clearer warning message naming filestem/folder as the watchable options. 3. `_raw_source`/`get_state`/`set_state` round-trip the stem filter alongside the pre-existing explicit-paths handling; manually editing the path field clears either one (`_on_path_changed`), matching the existing explicit-paths behavior. **Verified:** new `tests/test_batch_data_source.py` (8 tests, dependency-free — PyQt5/numpy/tifffile only): stem-filter→glob and explicit-list→tiff_list `source_cfg()` shapes, manual-edit clearing, info-label text, get/set_state round-trip, `_ExplicitTIFFSource` frame order/content, and `BatchWorker. _open_source` dispatch for `"tiff_list"`. All pass. **Files:** `midas_gui/widgets.py`, `midas_gui/workers.py`, `midas_gui/tab_batch.py`, `midas_gui/dialogs.py`, `tests/test_batch_data_source.py` (new), `documentation/gui_documentation.md`. **Roll back:** `git revert af8066f`. Self-contained — restores the

Single-file/Full-folder-only Browse… restriction for Batch Integrate's Data field; no later commit depends on the `"tiff_list"` source type.

`a27790a` — Batch Integrate: cosmetic overhaul + Batch-Parallel workers, single-detector + Hydra (2026-08-28)

Effect: Five requested cosmetic/functional changes to the Batch Integrate tab, applied to both the single-detector tab and Hydra's per-panel page: 1. **Drift correction hidden from the GUI** (single-detector only — not used in production) — `drift.setVisible(False)`; `DriftWorker`/ `_fit_drift`/state save-restore all stay fully wired, re-enabled by removing one line. 2. **"Calibration values" is now a popup**, not an always-visible grid — new `helpers.make_calib_values_button()` reuses the existing `_clickable_menu_label` pattern (`make_pixel_label`/`make_kedge_label`) to pop a menu containing the same `render_calib_value_grid` output, rebuilt fresh on every open. Applied to `tab_batch.py`'s single card and each of `hydra_batch_widgets.HydraBatchPanelCard`'s 4 per-panel cards. 3. **Output format is a checkbox list** — new `widgets.OutputFormatSelector` (one `QCheckBox` per `constants.OUTPUT_FORMATS` entry, `.checked_keys()`/`.get_state()`/ `.set_state()`) replaces the single-select combo everywhere `_fmt` was used (`BatchWorker`/`FolderMonitorWorker` gained `fmts: list[str]` replacing `fmt: str`). 4. **Green Save button** (new `workers.write_all_profiles()`) writes the lineouts already computed this run to disk on demand, independent of whether an Output folder was set before running — the pre-existing "no output dir → nothing saved during the run" behavior is unchanged; Save is a separate, always-available write-what's-in-memory action. `BatchWorker.finished` now also carries `"sigmas"` (previously computed but never emitted — `project.append_integration_attempt` already expected this key and silently dropped it, a pre-existing gap this closes too). Start Integration is narrower (equal-thirds row with Abort/Save instead of taking most of the row); Abort is now red, Save green (`style.DANGER_BTN_QSS`/`SUCCESS_BTN_QSS`). 5. **Sequential / Batch Parallel run mode** — new `workers. BatchRunCoordinator` (same signal surface as `BatchWorker`: `progress`/ `frame_done`/`finished`/`failed`/`log_line`/`geom_ready`, plus `isRunning`/`start`/`requestInterruption`/`wait`, so existing call sites barely change). "Sequential" constructs one plain `BatchWorker` (identical to before, zero overhead). "Batch Parallel" resolves the frame_range into an explicit index list, computes `resolve_worker_count()` (auto-shrinks the requested worker count so each worker gets >=10 frames — `MIN_FRAMES_PER_WORKER`), splits into contiguous chunks (`_split_into_chunks`), builds the detector map once via a new one-shot `_GeomBuildWorker`, then fans out one `BatchWorker` per chunk (new `frame_indices` param — reads via `source.get(i)` for exact random access, since the old frame_range skip-by-continue loop would otherwise decode every frame up to a late chunk's start, defeating the point of parallelism) sharing that one context; merges their `finished` payloads in chunk order and writes one combined HDF5 if `"h5"` is selected (instead of each chunk writing its own colliding `integrated.h5`). Threads, not OS processes — numpy/torch release the GIL during their heavy compute, matching the existing precedent of Hydra's own per-panel Sequential/Parallel toggle (multiple concurrent `BatchWorker` QThreads in one process). Hydra gets this as a **second, independent** level of parallelism (`HydraBatchPage`'s new shared "Per panel:" control) layered under its existing "Panels:" Sequential/Parallel toggle — a Parallel panel run with Batch Parallel selected can have up to (panels × workers) concurrent threads. **Verified:** `tests/test_batch_data_source.py` extended to 17 tests (frame-index resolution, chunk-splitting, `resolve_worker_count`'s auto-shrink math, `write_all_profiles` writing every frame/format and skipping `2d_csv`, `_ExplicitTIFFSource.get()` random access) — all pass. `tests/test_hydra_batch_ui.py` updated for the `BatchWorker`→ `BatchRunCoordinator` swap (its fake gained `run_mode`/`n_workers` params and a `MIN_FRAMES_PER_WORKER` class attr) and the removed `_calib_val_note` widget (assertions now read `_calib_fields_in_use()`'s note directly) — both tests pass. `tests/test_project.py`'s `integrate_attempt_gui_fields` test updated for `fmt` becoming a list (`fields["fmt_keys"]` instead of `fields["fmt"]`; legacy single-string values still wrap into a 1-element list for old project files) — passes. Full single-detector and Hydra Batch tabs build and round-trip save/restore state correctly in an offscreen smoke check. **Files:** `midas_gui/workers.py`, `midas_gui/widgets.py`, `midas_gui/helpers.py`, `midas_gui/style.py`, `midas_gui/tab_batch.py`, `midas_gui/hydra_batch_widgets.py`, `midas_gui/hydra_batch_page.py`, `midas_gui/project.py`, `tests/test_batch_data_source.py`, `tests/test_hydra_batch_ui.py`, `tests/test_project.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert a27790a`. Self-contained — restores the

always-visible Drift/Calibration-values cards, the single-select format combo, the plain Start/Abort row with no Save button, and single-worker `BatchWorker`-only runs; no later commit depends on `BatchRunCoordinator`/`OutputFormatSelector`/`write_all_profiles`.

`e6f2e50` — Batch Integrate: Output format + Run mode as popups, fix calib-popup rebuild (2026-08-28)

Effect: Same-day follow-up to `a27790a`, tightening three of its pieces: 1. **`widgets.OutputFormatSelector`** — the checkbox list no longer sits permanently in the Output card; the checkboxes now live inside a `QMenu` behind a clickable "Output format ▾" `QToolButton` (same click-to-see-options interaction as `helpers.make_calib_values_button`). The button's own text is kept in sync with the checked set (e.g. "Output format: CSV, XYE ▾") via a new `_sync_button_text()` called from each checkbox's `toggled` signal and from `set_state()`. 2. **Run mode explanation → tooltip.** The permanent explanatory `QLabel` under the Sequential/Batch-Parallel combo (single-detector "Mode:" row in `tab_batch.py`, Hydra "Per panel:" row in `hydra_batch_page.py`) is removed; the same text is now a `setToolTip()` on both the row label and the combo box, freeing up card vertical space. 3. **Fix `helpers.make_calib_values_button` popup staleness** — the `QMenu`'s widget tree (`QVBoxLayout`/grid/note label/`QWidgetAction`) was previously built once at construction and only its *contents* refreshed in `_populate()`; a `QMenu` computes its popup size from the `QWidgetAction`'s `sizeHint()` at show time, so the very first open (before any calibration fields exist) could cache a near-zero size that then stuck on later opens even after fields were populated. `_populate()` now calls `menu.clear()` and rebuilds the whole host widget/layout/action fresh on every open, guaranteeing the size hint matches the content about to be shown. **Verified:** manual reasoning + existing `tests/test_hydra_batch_ui.py`/`tests/test_batch_data_source.py` suites (no behavior they assert on changed — `OutputFormatSelector.checked_keys()`/`get_state()`/`set_state()` and the calib-popup's field content are unchanged, only presentation). `gui_documentation.md` updated (Batch Integrate summary + §7 body) + PDF rebuilt. **Files:** `midas_gui/widgets.py`, `midas_gui/tab_batch.py`, `midas_gui/hydra_batch_page.py`, `midas_gui/helpers.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert e6f2e50`. Self-contained — restores the always-visible checkbox list, the permanent Run-mode note label, and the mutate-in-place calibration popup from `a27790a`; no later commit depends on the button/tooltip presentation.

`c67ad1b` — Calibrate: multi-panel pipelines actually refine panel shifts, persist across save/reload (2026-08-29)

Effect: Root-cause fix + persistence follow-through for Multi-panel detector calibration: 1. **Root cause:** every Multi-panel-detector pipeline (one-shot, first-time, four-stage, bayesian, joint) passed `panel_layout` through to `midas_calibrate_v2` for the fixed forward geometry only — no per-panel δy/δz/δθ was ever registered as something to refine, so a Fit run produced no real panel correction regardless of how it was later exported. New `calib._panel_spec()` builds a `CalibrationSpec` via `spec_from_v1_params()` + `add_panel_parameters()` (tolerances matching `midas_calibrate_v2.calibrate()`'s own `panel_mode="shift"` defaults) and every pipeline branch in `run_pipeline()` now passes it as `spec=`, so panel shifts are real and nonzero for the first time. `normalize_result()`'s one-shot branch also gained the missing `_attach_panel_result()` call (it built the result but never attached panel data, unlike the other three modes). 2. **Save calibration.json / Save paramstest.txt** (`tab_calibrate.py`) now rewrite a co-located `<name>_panelshifts.txt` sidecar at save time from the result's `_panel_unpacked` tensors, instead of trusting whatever `panel_shifts_path` was live when Fit finished — which, with no Output folder set, is an anonymous tempfile with no guarantee of surviving. `calib._attach_panel_result()` now also logs to the tab's Log panel when it falls back to that tempfile, so the gap is visible immediately rather than discovered later. Paramstest's sidecar name changed from a shared `panel_shifts.txt` to `<stem>_panelshifts.txt` so multiple saves into one folder don't overwrite each other's data. 3. **`helpers.geometry_fields_from_file`** now retries a stale/missing `panel_shifts_path` next to the geometry file itself before giving up, so a `.json`/paramstest copied or moved together with its sidecar still resolves it on reload. 4. **Project (`.h5`) persistence** (`project.py`): calibration attempts now embed the raw panel-shifts array (`_panel_shifts_array()`, `append_calibration_attempt()` writing an `att["panel_shifts"]`

dataset + `panel_shifts_embedded` metadata flag). Reopening a project and populating Tab 2 from such an attempt (`app.py`'s calibration population path) calls the new `materialize_panel_shifts()` to write a real `<attempt>_panelshifts.txt` into a `<project>_panel_shifts/` folder beside the project file and points the restored result at it — so the panel correction survives even if the original run's file (or the whole project) has moved to another machine. **Verified:** `tests/test_project.py` gained `test_calibration_attempt_embeds_and_materializes_panel_shifts` + `test_read_calib_attempt_panel_shifts_none_when_absent`; new `tests/test_panel_refinement.py` (spec-building/pipeline wiring), `tests/test_panel_shifts.py`, and `tests/test_calibrate_panel_save.py` (Qt-backed, exercises `CalibrationTab._save_json`'s sidecar rewrite). All pass per-file isolated (the pre-existing `midas_gui/workers.py` `build_geom` interpreter-teardown crash — confirmed present on HEAD before this commit too, unrelated — still fires after all tests pass in a combined run; see "Open questions" in `.context/STATE.md`). `gui_documentation.md` §5 "Export" and §16 rewritten + PDF rebuilt. **Files:** `midas_gui/calib.py`, `midas_gui/tab_calibrate.py`, `midas_gui/helpers.py`, `midas_gui/project.py`, `midas_gui/app.py`, `tests/test_project.py`, `tests/test_calibrate_panel_save.py`, `tests/test_panel_refinement.py`, `tests/test_panel_shifts.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert c67ad1b`. Self-contained — later commits don't depend on real panel-shift refinement; reverting restores the fixed-geometry-only (no refinement) behavior and the old tempfile-trusting save/reload paths.

`21faaf8` — Project: redesign .h5 schema into gui_workspace (per-tab) + analysis (mask/calibrate/integrate), unified Open Project dialog (2026-08-29)

Effect: Requested redesign of the project `.h5` schema plus a richer Open Project experience — a **clean-cutover breaking change**, `schema_version` 2 → 3, no backward compatibility with older project files: 1. **`/workspace` (single mutable JSON blob for all 10 tabs) → `/gui_workspace/<tab name>/{state, sidecars/}`**, one independently readable/restorable group per tab. `project.write_gui_workspace()` replaces `write_workspace()`; new `list_workspace_tabs()`/ `read_workspace_tab()`/`read_workspace_meta()` replace `read_workspace()`. `app.py`'s `_serialize_workspace()`/`_apply_workspace_state()` now give each tab its own sidecar subdirectory (was one shared stem across every tab, distinguished only by filename-suffix convention). 2. **`/<panel_key>/{calib,integrate}/attempt_NNNN` → `/analysis/{calibrate,integrate}/<panel_key>/attempt_NNNN`.** `append_calibration_attempt`/`append_integration_attempt`/ `discover_panels`/`list_attempts` updated for the new path prefix — no signature changes, confirmed against all 4 existing call sites (`tab_calibrate.py`, `tab_batch.py`, `hydra_calib_page.py` ×2, `hydra_batch_page.py`). Every read-side function keyed by an opaque `ref` string (`read_attempt`, `read_attempt_results`, `read_calib_attempt_results`, `read_calib_attempt_panel_shifts`, `materialize_panel_shifts`) needed zero changes. 3. **New `/analysis/mask`** — a *global* (not per-panel; Mask Builder is one shared tab, unlike Calibrate/Integrate's single-detector-vs-Hydra-panel split) FAIR-provenance history: `append_mask_attempt()`/ `list_mask_attempts()`/`read_mask_attempt_array()`. `tab_mask.py` gains a **"Log to Project"** button (next to Save, disabled until a mask exists) and `apply_project_mask()` (restores threshold/statistical/spike/cosmic/ azimuthal/learnable field settings, reloads the source image, and sets the mask directly from the recorded array — already fully processed, so it bypasses `_set_mask()`'s dilation step to avoid double-dilating). Unlike Calibrate/Batch-Integrate there's no single "run finished" moment to auto-log from (a mask can come from Compute, Load, hand-drawn shapes, or any combination), so this is an explicit, click-when-ready action — confirmed with the user before implementing. 4. **Unified Open Project dialog** (`dialogs.py`) replaces the old two-stage flow (a blind Yes/No "restore the whole workspace or nothing" box, then a separate `ProjectLoadDialog` calibrate/integrate-only attempt picker, now removed): new `ProjectContentsPicker` (a `QWidget` that, given a project path, builds a checkbox tree of every `gui_workspace` tab and every mask/calibrate/integrate attempt present, everything checked by default, rebuilding its whole layout on every `set_project()` call rather than mutating in place) is wrapped by `ProjectOpenDialog` (adds a file-tree browser pane on the left, styled after `BrowseFilesDialog`'s navigation, refreshing the picker live as a `.h5` is clicked — for File ▸ Open Project…) and `ProjectSelectionDialog` (picker only, no browser — for Recent Projects

entries where the path is already known). A pre-3 or non-project `.h5` shows an inline warning naming its schema version instead of a silently empty tree. `app.py`'s `_open_project_dialog`/ `_open_project_path` collapse into one shared `_open_project_selection()` that restores exactly the checked items (calling the existing `apply_project_calibration`/`apply_project_integration`/new `apply_project_mask` per tab) — nothing restores until Open is clicked. `ProjectHistoryDialog` also lists mask attempts (`"Mask"` kind, panel column shows `"–"`). **Verified:** `tests/test_project.py`'s workspace tests rewritten for the per-tab API; every hardcoded old-schema path assertion updated there and in `tests/test_hydra_batch_ui.py`/`tests/test_hydra_calib_ui.py` (caught by a full per-file test run, not just the new schema's own unit tests); new tests for `append_mask_attempt`/`list_mask_attempts`/ `read_mask_attempt_array`/`apply_project_mask` and for `ProjectContentsPicker`/`ProjectSelectionDialog` (defaults-all-checked, unchecking a tab/panel excludes it from `selection()`, an invalid/old file reports no content). Manual offscreen smoke of the full `_open_project_selection` path (create project → log a mask attempt + save a workspace tab → reopen with a mixed selection → confirm only the checked items restore). All pass per-file isolated; the pre-existing pyqtgraph/ interpreter-teardown crash on a combined run (also independently confirmed non-deterministic on HEAD before this commit, via repeated runs) is unrelated. `gui_documentation.md` §4 and §16 rewritten + PDF rebuilt. **Files:** `midas_gui/project.py`, `midas_gui/app.py`, `midas_gui/dialogs.py`, `midas_gui/tab_mask.py`, `tests/test_project.py`, `tests/test_workspace_ux.py`, `tests/test_hydra_batch_ui.py`, `tests/test_hydra_calib_ui.py`, `documentation/gui_documentation.md`. **Roll back:** `git revert 21faaf8`. Breaking either direction — any project `.h5` saved under schema_version 3 (this commit) is not readable by the pre-redesign code the revert restores, and vice versa; there is no migration path between the two schemas.

`0332683` — Batch Integrate: fix frame ordering in Batch Parallel's live waterfall/stacked-profile view (2026-08-29)

Effect: In Batch Parallel mode, `BatchRunCoordinator` split the frame list into one contiguous chunk per worker thread and ran the chunks concurrently, but wired each chunk's `frame_done` signal straight through to its own `frame_done` — so whichever chunk finished a given frame first delivered it first. `WaterfallViewer.add_profile`/ `StackedProfileViewer.add_profile` (`widgets.py`) both append purely positionally (row = arrival order), so the live views showed frames in wall-clock completion order rather than frame order whenever a later chunk happened to finish ahead of an earlier one — Sequential mode was unaffected (one thread, in-order emission). The final merged result used by Save/HDF5 was already correctly ordered (`_on_chunk_finished` iterates chunk workers in dispatch order); only the incremental live display was wrong. Added `_on_chunk_frame`: tracks, per chunk worker, which absolute frame index its next `frame_done` corresponds to (`BatchWorker._iter_frames` processes a chunk's `frame_indices` strictly in order), buffers out-of-order arrivals, and re-emits `frame_done` only once frames become available in ascending frame-index order. Both `tab_batch.py` and `hydra_batch_page.py` consume the same `BatchRunCoordinator.frame_done` signal, so this single fix covers both the single-detector and Hydra per-panel batch pages. **Verified:** new unit test `test_batch_parallel_frame_done_reorders_out_of_completion_order` (simulates a fast chunk completing all its frames before a slower, earlier chunk reports anything; asserts `frame_done` still emits in ascending frame-index order). Ran `tests/test_batch_data_source.py` and `tests/test_hydra_batch_ui.py` per-file — all pass. **Files:** `midas_gui/workers.py`, `tests/test_batch_data_source.py`. **Roll back:** `git revert 0332683`. Self-contained; no dependents. Restores the wall-clock-order live display for Batch Parallel runs (final Save/HDF5 output was never affected).

Rollback recipes (common intents)

- **Undo the Pump Probe tab only:** `git revert 590b410` removes Pump Probe *and* modular tabs. To drop Pump Probe alone, hand-remove `midas_gui/tab_pumpprobe.py`, its import/construction/wiring in `app.py`, `PumpProbeWorker` in `workers.py`, and its entry in `constants.OPTIONAL_TABS/DEFAULT_VISIBLE_TABS`.

- **Undo modular tabs (show all 10 fixed again):** remove the `apply_tab_visibility` logic in `app.py` (add all tabs directly), the Tabs section in `prefs_dialog.py`, and the tab-list constants — see the `590b410` diff for the exact hunks.
- **Undo the config system:** `git revert d30be2c` — but first revert `590b410`'s config touches, since it extends the same `ui` overlay and Preferences dialog.
- **Revert the azimuthal-averaging change:** don't blanket-revert `41f28f5` (bundled with UI); instead set the `weighted` default back to the legacy η-bin mean in the batch/pump workers.
- **Go back to the pre-3-panel single-column tabs:** `aa9395e` is the pivot; a revert is large and cascades to later tabs — prefer fixing forward.

Generated from `git log`. To refresh after new commits: `git log --reverse --stat --date=short` and append entries above.